Ministry of Higher Education
and Scientific Research

ACADEMIC YEAR
2025/2026

EPI
Private International Higher
School of Polytechnic

Graduation Project

A Project Report on Software Engineering

TOPIC

Design and Development Virtual Laboratory Platform for Remote IOT Access

Company Name
NOVATION CITY

Achieved by
Abdelaziz Barhoumi

Supervised by
Mr. Ahmed ben Ayed
Mr. Mohamed Chrifa

سُبْحَانَكَ لاَ عِلْمَ لَنَا إِلاَّ مَا عَلَّمْتَنَا ۖ إِنَّكَ أَنْتَ الْعَلِيمُ الْحَكِيمُ

Dedications

In the name of Allah, the Most Gracious, the Most Merciful

To our beloved parents,

whose unwavering support, endless sacrifices, and constant prayers
have been the foundation of our success.

To our teachers and mentors,
who guided us with knowledge and wisdom
throughout our academic journey.

To the team at Novation City,
who welcomed us with open arms and provided
invaluable learning opportunities.

To our families and friends,
whose encouragement and understanding
made this achievement possible.

With deep gratitude and appreciation.

Acknowledgments

I would like to express my sincere gratitude to all those who contributed to the success of this professional internship experience, including academic supervisors, industry mentors, and institutional staff whose guidance, feedback, and practical support were essential to achieving the project objectives.

First, I extend my deepest appreciation to my academic supervisor, **Mr. Ahmed Ben Ayed**, for their invaluable guidance and continuous support throughout this project. Their expertise was instrumental in ensuring academic rigor and alignment with university requirements.

I am profoundly grateful to the Novation City team who welcomed me into their innovative ecosystem. Special thanks to my professional supervisor, **Mr. Mohamed Chrifa**, for their professional insights and technical guidance in Industry 4.0 implementation and digital transformation consulting.

My sincere appreciation goes to Novation City competitiveness cluster for entrusting me with developing the **IoT-REAP (Internet of Things - Remote Equipment Access Platform)**. Building this secure industrial remote operations platform integrating Proxmox VE virtualization and Apache Guacamole remote desktop has been an invaluable experience for my professional growth in full-stack development and industrial systems integration.

I extend gratitude to EPI Digital School's administration and faculty for providing quality education and academic resources essential for understanding enterprise-grade web application development, security architecture, and modern software engineering practices.

Finally, I am grateful to my family and friends for their unwavering support and encouragement throughout this journey.

Contents

List of Figures

List of Tables

List of Acronyms

Acronym	Definition	Acronym	Definition
AI	Artificial Intelligence	OAuth	Open Authorization
API	Application Programming Interface	ORM	Object-Relational Mapping
CI/CD	Continuous Integration/Continuous Delivery	OT	Operational Technology
CPU	Central Processing Unit	OWASP	Open Web Application Security Project
CSP	Content Security Policy	PDF	Portable Document Format
CRUD	Create, Read, Update, Delete	PHP	PHP Hypertext Preprocessor
CSS	Cascading Style Sheets	PHP-FPM	PHP FastCGI Process Manager
DNS	Domain Name System	PLC	Programmable Logic Controller
ESP32	Espressif 32-bit Microcontroller	PTZ	Pan-Tilt-Zoom
ESP32-CAM	ESP32 Camera Module	QoS	Quality of Service
FFmpeg	Fast Forward Moving Picture Experts Group	R&D	Research and Development
FIFO	First In, First Out	RDP	Remote Desktop Protocol
HLS	HTTP Live Streaming	REST	Representational State Transfer
HMI	Human Machine Interface	RTSP	Real-Time Streaming Protocol
HMAC	Hash-based Message Authentication Code	SCADA	Supervisory Control and Data Acquisition
HSTS	HTTP Strict Transport Security	SEO	Search Engine Optimization
HTTP	Hypertext Transfer Protocol	SHA	Secure Hash Algorithm
HTTPS	Hypertext Transfer Protocol Secure	SSH	Secure Shell
IEC	International Electrotechnical Commission	SSL	Secure Sockets Layer
IDMZ	Industrial Demilitarized Zone	SSO	Single Sign-On
IoT	Internet of Things	TLS	Transport Layer Security
IoT-REAP	Internet of Things - Remote Equipment Access Platform	TOTP	Time-based One-Time Password
IP	Internet Protocol	UI	User Interface
ISP	Internet Service Provider	ULID	Universally Unique Lexicographically Sortable Identifier
IT	Information Technology	UML	Unified Modeling Language
JS	JavaScript	USB	Universal Serial Bus
KPI	Key Performance Indicator	USB/IP	USB over Internet Protocol
LMS	Learning Management System	VM	Virtual Machine
MENA	Middle East and North Africa	VNC	Virtual Network Computing
MJPEG	Motion JPEG	VPN	Virtual Private Network
MQTT	Message Queuing Telemetry Transport	VPS	Virtual Private Server
MVC	Model-View-Controller	WebRTC	Web Real-Time Communication
NAT	Network Address Translation	XSS	Cross-Site Scripting
NPM	Node Package Manager	ZAP	Zed Attack Proxy

General Introduction

The Fourth Industrial Revolution, or Industry 4.0, has fundamentally transformed industrial operations by integrating artificial intelligence (AI), the Internet of Things (IoT), cloud computing, and advanced automation into manufacturing and operational processes. As industrial systems become increasingly digitized and interconnected, engineers and technicians require secure, remote access to specialized equipment - programmable logic controllers (PLCs), human-machine interfaces (HMIs), SCADA systems, and industrial robots - for training, development, and maintenance purposes. Traditional approaches to industrial remote access present significant limitations: VPN-based solutions are complex to configure and, in many Operational Technology (OT) deployment contexts, incompatible with OT security requirements; conventional remote desktop tools lack native integration with industrial communication protocols; and no standardized platform has emerged to consolidate virtual machine (VM) provisioning, browser-based remote desktop access, and IoT device control within a single, coherent system. As the comparative analysis presented in Chapter 1 will demonstrate, existing solutions address these challenges only partially. This raises a central engineering question: *how can industrial engineers and trainers securely access remote equipment from any browser, without client-side software installation, while preserving OT network integrity and enforcing strict access control?*

It is within this industrial and technological context that the present internship was conducted at **Novation City** in Sousse, Tunisia, from FEB 2026 to MAY 2026, as part of the final-year project requirements for the Software Engineering Bachelor at EPI Digital School. Novation City is a competitiveness cluster established in 2009 as a public-private partnership dedicated to fostering innovation and industrial transformation within the mechatronics and Industry 4.0 sectors [1]. Novation City operates through several specialized business units, among which **Novation Akademy** (industrial training and professional skills development) and **Innovation** (applied research and R&D initiatives) are the primary drivers of the operational requirement for a secure remote laboratory infrastructure. This convergence of training needs and R&D capability motivated the development of a dedicated platform to provide trainees and engineers with secure remote access to industrial equipment without the constraints of physical presence.

To address these challenges, Novation City initiated the **IoT-REAP** (**Internet of Things - Remote Equipment Access Platform**) project. The platform provides industrial engineers, technicians, and trainers with browser-based access to virtual machines through Proxmox VE integration [2], and clientless remote desktop sessions via Apache Guacamole, supporting the RDP, VNC, and SSH protocols [3]. Camera controls are dispatched and monitored via the lightweight MQTT messaging protocol [4], enabling real-time communication with connected industrial devices. IoT-REAP adheres to strong industrial cybersecurity principles,

implementing zero-trust access controls and enforcing OT network segmentation drawing upon the principles of the Purdue reference model. The platform requires no client-side software installation, thereby reducing the client-side attack surface and simplifying access provisioning for geographically distributed users.

The primary objective of this internship was therefore to design and implement the IoT-REAP platform, with specific focus on the development of the unified web portal, the integration of Proxmox VE and Apache Guacamole for virtualization and remote desktop orchestration, and the implementation of the secure MQTT-based IoT control interface. The deployment of underlying network infrastructure, the administration of the Proxmox hypervisor layer, and the development of proprietary hardware drivers lie outside the scope of this report and were addressed by Novation City's internal IT operations team as part of the broader infrastructure initiative.

The report comprises three chapters documenting the complete development lifecycle of IoT-REAP. **Chapter 1** establishes the professional context at Novation City, characterizes the challenges of secure industrial remote access, critically reviews existing industrial remote access solutions and their limitations, defines the platform requirements and scope, and presents the Agile Scrum methodology adopted, including the five-sprint development timeline. **Chapter 2** presents the conceptual study of the platform, modelling the system through a general use case diagram and actor-specific refinements covering all four user roles, a class diagram formalizing the domain model, and sequence diagrams capturing the dynamic behaviour of critical workflows including VM session provisioning, camera control, and user authentication. **Chapter 3** covers the realization and experimentation phase, describing the software architecture adopted for the platform - its layered organization, asynchronous provisioning pipeline, and external infrastructure boundary - presenting the hardware and software development environment, the principal interfaces implemented for each user role, the network topology and segmentation strategy, the production deployment and security configuration, and the verification activities confirming functional correctness, role-based access control, remote laboratory workflows, and performance. The report closes with a critical assessment of achievements and limitations encountered during development, and concrete recommendations for future platform enhancements.

Chapter 1

Preliminary Study

Introduction

This chapter establishes the foundations of the IoT-REAP (Internet of Things - Remote Equipment Access Platform) project by developing the argument that motivates its design and implementation. It begins by situating the work within its professional and academic context at Novation City, then identifies the operational problem driving the platform's development. A systematic review of existing industrial remote access solutions exposes the specific gaps that no current commercial product addresses. Building on this gap analysis, the proposed platform architecture, technology choices, and requirements specification are presented. The chapter concludes with the Agile Scrum methodology selected to manage the project and the sprint-based development timeline that structured its execution.

1 General Project Context

This section situates the IoT-REAP project within its professional and academic framework. It describes the host organization, its business units, infrastructure strategy, and the academic scope of the end-of-study internship, all of which explain the context in which a secure remote industrial access platform was identified as a necessary development.

1.1 Host Organization

Novation City is a competitiveness cluster established in 2009 in Sousse, Tunisia, as a public-private partnership. The cluster is designed to foster innovation ecosystems and enhance industrial competitiveness through digital transformation and advanced technologies, addressing the growing demand for mechatronics expertise and Industry 4.0 adoption in the MENA region [1].

Figure 1: Novation City Logo

Novation City's mission is to foster an innovation ecosystem in the mechatronics sector by bringing together companies, industrial entities, research centers, universities, and engineering schools, with the objective of enhancing regional and national competitiveness through

technology advancement, knowledge transfer, and strategic investment attraction. The cluster's long-term strategic vision positions it as a leading Industry 4.0 competitiveness hub for North Africa and the broader MENA region, pursued through investment in advanced innovation infrastructure, highly skilled technical talent development, and sustainable academia-industry partnerships.

1.2 Business Units and Services

Novation City operates through four specialized business units, each addressing a distinct aspect of the innovation ecosystem.

Acceleration Unit: Supports startup growth through business incubation services, mentorship programs, and access to funding opportunities. Services include workspace provision, networking facilitation, and investor connections.

Advisory Unit: Delivers strategic consulting focused on digital transformation and Industry 4.0 implementation. Advisory services encompass digital readiness assessments, technology selection, change management, security compliance audits, and infrastructure planning for remote operations platforms.

Formation Unit (Novation Akademy): Provides training and skills development programs covering mechatronics, automation, digital technologies, project management, and innovation methodologies in collaboration with universities and industry partners.

Innovation Unit: Drives research and development initiatives, facilitating collaborative R&D between academic institutions and industrial partners while exploring emerging technologies for the regional ecosystem.

1.3 Infrastructure and Strategic Focus

Novation City manages three specialized geographic zones: **Mechatronic City**, which houses companies focused on mechatronics, robotics, and advanced manufacturing, supported by specialized laboratories and testing facilities; **Business City**, which accommodates service-oriented companies and consulting firms with modern collaborative office environments; and **Industrial City**, dedicated to manufacturing operations with infrastructure supporting advanced manufacturing processes and Industry 4.0 technologies. The geographic separation of these zones means that engineers, students, and technical partners operating across sites cannot rely on physical co-location for equipment access, directly motivating the need for a unified remote access platform.

A central pillar of Novation City's strategy is promoting Industry 4.0 principles and digital transformation. As documented in the industrial adoption literature [1], many manufacturing

companies face significant challenges in implementing Industry 4.0 technologies, including limited awareness, uncertainty about return on investment, skills gaps, and integration difficulties with legacy systems. To address these challenges at scale, Novation City extends its capabilities with secure remote access platforms that enable engineers to interact with industrial systems remotely through the IoT-REAP initiative, combining Infrastructure-as-a-Service provisioning with zero-trust security principles.

1.4 Academic Framework

This project was carried out as part of an end-of-study software engineering internship (*stage de fin d'études de licence*) at Novation City, conducted during the 2025–2026 academic year within the software engineering programme. The internship was supervised jointly by an academic advisor and an Advisory Unit technical referent, with deliverables including the present technical report, and a fully operational platform deployment. The academic objective was to apply web engineering, software architecture, database modeling, remote infrastructure integration, and secure system design to a concrete industrial problem. This required formal documentation through UML modeling, requirements analysis, implementation reporting, and systematic validation of the delivered platform against specified requirements. The project thus bridges academic software engineering practice with real Industry 4.0 operational needs, providing both a meaningful professional contribution and a comprehensive vehicle for assessing end-of-study technical competencies.

2 Problem Statement

As manufacturing systems become increasingly digitized and interconnected under the Industry 4.0 paradigm, engineers and technicians working within organizations such as Novation City require hands-on access to specialized industrial equipment - Programmable Logic Controllers (PLCs), Human-Machine Interfaces (HMIs), SCADA systems, and industrial robots - for training, development, and maintenance purposes. Meeting this demand through physical presence alone is no longer sustainable at scale, yet the available remote access alternatives introduce their own operational, security, and logistical challenges. This section structures the problem along three complementary axes: the intrinsic limitations of remote access approaches, the security vulnerabilities they expose in operational technology (OT) environments, and the resource management difficulties they create for shared industrial infrastructure.

2.1 Remote Access Limitations

Traditional approaches to providing engineers with access to industrial systems impose physical constraints and operational overhead that directly limit training scalability and operational agility.

Physical Presence Requirements: Industrial training and maintenance activities have historically required engineers to be on-site, in front of the equipment. For an organization such as Novation City, which serves engineers, students, and industrial partners distributed across multiple sites in the MENA region, this model caps the number of simultaneous trainees at the laboratory's physical seating capacity. Academic literature confirms this bottleneck: remote industrial laboratories are specifically motivated by the inability to scale equipment access to growing student cohorts and the high cost of replicating physical setups at multiple locations [5, 6].

VPN Complexity and Operational Overhead: Virtual Private Networks remain the most widely deployed approach for extending remote access to industrial systems. However, VPN deployments require significant IT expertise to configure, maintain, and keep patched. Split-tunneling configurations create security vulnerabilities, and VPN credentials are prime targets for credential theft. According to a 2023 industry survey, nearly half of organizations reported being targeted by attackers exploiting VPN vulnerabilities, with one in five experiencing an attack in the preceding year [7]. The Dragos 2025 OT/ICS Cybersecurity Report documented 1,693 ransomware attacks targeting industrial organizations in 2024, an increase of 87 percent over the prior year, with threat actors explicitly exploiting unpatched VPN appliances and exposed remote desktop services as the primary initial access vector [8]. Furthermore, more than 50 percent of the ransomware incidents responded to in 2024 involved a remote service such as a VPN appliance or an RDP server being leveraged as the entry point [9].

Fragmented Tooling: Organizations attempting to support remote industrial access currently deploy separate, disconnected systems for virtual machine management, remote desktop sessions, IoT device control, and camera monitoring. Each tool carries its own authentication model, logging format, and maintenance overhead. Secomea's *State of Industrial Remote Access 2026* report, based on a global survey of industrial organizations, confirms that most still depend on overlapping combinations of VPNs, OEM-supplied tools, privileged access solutions, and newer OT-specific platforms. This fragmentation produces inconsistent audit trails and redundant access pathways even where individual tools are individually well-configured [10].

Absence of Browser-Native Access: All dominant industrial remote access tools require client-side software installation - a VPN client, a proprietary gateway agent, or a vendor-specific desktop application. This dependency increases deployment complexity, raises endpoint security exposure in shared training environments, and prevents access from workstations where

users do not hold administrative rights. No widely adopted commercial solution currently provides full interactive access to PLCs, HMIs, and virtual machines through an unmodified browser without client installation [10, 11].

2.2 Security Concerns in OT Environments

Industrial networks operate under security constraints that are fundamentally different from enterprise IT environments. The Purdue Reference Model organizes industrial network zones into five hierarchical levels, from field devices at Level 0 through control systems at Levels 2–3 to enterprise IT at Levels 4–5. Mixing IT remote access tools with OT environments violates these zone boundaries and introduces attack surface that cannot be managed by perimeter defenses alone.

IT/OT Network Boundary Violations: Traditional IT remote access solutions are designed to provide broad network access once authenticated. Once inside a VPN, lateral movement across the OT network is often unrestricted, meaning a compromised engineer's credential can expose field devices, safety PLCs, and production systems that were never intended as remote access targets. Cisco's industrial security guidance confirms that always-on VPNs installed in the Industrial Demilitarized Zone (IDMZ) provide all-or-nothing access to OT assets, making fine-grained zone isolation impossible without additional tooling [12]. Forescout researchers further found that remote access is "one of the least governed paths into CPS environments," where VPNs and jump hosts extend network trust rather than enforcing granular control, frequently relying on shared or persistent credentials [11].

Absence of Session Governance: Existing solutions provide no automated session expiration, resource quotas, or per-session authorization scoping. An authenticated engineer retains persistent access until manually disconnected, with no platform-enforced idle timeout or automatic revocation when a reservation window ends. In shared training environments, this creates security exposure extending well beyond the scheduled access period.

Insufficient Audit Trail for Industrial Compliance: Industrial security standards and emerging regulations including the NIS2 Directive and the Cyber Resilience Act require comprehensive logging of all access and command events. Existing remote access tools provide basic connection logs at best, with limited session-level granularity. The Secomea 2026 report notes that fragmented access stacks produce "inconsistent oversight, redundant access pathways, and uneven coordination between IT, OT, and external vendors," and that even organizations with high self-reported security confidence frequently exhibit audit trail gaps traceable to their multi-tool architectures [10].

Lack of Fine-Grained Role-Based Access: No existing general-purpose industrial remote access tool enforces role-aware access policies that distinguish between a trainee provisioning

a safe virtual environment for practice, an experienced engineer accessing a live PLC, and an administrator managing infrastructure. All authenticated users typically receive equivalent network-level access, with access differentiation left to external firewall rules.

2.3 Resource Sharing and Training Challenges

Beyond connectivity and security, the operational context of a technology cluster such as Novation City introduces a third problem dimension: coordinating shared access to expensive, limited industrial resources across concurrent users with heterogeneous skill levels and training objectives.

Equipment Scarcity and Cost: Industrial robots, specialized PLC hardware, and high-fidelity virtual machine images configured for specific training scenarios represent significant capital investment. Academic literature identifies equipment cost as a leading constraint on the scalability of industrial automation education, noting that the inability to replicate physical setups across multiple sites or time slots fundamentally limits the number of trainees who can receive meaningful hands-on experience [13]. Virtual environments can extend equipment reach, but only when provisioning, lifecycle management, and cleanup are automated within the same platform used to schedule access.

Scheduling Conflicts and Double-Booking: Without an integrated reservation system that enforces exclusive access to physical devices and applies session quotas to virtual machines, concurrent users will conflict over shared resources. No current industrial remote access product provides automated reservation management with atomic transaction guarantees to prevent double-booking. Organizations relying on manual scheduling via shared calendars or email coordination cannot guarantee conflict-free access at scale.

Lack of Integrated Training Context: Simulation platforms and remote laboratories address the access problem but leave trainees without the instructional scaffolding needed to make hands-on sessions productive. A trainee who provisions a virtual PLC environment needs prerequisite theory, guided exercises, quizzes, and a record of progress - none of which are provided by any existing industrial remote access platform. The result is a pedagogical gap between content delivery (managed in a separate LMS) and practical laboratory work (managed in a separate remote access tool), creating coordination overhead for instructors and context-switching friction for trainees.

Table 1: Critical Limitations of Current Industrial Remote Access Approaches

Limitation Category	**Specific Problem**	**Operational Impact**
Physical Access Dependency	On-site presence required for all equipment interaction	Training capacity capped by lab seating; high travel costs
VPN Security Exposure	Credential theft, unpatched appliances, unrestricted lateral movement	87% increase in OT ransomware; VPN as primary initial access vector
Fragmented Tooling	Separate systems for VMs, remote desktop, IoT devices, cameras	Inconsistent audit trails, redundant credentials, governance gaps
No Browser-Native Access	Client installation required on every engineer's workstation	Deployment overhead; impossible on locked-down training machines
No Session Governance	No idle timeout, no quota enforcement, no reservation system	Resource conflicts, security exposure beyond scheduled windows
Insufficient Audit Trail	Limited event granularity; basic connection logs only	Non-compliance with NIS2, CRA, and industrial security standards
Disconnected Training	LMS and remote access are separate, unintegrated systems	Context-switching; no coupling of theory to hands-on practice

3 Study of the Existing

The limitations established in Section 2 motivate a systematic examination of the industrial remote access solutions currently deployed in the field. Rather than describing solution categories in the abstract, this analysis examines four representative commercial products that collectively cover the main approaches in use: Secomea SiteManager/GateManager, HMS eWON Cosy+ with Talk2M, Siemens SINEMA Remote Connect, and TeamViewer Tensor with Agentless Access. Each product is analyzed in terms of its architecture, core capabilities, and the specific limitations it exhibits relative to the IoT-REAP requirements. The analysis then draws a consolidated criticism that identifies the gaps no existing solution addresses, justifying the need for a purpose-built platform.

3.1 Analysis of Existing Industrial Remote Access Solutions

3.1.1 Secomea SiteManager / GateManager

Secomea, a Danish industrial networking company founded in 2008, offers one of the most widely deployed purpose-built remote access platforms for operational technology environments. The solution is structured around three tightly integrated components [14].

Figure 2: Secomea Logo

The **SiteManager** is an edge gateway deployed at the industrial site, available either as a DIN-rail-mountable hardware appliance (models 15XX/35XX) or as an embeddable software agent installable on Windows, Linux, HMIs, or industrial PCs. A single hardware SiteManager aggregates up to 100 industrial devices from any vendor via Ethernet, serial, and USB interfaces, using pre-defined device agents for PLCs, HMIs, and SCADA systems. The **GateManager** is a cloud or on-premises management server acting as a certificate-based rendezvous broker: the SiteManager and the remote engineer each initiate outbound connections to the GateManager, which verifies participant identities through X.509 certificate exchange before bridging the tunnel. This design eliminates the need for inbound firewall rule modifications at the industrial site. The **LinkManager** is the client application installed on the engineer's workstation to establish and manage remote sessions.

Secomea integrates with enterprise identity providers (Azure AD, Okta) via SSO, offers drag-and-drop user and access-right management on the GateManager portal, and holds IEC/PAS 62443-3 compliance certification as well as conformance with the Reference Architecture Model Industrie 4.0 (RAMI4.0) [15].

Limitations: Despite its OT pedigree, Secomea exhibits three critical constraints for the IoT-REAP context. First, full interactive remote desktop access to PLCs and HMIs still requires the LinkManager desktop client installed on the engineer's workstation; browser-native access is limited to specific supported device types and does not provide generic RDP or VNC capability over arbitrary VM targets. Second, the platform is designed exclusively for maintenance and teleservice of physical field devices: it provides no virtual machine provisioning, template management, or session lifecycle automation. Third, the per-device licensing model becomes cost-prohibitive in training environments where dozens of concurrent VM sessions must be provisioned and cleaned up on demand. There is no integrated resource reservation system, no session quota management, and no e-learning content delivery layer.

3.1.2 HMS eWON Cosy + and Talk2M

HMS Networks, a Swedish industrial connectivity specialist with over 600,000 registered gateway devices globally, delivers industrial remote access through the **eWON Cosy+** hardware gateway and the **Talk2M** cloud service [16].

Figure 3: HMS eWON Cosy+ Gateway Logo

The eWON Cosy+ is a compact DIN-rail gateway installed inside a machine's control panel, connecting to PLCs and HMIs via Ethernet, serial, or USB. It establishes an outbound SSL/TLS-encrypted VPN tunnel to Talk2M using X.509 certificate-based mutual authentication and hardware secure element tamper resistance. Talk2M acts as a secure broker completing the encrypted end-to-end tunnel between the remote engineer and the connected machine. The **eCatcher** desktop VPN client manages tunnel establishment and access rights on the engineer's workstation, while the **M2Web** web portal provides a browser-accessible interface for machine KPI monitoring and alarm review [17].

Talk2M is offered in three plans: **Talk2M Free** (included with gateway purchase, valid for 12 months per device, single concurrent connection), **Talk2M Light** (two simultaneous VPN connections, 99.5% SLA-backed availability), and **Talk2M Pro** (configurable concurrent connection pools, firmware OTA management, visualization dashboards, and a DataMailbox API for multi-device data retrieval) [18]. Administrators can consult session initiator, duration, and target gateway in eCatcher audit reports, and can define user groups and device pools for access segmentation on the Pro plan.

Limitations: The eWON/Talk2M solution presents several critical gaps. Full remote desktop access to PLCs and HMIs requires eCatcher installed on the engineer's workstation; M2Web provides only data visualization and KPI monitoring, not interactive remote control sessions. This client installation requirement is incompatible with browser-only access for scalable training deployments. The concurrent connection limits create a hard scalability ceiling: Talk2M Light caps at two simultaneous VPN connections, meaning a cohort of 20 engineers cannot access shared equipment concurrently without the account being capped until the end of the billing month [19]. There is no virtual machine provisioning capability, no automated

resource reservation or scheduling system, no session quota enforcement, and no training content delivery. The solution is optimized for point-to-point maintenance access to physical machines, not for coordinating concurrent multi-user access to shared virtual laboratory environments.

3.1.3 Siemens SINEMA Remote Connect

Siemens AG offers **SINEMA Remote Connect** (SINEMA RC) as the dedicated remote access management platform within its industrial communication portfolio, primarily targeting industrial sites operating Siemens automation infrastructure [20].

Figure 4: Siemens SINEMA Remote Connect Logo

SINEMA RC operates as a rendezvous server using OpenVPN: service technicians and the SCALANCE hardware router at the machine each establish separate OpenVPN connections to the SINEMA RC Server, which verifies participant identities through certificate exchange before bridging the session. The required on-site hardware is a **SCALANCE** industrial router from the M or S series (e.g., SCALANCE S615, SCALANCE M874). The SINEMA RC Client software must be installed on the technician's workstation to establish the tunnel. The management interface provides an address book of registered devices, participant group management (grouping users, devices, and subnets), and REST API support for automated bulk management tasks [21]. The cloud instance is hosted in an ISO/IEC 27001 certified data center, and an on-premises deployment option is available. The platform supports cellular, DSL, and private WAN connectivity and handles up to 800 Mbps aggregate transfer across an unlimited number of registered devices.

Limitations: SINEMA RC imposes significant constraints outside of its native Siemens ecosystem. The mandatory SCALANCE hardware at the machine side creates strong vendor lock-in: facilities running Rockwell, Schneider, or third-party PLCs alongside Siemens equipment cannot use SINEMA RC as a unified access layer without deploying SCALANCE routers at every OT segment. The platform provides no browser-native remote desktop capability; the SINEMA RC Client must be installed on the technician's workstation. There is no virtual machine provisioning or lifecycle management, no resource reservation or scheduling system, no MQTT or IoT telemetry integration, and no training content delivery layer.

The solution was designed for teleservice and remote maintenance by qualified automation engineers, not for scalable access coordination across a mixed population of trainees and instructors.

3.1.4 TeamViewer Tensor with Agentless Access

TeamViewer, a German remote support company originally serving IT helpdesk scenarios, has progressively extended its **Tensor** enterprise platform toward operational technology. In November 2025, TeamViewer launched **Agentless Access** for Tensor, allowing manufacturers and OEM machine builders to access PLCs, HMIs, and other OT endpoints remotely without installing agent software on each controlled device [22].

Figure 5: TeamViewer Tensor Logo

TeamViewer Tensor for OT provides zero-trust remote access with granular role-based access controls, protocol isolation, session audit logging, and centralized device management. Manufacturers such as Schwäbische Werkzeugmaschinen GmbH already deploy it for remote PLC programming and shop-floor automation control. Agentless Access extends these capabilities to legacy devices and closed systems by allowing engineers to work directly on OT endpoints from a central location without making software changes to the controlled devices [22]. The platform additionally supports terminal access, application control, edge management, and integration with third-party services such as IBM Maximo for asset management workflows.

Limitations: TeamViewer's OT capabilities remain recent additions to an architecture whose origins and operational model are rooted in IT remote support. The platform does not natively enforce industrial network segmentation according to the Purdue Reference Model; enforcement of OT/IT zone boundaries requires external network configuration and is not managed as a first-class platform policy. There is no virtual machine lifecycle management: Tensor cannot provision, clone, snapshot, or terminate Proxmox or KVM-based VMs on behalf of a user. No resource reservation or scheduling mechanism prevents double-booking of shared physical or virtual equipment. Session audit logging is present but does not provide granular administrative oversight for non-administrative actions. No MQTT-based camera control is supported. No training content delivery layer exists. Enterprise Tensor licensing follows a per-seat subscription model at significant cost, making it poorly adapted to academic institutions that need to provision transient access for large, rotating cohorts of trainees. Finally, all sessions route through TeamViewer's proprietary cloud relay infrastructure rather than

institution-controlled servers, which raises data sovereignty concerns for OT environments subject to industrial compliance obligations.

3.2 Criticism of the Existing

The systematic examination of the four representative solutions reveals a consistent structural pattern: each product was designed to address a specific, well-defined operational scenario (field maintenance access, OT-compatible VPN tunneling, Siemens ecosystem teleservice, or IT helpdesk-style remote support) without considering the integrated requirements of a training-oriented, multi-user virtual laboratory platform. The following critical deficiencies are shared across all reviewed solutions and are directly addressed by IoT-REAP.

Client Installation Dependency Across All Solutions: Secomea LinkManager, eWON eCatcher, Siemens SINEMA RC Client, and TeamViewer Tensor all require desktop client software installation for full-capability access. This dependency increases deployment overhead on training machines where users may not hold administrative rights, raises endpoint security exposure in shared environments, and eliminates the possibility of zero-friction access from an unmodified web browser. IoT-REAP eliminates this barrier entirely through Apache Guacamole, which tunnels RDP, VNC, and SSH protocols over WebSockets from a standard browser with no client installation whatsoever.

Universal Absence of Virtual Machine Provisioning: None of the four solutions provides any VM lifecycle management capability. Engineers cannot self-provision a fresh virtual environment from a template catalog, extend an active session, hibernate an idle instance to preserve resources, or trigger automated VM cleanup on session expiry. This gap means that virtual laboratory infrastructure must be managed through a separate, unintegrated tool, reintroducing the fragmentation problem that motivates IoT-REAP. The platform integrates directly with Proxmox VE through a REST API client to enable on-demand provisioning, extension, hibernation, and cleanup within the same unified interface.

No Resource Reservation or Session Quota Management: Secomea, eWON, SINEMA RC, and TeamViewer Tensor provide no mechanism to prevent double-booking of shared physical devices or shared virtual machines. Talk2M's concurrent connection cap (two on the Light plan) illustrates the problem without solving it: it enforces a hard ceiling on simultaneous users rather than providing per-resource reservation with waiting queue management. IoT-REAP implements an atomic reservation system using database transaction locks, a waiting queue for devices already reserved, and session quota enforcement per user and per resource type.

Incomplete OT/IT Network Segmentation as a Platform Policy: All four solutions require network-layer configuration external to the platform to enforce Purdue model zone isolation. Secomea and eWON provide machine LAN segregation at the gateway level, and SINEMA RC

enforces isolation through SCALANCE routing rules, but in all cases zone enforcement is a network infrastructure concern rather than a platform access policy applied consistently to all session types. IoT-REAP enforces IT/OT zone separation through Proxmox cluster VLAN and firewall configuration bound to administrative policies that govern which VM session can access which network zone, without requiring users to understand or configure the underlying network topology.

No Integrated Training Content Delivery in Any Solution: All four reviewed solutions focus exclusively on connectivity. None provides any mechanism to associate a remote access session with prerequisite instructional content, progress tracking, quizzes, or certificates within the same interface. Engineers using any of these tools to access a training PLC or virtual laboratory environment must navigate to a separate LMS for theory, then context- switch to the remote access tool for practice. IoT-REAP directly couples training paths, modules, and practical VM laboratory sessions within a single interface, enabling seamless transition from instructional content to hands-on practice without leaving the platform.

Table 2: Comparative Analysis of Existing Industrial Remote Access Solutions

Requirement	**Secomea SiteManager**	**HMS eWON / Talk2M**	**Siemens SINEMA RC**	**TeamViewer Tensor**
Browser-Native Full Access	Partial: limited device types only	Poor: eCatcher client required	Poor: SINEMA RC client required	Poor: agent/client required
VM Lifecycle Management	None	None	None	None
Resource Reservation & Quotas	None	Poor: connection caps only	None	None
OT/IT Zone Enforcement	Good: OT-native gateway design	Good: machine LAN segregation	Good: SCALANCE-enforced zones	Partial: external config required
Session Audit Logging	Partial: access events only	Partial: initiator and duration	Partial: VPN connection logs	Good: full session record
Multi-Vendor Hardware Support	Good: vendor-agnostic agents	Good: any device via Ethernet	Poor: Siemens ecosystem only	Good: agentless OT endpoints
IoT Command Dispatch (MQTT)	None	None	None	None
Integrated E-Learning Platform	None	None	None	None

The comparative analysis confirms that no existing commercial solution addresses the combination of browser-based VM provisioning, remote desktop access, IoT command dispatch, resource reservation, and integrated training content delivery that the Novation City operational context requires. Each reviewed product fills a genuine industrial need but leaves the multi-dimensional requirements of a training-oriented virtual laboratory platform entirely unaddressed. These deficiencies, taken together, constitute a clear and specific justification for

designing and implementing IoT-REAP as a purpose-built platform for Industry 4.0 training and remote laboratory operations.

4 Proposed Solution

Based on the deficiencies consolidated in Section 3, this section presents the IoT-REAP platform as a purpose-built response to each identified gap. It describes the platform's conceptual definition, its four functional pillars, the technology stack and selection rationale, the architectural approach, and the security and resource management model. The goal is to demonstrate that the proposed design is not merely a combination of existing tools but a coherent platform whose architecture is specifically shaped by the requirements that no commercial product currently satisfies.

4.1 Platform Definition and Core Concept

IoT-REAP (Internet of Things - Remote Equipment Access Platform) is a web-based platform that provides engineers, trainees, and instructors with secure, browser-native access to industrial virtual machines, IoT devices, and physical laboratory resources, within a unified interface that combines remote infrastructure provisioning, e-learning delivery, and resource governance. The platform is conceived as a single point of access that replaces the fragmented combination of VPN clients, proprietary gateway agents, separate LMS systems, and manual scheduling that currently characterizes industrial training environments.

The core design principle is **zero client installation**: all interactive capabilities - remote desktop sessions over RDP and VNC, live camera streaming, IoT command dispatch, training content consumption, and administrative governance – are delivered entirely through an unmodified browser. This eliminates the endpoint security exposure and deployment overhead inherent in all four reviewed commercial solutions and makes the platform accessible from shared or locked-down training workstations where users do not hold administrative rights.

The platform is further defined by **unified resource lifecycle management**: virtual machines are not merely accessed through the platform but are provisioned, extended, hibernated, and cleaned up within it, using direct integration with Proxmox VE. Physical IoT devices and cameras are reserved, attached to sessions, and released through the same interface, with atomic transaction guarantees preventing double-booking. This distinguishes IoT-REAP from all reviewed products, none of which provide any VM lifecycle management or reservation system.

4.2 Platform Pillars

The platform is structured around four functional pillars that collectively address the dimensions of the problem identified in Section 2.

Pillar 1 - Virtual Machine Provisioning and Remote Desktop Access: Engineers self-provision virtual machine sessions from a curated template catalog through a browser interface. The platform communicates with Proxmox VE clusters through a REST API client to clone templates, configure network isolation according to Purdue model zone assignments, and return session metadata to the requesting user. Once a session is active, browser-based remote desktop access is established through Apache Guacamole, which tunnels RDP, VNC, and SSH connections over WebSocket without any client-side installation. Session lifecycle operations - extension in configurable increments, explicit termination with automatic VM cleanup, and idle hibernation after inactivity - are all managed within the platform, eliminating the uncoordinated session sprawl identified as a security and resource management failure in existing solutions.

Pillar 2 - Camera Integration: Laboratory cameras (ESP32-CAM and PTZ units) stream live MJPEG feeds to the platform gateway, which proxies them to the authorized user's browser and accepts pan-tilt-zoom control commands. All device access is gated by the reservation system described under Pillar 4, ensuring that camera streams are only exposed to users with active, authorized reservations.

Pillar 3 - Integrated E-Learning and Practical Training Delivery: A training platform is embedded directly within IoT-REAP, replacing the disconnected LMS-plus-remote-access-tool model that creates context-switching friction for trainees. Instructors author structured training paths composed of modules and training units containing articles, HLS-streamed instructional videos, and timed quizzes with auto-grading. Crucially, practical virtual machine laboratory sessions are coupled directly to training units: an engineer studying a PLC programming lesson can launch a pre-configured VM session from within the same interface without navigating to a separate tool. Progress tracking, timestamped notes, course reviews, and verifiable digital certificates are all managed within the platform, providing a complete pedagogical lifecycle from enrollment to certified completion.

Pillar 4 - Resource Reservation, Governance, and Audit: An atomic reservation system governs access to all shared resources - physical devices, camera units, and virtual machine quotas - using database-level transaction locks to guarantee conflict-free scheduling. Users attempting to reserve an already- reserved resource enter a waiting queue and are notified upon availability. Session quotas per user and per resource type are enforced by the platform, independent of network-layer controls. All sensitive operations - session creation, device attachment, administrative user actions, and payment events - generate persistent audit records: each record captures the initiator, timestamp, and operation details, providing a verifiable history of system activity. This audit mechanism directly addresses the compliance gaps identified relative to the NIS2 Directive and the Cyber Resilience Act.

5 Requirements Specification

The requirements specification translates the problem statement and proposed solution into concrete actors, system services, and quality constraints. It identifies who interacts with the platform, what each actor must be able to do, and the measurable quality properties expected from the solution.

5.1 Identification of Actors

The platform is organized around role-based access control implemented through three application roles: `engineer`, `teacher`, and `admin`. An unauthenticated state represents public access. Each actor interacts with the same platform from a different operational perspective, and each role is associated with a distinct set of services, permissions, and workflows. **Unauthenticated User (Guest)** represents any visitor who has not yet authenticated. This actor can browse public training paths, access legal and informational pages, consult public discussion threads in read-only mode, view training path reviews, verify public certificates, initiate authentication and registration workflows, and access public checkout result pages after external payment redirection.

Engineer is the primary trainee and remote laboratory user. This actor enrolls in training paths, consumes educational resources (articles, videos, quizzes), tracks progress, writes notes, participates in discussions, posts reviews, claims certificates, and initiates payment and refund operations. Beyond the learning dimension, the engineer also provisions virtual machine sessions, opens browser-based remote desktop access through Guacamole, configures connection preferences, reserves USB devices and cameras, attaches hardware to active sessions, enters waiting queues when devices are busy, and interacts with session-bound laboratory resources.

Teacher is the content producer and pedagogical supervisor. This actor creates and manages training paths, modules, and training units; authors articles; uploads and manages videos and captions; creates and publishes quizzes; reviews trainee interactions through analytics dashboards; moderates instructional discussions; submits training paths for administrative approval; and requests payout for generated revenue.

Administrator is the platform governance and infrastructure actor. This role supervises users, teacher approval, training path approval, featured training path ordering, payment-related review processes, payout validation, infrastructure supervision, Proxmox server and node management, VM assignment approval, hardware gateway verification, camera and reservation management, flagged forum content moderation, maintenance operations, alerts, and activity logging. Administrators also hold compliance-sensitive permissions such as user impersonation, suspension, and deletion.

Stripe (External Payment Processor) is an external system actor that supports platform commerce workflows. IoT-REAP integrates with Stripe through hosted checkout redirections and signed webhook callbacks to confirm payments, create internal payment records, and react to asynchronous state changes. Stripe also participates in finance workflows such as refunds and instructor payouts [24].

Google OAuth (External Identity Provider) is an external system actor that provides social authentication services. The platform delegates identity verification to Google's OAuth 2.0 endpoint, allowing users to authenticate using existing Google credentials. Upon successful authorization, Google returns verified identity claims that the platform uses to create or link local user accounts, reducing registration friction.

Camera Hardware (Secondary Actor) represents the ESP32-CAM and compatible PTZ units deployed in the laboratory environment. These devices stream MJPEG video feeds to the platform gateway and accept pan-tilt-zoom control commands initiated by authorized users. The platform mediates all camera interactions, handling stream proxying, resolution negotiation, and reservation state synchronization.

The interaction among these actors reveals that the platform combines four complementary dimensions: e-learning delivery, instructor authoring, virtual laboratory provisioning, and infrastructure governance. This combination distinguishes IoT-REAP from a conventional learning management system because educational workflows are directly coupled to remote computing resources and physical IoT device access.

5.2 Functional Requirements

The platform must support the following functional requirements according to actor responsibilities and shared system behavior.

Unauthenticated User (Guest):

- Access account registration and login workflows
- Consult the landing page and public legal pages
- Browse public training paths and training path details
- Read public reviews and forum threads in read-only mode
- Verify and download publicly accessible certificates through verification links
- View public search results, trending queries, and category-based discovery
- Access checkout success and cancellation pages after external payment flows

Engineer:

- Enroll in and unenroll from training paths
- Access owned or enrolled training units and learning materials
- Read articles and track article completion
- Stream instructional videos and update playback progress
- Attempt quizzes, submit answers, and review attempt history
- Mark training units complete or incomplete based on learning progress
- Create, update, and delete personal notes for training units
- Create, edit, and delete reviews for completed or relevant training paths
- Participate in forum discussions by creating threads, replying, voting, and flagging content
- View notifications and manage read status
- Claim, verify, and download certificates
- Initiate checkout for paid training paths
- Review payment history and submit refund requests

- Provision, extend, view, and terminate virtual machine sessions
- Retrieve Guacamole access tokens for remote desktop connectivity
- Manage connection preference profiles for available protocols
- Browse available Proxmox virtual machines and snapshots
- Reserve USB devices and cameras
- Attach or detach session hardware and join hardware waiting queues
- Access session camera streams, request PTZ control, and adjust stream resolution

Teacher:

- Create, edit, archive, restore, and delete training paths
- Organize training paths into modules and training units
- Reorder modules and training units
- Create and maintain quiz structures, questions, and publication status
- Create, update, and delete article content
- Upload, retry, and delete training videos
- Manage video captions
- Submit training paths for administrative review
- View teacher analytics for enrollment, earnings, and learning funnel data
- Access a teacher discussion inbox and moderate course-related threads
- Request revenue payout
- Submit training unit virtual machine assignment requests

Administrator:

- Access the administrative dashboard and KPI views
- Approve, reject, feature, unfeature, and order training paths
- Review pending VM assignment requests
- Manage reservations for devices and cameras

- Manage Proxmox servers and nodes
- Start, stop, reboot, and shut down virtual machines through node operations
- Register, verify, update, and remove gateway nodes
- Discover hardware devices and refresh infrastructure status
- Bind, unbind, attach, detach, dedicate, and configure devices and cameras
- Assign and unassign cameras to virtual machines
- Approve teacher accounts and manage user roles
- Suspend, unsuspend, impersonate, and delete user accounts where authorized
- Review and process refund requests
- Review, approve, reject, and process payout requests
- Monitor video processing status
- Place devices and cameras in maintenance mode
- Moderate flagged threads and replies
- Manage system alerts and consult activity logs

System-Wide Functional Requirements:

- Enforce authentication and email verification for protected features
- Apply authorization policies according to role and ability
- Persist learning, payment, infrastructure, and moderation data
- Support public and protected routes through a unified web interface
- Provide rate limiting for sensitive or abuse-prone operations
- Maintain notification and audit mechanisms for critical events
- Support asynchronous or stateful workflows (video processing, VM lifecycle operations, refund handling, hardware attachment)

5.3 Non-Functional Requirements

The quality constraints of the platform are defined by its educational, infrastructure, and operational nature. Where applicable, measurable thresholds are specified to allow objective verification during the testing phase.

Performance:

- API response time for standard CRUD operations must not exceed 500 ms under normal load conditions
- VM provisioning and session initialization must complete within 30 seconds for standard configurations
- Remote desktop connection establishment through Guacamole must not exceed 5 seconds post-authentication
- Video streaming and progress tracking must maintain end-to-end latency under 2 seconds
- Search, analytics summaries, and notification retrieval must execute within acceptable response windows under concurrent demand

Security:

- Authentication must rely on secure identity management with mandatory email verification
- Authorization must be enforced by middleware and policy gates at every protected endpoint
- Payment workflows must validate external webhook signatures and preserve transaction integrity
- Access to VM sessions, cameras, and USB devices must be restricted to authorized users with active reservations

Reliability:

- Learning progress, notes, payments, and reservations must remain durable and consistent under concurrent access
- Hardware and infrastructure actions must tolerate transient failures and preserve observable system state

- Session resources must support full lifecycle management including creation, extension, hibernation, and cleanup
- Alerting and logging mechanisms must assist recovery from operational issues

Usability:

- The user interface must remain coherent and navigable for trainees, teachers, and administrators without requiring specialized infrastructure knowledge
- Administrative screens must expose complex operations in a controlled and structured form
- Forum, notes, review, and certificate features must integrate naturally into the learning workflow

Scalability and Maintainability:

- The architecture must support extension through modular routes, controllers, models, and services without disrupting existing functionality
- Domain separation between learning, commerce, virtual laboratory, and infrastructure operations must remain clearly bounded
- The platform must support future growth in training content, users, devices, and compute nodes without architectural refactoring
- The codebase must be supported by automated tests, strongly typed frontend code, and structured backend services

6 Work Methodology

The development of IoT-REAP required a structured approach to coordinate complex technical requirements with evolving stakeholder needs. This section presents the Agile Scrum methodology selected for the project, the product backlog that governed sprint priorities, the associated ceremonies, and the UML modeling language used for system documentation.

6.1 The Agile Scrum Methodology

Agile Scrum was selected for IoT-REAP because the platform combines evolving infrastructure requirements (Proxmox integration, Guacamole, MQTT), camera integration, and iterative stakeholder validation - conditions that make fixed, waterfall-style planning impractical. This subsection defines the Scrum framework and describes its application to the project.

6.1.1 Definition

Agile Scrum is a structured framework for implementing Agile principles in software development [25]. While Agile is a broad philosophy emphasizing iterative delivery and adaptability, Scrum is a concrete instantiation that defines roles, ceremonies, and artifacts. Scrum organizes work into fixed-duration iterations called **sprints** (typically one to four weeks), during which a cross-functional team commits to delivering a potentially shippable product increment. Each sprint follows a cycle of planning, daily standups, development, review, and retrospective ceremonies.

Scrum defines three roles: the **Product Owner** (prioritizes the backlog and validates requirements), the **Scrum Master** (facilitates ceremonies and removes blockers), and the **Development Team** (delivers the increment). The framework manages work through three artifacts: the **Product Backlog** (prioritized feature list), the **Sprint Backlog** (items committed for the current sprint), and the **Increment** (working software produced at sprint end).

The Agile Scrum framework builds on the Agile Manifesto (2001) [25], which prioritizes:

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

For IoT-REAP, these principles translate directly into practice: infrastructure integration decisions (Guacamole configuration, Proxmox API behavior) were refined iteratively based on empirical results rather than upfront specification, and the training platform scope was introduced mid-project in response to stakeholder feedback without disrupting the core laboratory platform delivery.

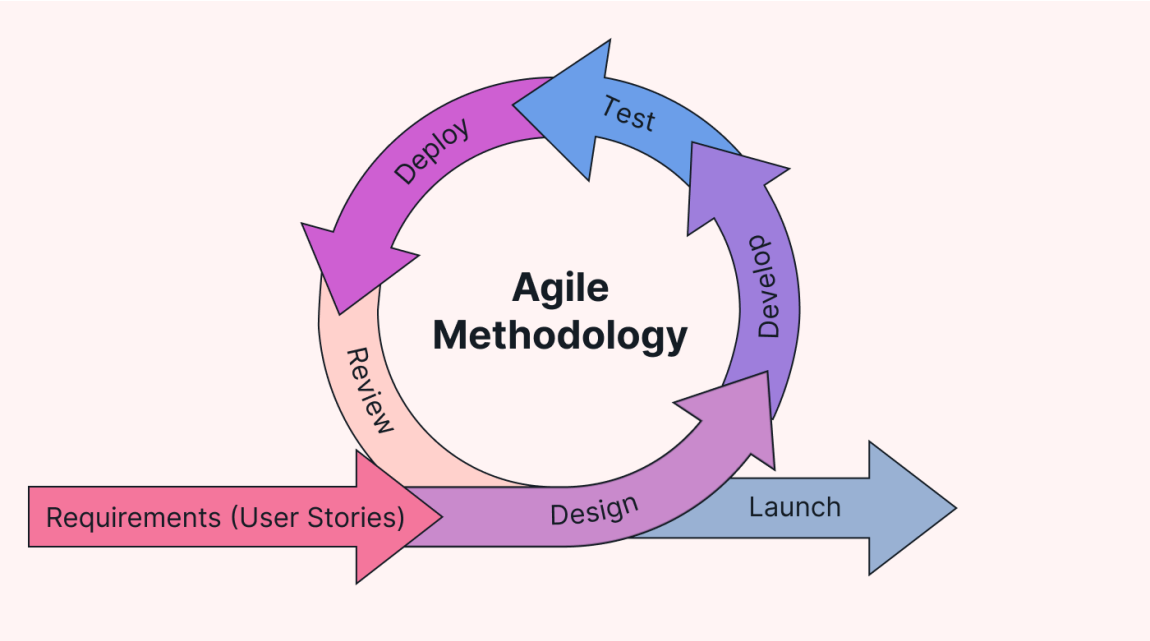


Figure 6: Agile Scrum Methodology Workflow

6.1.2 Characteristics of Agile Scrum

Sprint-Based Delivery: The IoT-REAP project is organized into fixed 3-week sprints, each with a starting Sprint Planning session, daily progress tracking, and a closing Sprint Review and Retrospective. This structure provides predictable delivery cycles and regular stakeholder engagement.

Sprint Planning & Commitment: At the beginning of each sprint, the team commits to specific user stories with clear acceptance criteria. In the solo developer context, this involved supervisor and Advisory Unit stakeholder alignment on sprint goals, ensuring shared understanding before development began.

Product Backlog Management: Features and requirements are maintained in a prioritized Product Backlog. Items are ordered by business value, risk, and dependency. New requirements (infrastructure changes, the training platform addition) are incorporated into subsequent sprints without disrupting committed work.

Sprint Increments and MVP Validation: Each sprint produces a working software increment. Sprint 3 delivers the MVP (VM provisioning + remote desktop access), providing an early validation milestone with stakeholders before the full IoT control layer, camera integration, and training platform are implemented in subsequent sprints.

Sprint Review & Stakeholder Feedback: At each sprint boundary, completed work is demonstrated to stakeholders. Reviewers examine running features, validate acceptance criteria, and provide feedback that influences subsequent sprint prioritization.

Sprint Retrospective & Continuous Improvement: Following each review, retrospectives identify improvements in process, tooling, and code quality. Early retrospectives surfaced the need for more comprehensive API documentation and identified infrastructure bottlenecks requiring architectural adjustments.

Definition of Done: For IoT-REAP, a user story is considered complete when the implementation is written and validated against its acceptance criteria by the developer and supervisor, unit and integration tests covering the delivered behavior are written and passing, and any relevant API or user-facing behavior is documented. Quality assurance is embedded throughout every sprint rather than deferred to a dedicated phase, ensuring continuous validation of the delivered increment.

6.2 Product Backlog

The IoT-REAP platform development followed a prioritized product backlog organized by sprint. Table 3 presents the representative user stories per sprint with their priorities and assignments. Items 28–40 correspond to granular infrastructure hardening and API refinement tasks tracked internally; this table presents the principal user-facing stories for conciseness. Quality assurance, security scanning, load testing, and documentation activities are integrated into each sprint's scope as part of the Definition of Done rather than isolated in a dedicated delivery phase.

Table 3: IoT-REAP Platform Product Backlog

ID	User Story / Feature	Priority	Sprint
Sprint 1: Foundation, Authentication			
01	As a user, I want to register and log in securely using session-based and Google OAuth authentication	High	Sprint 1
02	As an admin, I want role-based access control (engineer, teacher, admin) with distinct permissions	High	Sprint 1
03	As a user, I want to enhance my account security using Two-Factor Authentication (2FA)	High	Sprint 1
04	As a developer, I need a database schema for users with ULID primary keys for secure, time-sortable identifiers	High	Sprint 1
05	As a developer, I need an audit log system to track user and administrative actions for security verification	High	Sprint 1
06	As a user, I want password reset functionality and email verification	Medium	Sprint 1
07	As a developer, I need a React 18 frontend with TypeScript and Inertia.js as the application baseline	High	Sprint 1

Table 3 (continued)

ID	User Story / Feature	Priority	Sprint
08	As a developer, I need automated CI/CD pipelines for linting, testing, and deployment validation	Medium	Sprint 1
Sprint 2: Proxmox Integration, Multi-Cluster & Network Isolation			
09	As a developer, I need a database schema for VM nodes, templates, and sessions	High	Sprint 2
10	As a developer, I need a Proxmox API client with retry logic and error handling	High	Sprint 2
11	As an engineer, I want to browse VM templates and launch sessions	High	Sprint 2
12	As a developer, I need a load balancer to distribute VMs across Proxmox nodes	High	Sprint 2
13	As an admin, I want to view all nodes with live CPU/RAM statistics	Medium	Sprint 2
14	As an admin, I want to register multiple Proxmox servers with encrypted credentials	High	Sprint 2
15	As a developer, I need to extend the Proxmox architecture to support multiple independent clusters	High	Sprint 2
16	As an engineer, I want to select a target Proxmox cluster when provisioning a VM	High	Sprint 2
17	As an admin, I want to enforce Purdue model network isolation (IT/OT zone separation)	High	Sprint 2
18	As an admin, I want a complete dashboard with quota management and activity logs	High	Sprint 2
Sprint 3: Guacamole Remote Access - MVP Demo			
19	As a developer, I need a Guacamole client service for connection management	High	Sprint 3
20	As an engineer, I want browser-based RDP/VNC access to my VM session	High	Sprint 3
21	As an engineer, I want to extend my session duration in 30-minute increments	High	Sprint 3
22	As an engineer, I want to terminate my session and have the VM cleaned up	High	Sprint 3
23	As an engineer, I want to save my preferred connection settings (display, audio)	Medium	Sprint 3
24	As a developer, I need unit and integration tests for the Guacamole client, token generation, and session lifecycle flows	High	Sprint 3
Sprint 4: Camera, Session Management & Integration Hardening			

Table 3 (continued)

ID	User Story / Feature	Priority	Sprint
25	As a developer, I need idle session hibernation (suspend after 10 minutes of inactivity)	Medium	Sprint 4
26	As an engineer, I want a live camera feed from the ESP32-CAM	High	Sprint 4
27	As a developer, I need load testing targeting 15 concurrent logins, 5 active VM provisioning operations, and 10 simultaneous Guacamole sessions	High	Sprint 4
Sprint 5: Technical Training Platform & Delivery			
32	As an instructor, I want to create structured courses with modules, lessons, and prerequisite tracking	High	Sprint 5
33	As an instructor, I want to upload videos with HLS streaming and multi-quality playback (360p-1080p) [29]	High	Sprint 5
34	As a trainee, I want to enroll in courses and track my progress across lessons and modules	High	Sprint 5
35	As an instructor, I want to create timed quizzes with auto-grading, passing scores, and retry limits	High	Sprint 5
36	As a trainee, I want hands-on VM lab sessions integrated with course lessons for practical training	High	Sprint 5
37	As a trainee, I want to receive verifiable digital certificates upon course completion	High	Sprint 5
38	As a trainee, I want to take timestamped notes during video lessons for later review	Medium	Sprint 5
39	As a trainee, I want to rate and review courses to help other trainees make informed decisions	Medium	Sprint 5
40	As an instructor, I want an analytics dashboard showing enrollments, completions, and revenue	High	Sprint 5
41	As an admin, I want to manage course categories and featured course placement	Medium	Sprint 5
42	As a developer, I need an OWASP ZAP scan with zero critical or high findings [30]	High	Sprint 5

6.3 Agile Scrum Ceremonies and Collaboration

Sprint Planning (2–4 hours): At the beginning of each 3-week sprint, planning sessions engage the developer, supervisors, and Advisory Unit stakeholders. Sessions define sprint goals, select user stories from the prioritized Product Backlog based on business value and dependencies, and break them into implementation tasks with effort estimates using story points. The output is a Sprint Backlog of committed work with clear acceptance criteria.

Daily Standup (15 minutes): Adapted to weekly formal checkpoints (Mondays) with supervisors, the standup ceremony addresses three questions: what was completed, what will be done, and what obstacles exist. This maintains transparency and enables rapid unblocking in the solo developer context.

Sprint Review & Demo (1–2 hours): At each sprint's end, completed work is demonstrated to stakeholders, who examine running features, validate acceptance criteria, and provide prioritization feedback for subsequent sprints. The Sprint 3 review constitutes the formal MVP demonstration, confirming core platform viability before proceeding to camera integration and compliance features.

Sprint Retrospective (1 hour): Following the review, the team examines what worked well, what can improve, and what specific action items to carry into the next sprint. Early retrospectives surfaced the need for more comprehensive API documentation and identified infrastructure bottlenecks requiring architectural adjustments.

Product Backlog Refinement (Ongoing): Between sprints, the Product Backlog is continuously refined. New requirements are added, estimated, and prioritized, ensuring Sprint Planning sessions remain efficient and focused on high-value delivery.

6.4 The UML Modeling Language

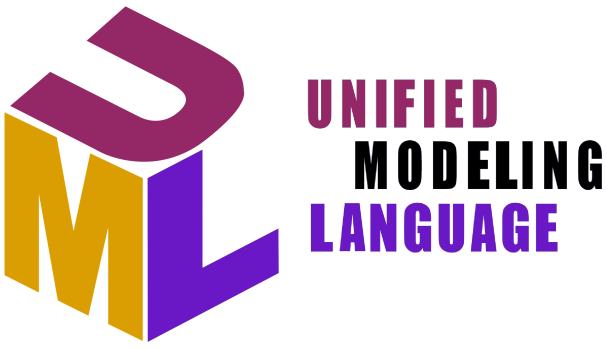


Figure 7: UML Modeling Language

Unified Modeling Language (UML) is a standardized visual modeling language used in software engineering to specify, visualize, construct, and document the artifacts of software systems [31]. UML encompasses two main categories of diagram types: **structural diagrams** (class, component, deployment, and package diagrams) and **behavioral diagrams** (use case, sequence, activity, and state machine diagrams).

For the IoT-REAP platform, UML modeling was applied purposefully across the design phase to document four distinct concerns. **Use case diagrams** were produced to capture system boundaries and actor interactions, including the global use case diagram presented in this chapter and per-feature diagrams developed in subsequent chapters for VM provisioning, hardware reservation, and training path enrollment. **Class diagrams** model the domain entities and their relationships, covering the core data model across the learning, laboratory, and infrastructure domains. **Sequence diagrams** document time-ordered interactions for critical workflows: VM provisioning through the Proxmox API, one-time Guacamole token generation and session establishment, MQTT command dispatch and camera control, and comprehensive audit log entry creation. These models served as architectural contracts between the developer and stakeholders, ensuring shared understanding of system boundaries and interaction protocols before implementation began.

7 Project Planning

The IoT-REAP platform development is organized as a phased timeline from February to end of May 2026, following an Agile Scrum sprint-based approach [25]. The project reaches two key milestones: the MVP demonstration at the conclusion of Sprint 3 and the complete platform delivery at Sprint 5.

7.1 Sprint-Based Development Phases

The development timeline is organized into five thematic sprints of three weeks each. The chronological order reflects a deliberate architectural dependency chain: authentication and audit foundations precede infrastructure integration, which precedes remote access validation, which precedes IoT and camera integration, which precedes the training platform with integrated quality assurance and final delivery. Each sprint embeds its own testing and validation activities, ensuring that quality is built incrementally rather than deferred to a separate phase.

Sprint 1: Foundation, Authentication & Audit (3 weeks)

This phase established the technical baseline and security audit foundation. Laravel 12 and React 18 were integrated via Breeze to provide a full-stack scaffold. Session-based authentication with role-based access control (engineer, teacher, admin) was implemented,

alongside the user database schema using ULID primary keys. Google OAuth social login was integrated as an alternative authentication path, with account linking logic to merge social and local credentials. A key architectural decision in this sprint was the use of ULID primary keys rather than auto-incremented integers: ULIDs provide globally unique, time-sortable identifiers that prevent enumeration attacks and simplify future cross-service data merges. Concurrently, a comprehensive audit log mechanism was implemented: each critical system event - ranging from authentication attempts to resource provisioning - is recorded in a persistent log, capturing the initiator, timestamp, and operation details. Development was performed on Ubuntu 24.04 LTS using Visual Studio Code with PHP and TypeScript language support, with source control managed through Git and GitLab CI/CD pipelines configured for automated linting and build verification. The third week was dedicated to writing unit and integration tests for the authentication flows, role-based access control policies, and audit logging mechanism, establishing

the platform's test suite baseline.

Sprint 2: Proxmox Integration, Multi-Cluster & Network Isolation (3 weeks)

The virtual laboratory backbone was built during this phase. A Proxmox VE API client was developed with retry logic and structured error handling to tolerate transient node failures. The database schema for VM nodes, templates, and sessions was finalized. An engineer-facing VM template catalog with session launch capability was implemented, alongside an admin dashboard displaying live CPU and RAM statistics per node. A load balancer distributing VM provisioning requests across available nodes was completed using a resource-aware scoring algorithm that accounts for current CPU utilization and active session count before selecting a target node. The architecture was extended to support multiple independent Proxmox clusters with encrypted credential storage and cluster selection logic for VM launch. Purdue model network isolation was enforced through VLAN and firewall rule configuration separating IT and OT zones. Unit tests for the Proxmox API client, load balancer scoring algorithm, and cluster selection logic were written and validated as part of sprint acceptance, with integration tests confirming correct behaviour under simulated node failure scenarios.

Sprint 3: Guacamole Remote Access - MVP Demo (3 weeks)

Apache Guacamole was deployed and integrated with the Laravel backend via its REST API. The implementation covered connection object creation, one-time token generation for secure URL distribution, and an embedded React viewer for browser-based RDP and VNC access without client installation. Session lifecycle management - extension in configurable increments, explicit termination with automatic VM cleanup, and persistent connection preferences - was delivered. The third week was used to write unit and integration tests for the Guacamole client service, token generation logic, and session lifecycle workflows, and to run an initial API security review against the newly exposed remote desktop endpoints. **This**

sprint concluded with the MVP demonstration, validating the core value proposition of browser-based remote industrial access.

Sprint 4: Camera, Session Management & Integration Hardening (3 weeks)

The ESP32-CAM hardware layer was integrated to provide live MJPEG video feeds during active sessions, with camera control and stream proxying through the platform gateway. Idle session hibernation was implemented: VMs are automatically suspended after 10 minutes of inactivity, reducing resource contention without terminating the session context. Race conditions in the VM provisioning pipeline surfaced under simultaneous write contention and were resolved through pessimistic locking at the database transaction boundary. Google OAuth account linking edge cases were hardened in parallel. Load testing targeting 15 concurrent logins, 5 active VM provisioning operations, and 10 simultaneous Guacamole sessions was executed in the final week, with results documented against the production-scale objectives defined in the backlog.

Sprint 5: Technical Training Platform & Delivery (3 weeks)

This phase introduced the e-learning dimension in response to stakeholder feedback following the MVP, and concluded with final quality assurance and platform delivery. The course structure - training paths, modules, and training units with prerequisite tracking - was implemented. Instructors gained the ability to upload videos processed into HLS adaptive streams (360p–1080p), create timed quizzes with auto-grading and retry limits, and monitor trainee progress through an analytics dashboard. Engineers could enroll in courses, track completion, take timestamped notes, and receive verifiable digital certificates with unique hash-based verification links. During the final week, an OWASP ZAP security scan was executed to identify and remediate vulnerabilities, user manuals and API reference documentation were finalized, and the defense demonstration was prepared for final delivery. The condensed scope relative to the e-learning feature set was made feasible by the incremental test coverage accumulated across prior sprints, which allowed the final week to focus on security validation, documentation, and demonstration preparation rather than regression testing.

7.2 Gantt Diagram

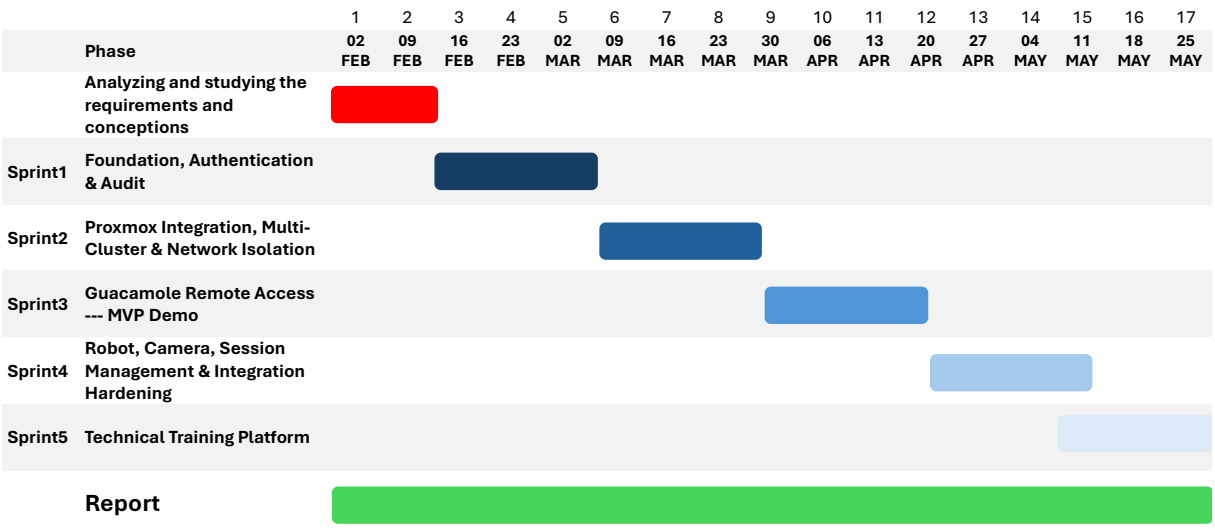


Figure 8: IoT-REAP Platform Development Timeline - Sprint Schedule and Key Milestones

Conclusion

This chapter has established the organizational and academic context of the IoT-REAP project, identified the operational problem motivating its development, and systematically demonstrated - through analysis of four representative commercial products - that no existing industrial remote access solution addresses the combined requirements of browser-native VM provisioning, IoT command dispatch, resource reservation, and integrated training content delivery. The proposed platform architecture, technology selection rationale, and requirements specification were then presented, defining the actors, functional capabilities, and measurable quality constraints that govern the implementation. Finally, the Agile Scrum methodology and five uniform 3-week sprints were introduced, each embedding its own testing and validation activities, providing the project management framework within which the technical work presented in the following chapters was executed.

Chapter 2

Conceptual Study

Introduction

This chapter presents the conceptual study of the IoT-REAP platform. It models the system through use case diagrams and representative sequence diagrams, and presents the class diagram used to guide realization.

1 Use Case Diagram

After identifying actors and requirements, the conceptual analysis of the platform is refined through use case modelling. The implemented application yields one general use case view and several actor-specific refinements. The general model places the platform at the centre and connects the four user actors (Guest, Engineer, Teacher, Administrator) as well as the external Stripe actor [24] to the services they invoke: public discovery, learning progression, content authoring, infrastructure access, commerce, and administrative governance.

1.1 General Use Case Diagram

The general use case diagram groups system behaviour into four principal subsystems. In this model, the Engineer actor generalizes the Guest, retaining public consultation capabilities and adding authenticated learning and virtual laboratory operations. The Teacher actor generalizes the Engineer, extending authenticated access with content authoring and instructional management. The Administrator occupies the supervisory position by controlling both business governance and infrastructure operations. Stripe is modelled as a standalone external actor that interacts with the platform through checkout, webhook callbacks, refunds, and payout-related flows. Figure 9 presents the general use case diagram of the IoT-REAP platform.

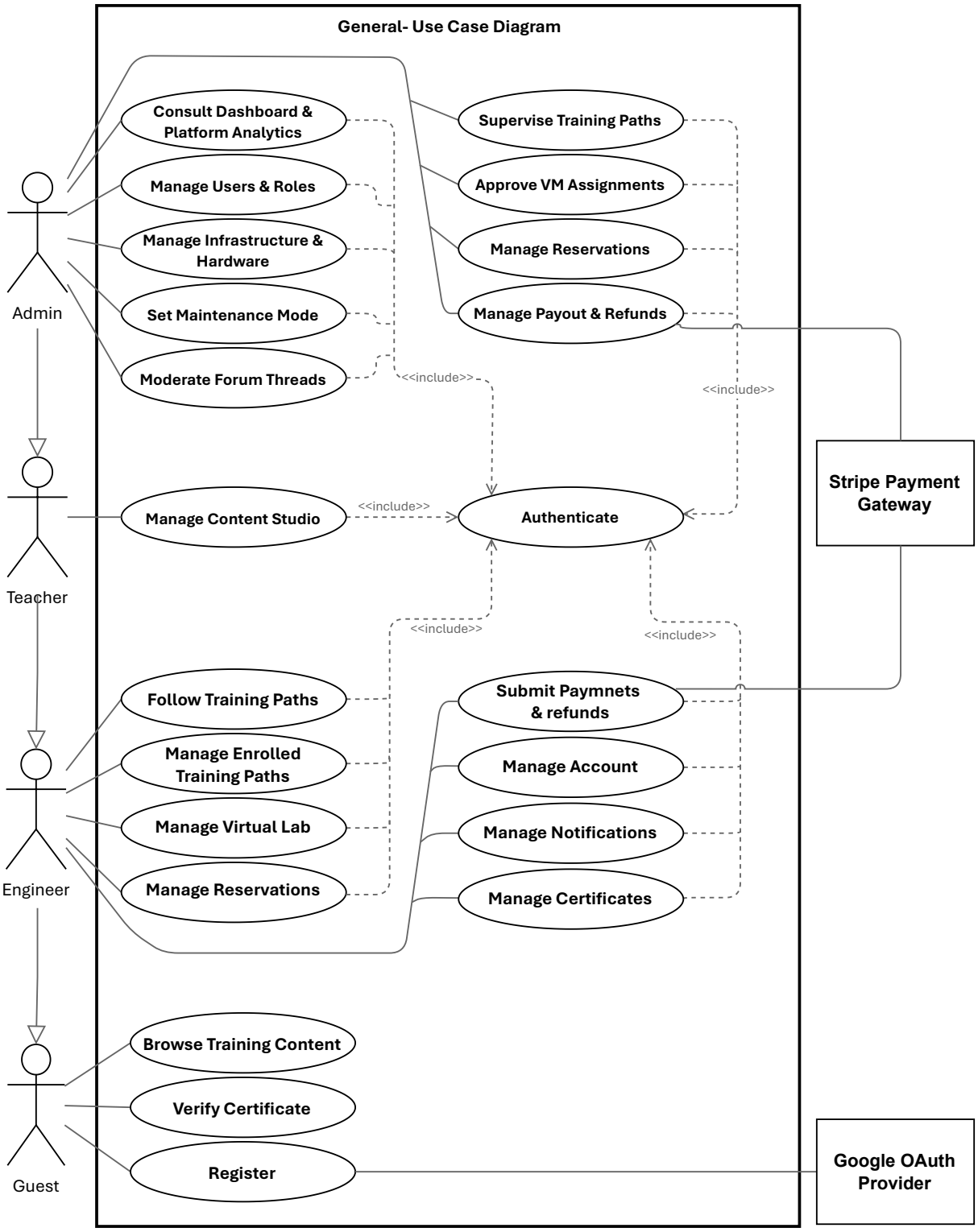


Figure 9: General Use Case Diagram of the IoT-REAP Platform

1.2 Actor-Specific Use Case Refinements

The following subsections present the actor-specific use case refinements derived from the implemented routes and domain behaviours of the platform. Each actor's interactions are specified in terms of preconditions, main flows, and alternate flows, providing the traceability link between requirements and subsequent design decisions.

1.2.1 Refinement of the Use Case Manage Users & Roles

Figure 10 presents the refinement of the Use Case Manage Users & Roles for the Administrator actor. This use case includes the following sub-use cases:

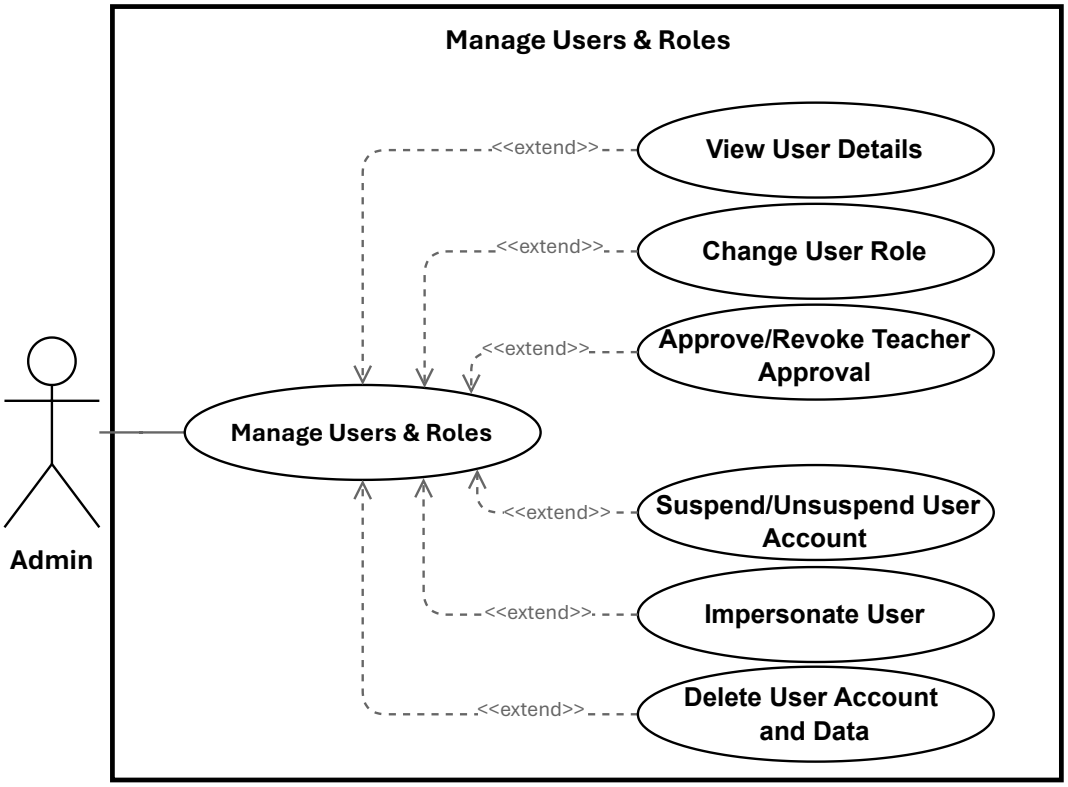


Figure 10: Use Case Manage Users & Roles

1.2.1.1 Specification of the Use Case Change User Role

Table 4: Use Case Specification: Change User Role

Title	Change User Role
Summary	Administrator updates a user's role (engineer, teacher, admin). Assigning `teacher` also records teacher-approval metadata; assigning a non-teacher clears any teacher-approval metadata.
Actor	Administrator
Precondition	Administrator is authenticated and has admin privileges. Target user exists. Requested role is one of: `engineer`, `teacher`, `admin`.
Post-condition	The user's role is persisted. If role = `teacher`, `teacher_approved_at` and `teacher_approved_by` are set; otherwise those fields are cleared. The change is logged and the updated user is returned.
Basic Scenario	1. Administrator opens the user management list or the target user's detail page. 2. Administrator selects a new role and submits the change. 3. System validates request authorization and that the role value is allowed. 4. System verifies the administrator is not attempting to change their own role. 5. System prepares the update payload (including teacher-approval metadata when applicable). 6. System persists the update to the database. 7. System logs the role change for audit purposes. 8. System returns the updated user record to the admin UI and displays confirmation.
Error Scenario	1. If the administrator attempts to change their own role, the system rejects the request and returns an error. 2. If the submitted role is invalid, the system returns a validation error and no update is made. 3. If persistence or service errors occur, the system aborts the update, logs the failure, and returns an error. 4. If the administrator is not authorized, the system returns an authorization error (403).

1.2.1.2 Specification of the Use Case Approve/Revoke Teacher Approval

Table 5: Use Case Specification: Approve/Revoke Teacher Approval

Title	Approve/Revoke Teacher Approval
Summary	Administrator approves a teacher account to mark it as authorized, or revokes an existing teacher approval to remove that authorization. Approving records teacher-approval metadata; revoking clears it.
Actor	Administrator
Precondition	Administrator is authenticated with admin privileges. Target user exists and has the `teacher` role.
Post-condition	If approved, `teacher_approved_at` and `teacher_approved_by` are set. If revoked, those fields are cleared. The change is logged and the updated user record is returned.
Basic Scenario	1. Administrator opens the teacher account details in the user management section. 2. Administrator selects *Approve Teacher* or *Revoke Teacher Approval*. 3. System validates that the target account is a teacher and that the request is authorized. 4. System prepares the update payload for the selected action. 5. System persists the approval change to the database. 6. System logs the action for audit purposes. 7. System returns the updated user record to the admin UI and displays confirmation.
Error Scenario	1. If the target user is not a teacher, the system returns a validation error and no change is made. 2. If the administrator is not authorized, the system returns an authorization error (403). 3. If persistence fails, the system aborts the update, logs the failure, and returns an error.

1.2.1.3 Specification of the Use Case Suspend/Unsuspend User Account

Table 6: Use Case Specification: Suspend / Unsuspend User Account

Title	Suspend / Unsuspend User Account
Summary	Administrator temporarily blocks or restores user access. May specify a reason and optional expiration date for temporary suspensions. All actions are recorded for audit.
Actor	Administrator
Precondition	Administrator is authenticated and authorized to manage accounts. Target user exists.
Post-condition	User access status is updated (suspended or active). Metadata (reason, expiration) is recorded. Action is logged for audit. If suspended, user can no longer log in. If unsuspended, access is restored.
Basic Scenario	1. Administrator opens user management and selects an account. 2. Administrator chooses Suspend Account or Restore Access. 3. For suspension: Administrator optionally provides a reason and expiration date. 4. System validates administrator authorization. 5. System checks that target user exists and is eligible (e.g., not an administrator). 6. System updates account status and records metadata. 7. System blocks or restores access immediately. 8. System logs the action for audit. 9. System notifies user of status change (if configured). 10. Administrator receives confirmation and updated status is displayed.
Error Scenario	1. If administrator lacks permission, request is denied and authorization error is displayed. 2. If target user does not exist, system displays error and does not proceed. 3. If target is an administrator, system prevents suspension and displays error. 4. If action cannot be completed due to system errors, system rolls back changes, preserves account state, and displays error. 5. If notifications are configured but unavailable, core action is completed, but system flags notification failure for manual follow-up.

1.2.1.4 Specification of the Use Case Impersonate User

Table 7: Use Case Specification: Impersonate User

Title	Impersonate User
Summary	A privileged administrator or support agent temporarily assumes a target user's identity for troubleshooting. Impersonation is time-limited, auditable, and constrained by authorization rules.
Actor	Administrator
Precondition	Initiator is authenticated and authorized to impersonate.
Post-condition	A temporary impersonation session is created. Actions performed are applied as the target user but recorded with initiator metadata. Session start and end are logged.
Basic Scenario	1. Administrator initiates impersonation via UI or API, specifying target user ID and mode (read-only/full). 2. System validates and sanitizes request parameters. 3. System verifies the initiator's permission to impersonate the target. 4. System loads the target user record and verifies eligibility (e.g., active status, lower privilege). 5. System creates a time-limited session/token scoped to the target user, annotated with initiator metadata and mode constraints. 6. System logs an immutable audit event for impersonation start (including session ID, mode, and reason). 7. System returns the token or redirects the initiator into the target's session context; initiator acts within the session and actions execute as the target user but include initiator audit metadata. 8. Initiator ends impersonation, or token expires; system revokes the session/token. 9. System logs impersonation end details (session ID, initiator, target, timestamps).
Error Scenario	1. Unauthorized initiator: System returns HTTP 403 and logs a failed attempt. 2. Target user not found or inactive: System returns HTTP 404/400 and logs failure. 3. Business rule violation (e.g., target has equal/higher privilege): System returns HTTP 403 and logs reason. 4. Session or audit logging failure: Impersonation aborts, returns an error, and clears partial session state.

1.2.1.5 Specification of the Use Case Delete User Account and Data

Table 8: Use Case Specification: Delete User Account and Data

Title	Delete User Account and Data
Summary	Administrator removes a user account by anonymizing personal information and removing non-essential data. Payment records preserved for audit.
Actor	Administrator
Precondition	Administrator is authenticated and authorized. Target user exists and is not an administrator. Administrator is not deleting own account.
Post-condition	Account removed. Personal information anonymized. Credentials revoked. Non-essential data removed. Payment history preserved. Deletion audited.
Basic Scenario	1. Administrator locates target account. 2. Administrator selects user and chooses *Delete Account*. 3. System displays confirmation dialog with deletion details. 4. Administrator confirms deletion. 5. System verifies target is not an administrator and not self. 6. System anonymizes personal information. 7. System removes credentials and session tokens. 8. System removes non-essential data. 9. System preserves financial records. 10. System records deletion in audit logs. 11. System displays success and removes user from active list.
Error Scenario	1. If administrator lacks permission, system rejects request and displays authorization error. 2. If target user is an administrator or administrator attempts to delete own account, system prevents deletion and displays error. 3. If target user does not exist, system displays error and does not proceed. 4. If deletion fails due to system errors, system rolls back changes, preserves user record, and displays error. 5. If partial cleanup fails, system preserves core deletion but flags incomplete cleanup for manual review.

1.2.2 Refinement of the Use Case Manage Infrastructure and Hardware

Figure 11 presents the refinement of the Use Case Manage Infrastructure and Hardware for the Administrator actor. This use case includes the following sub-use cases:

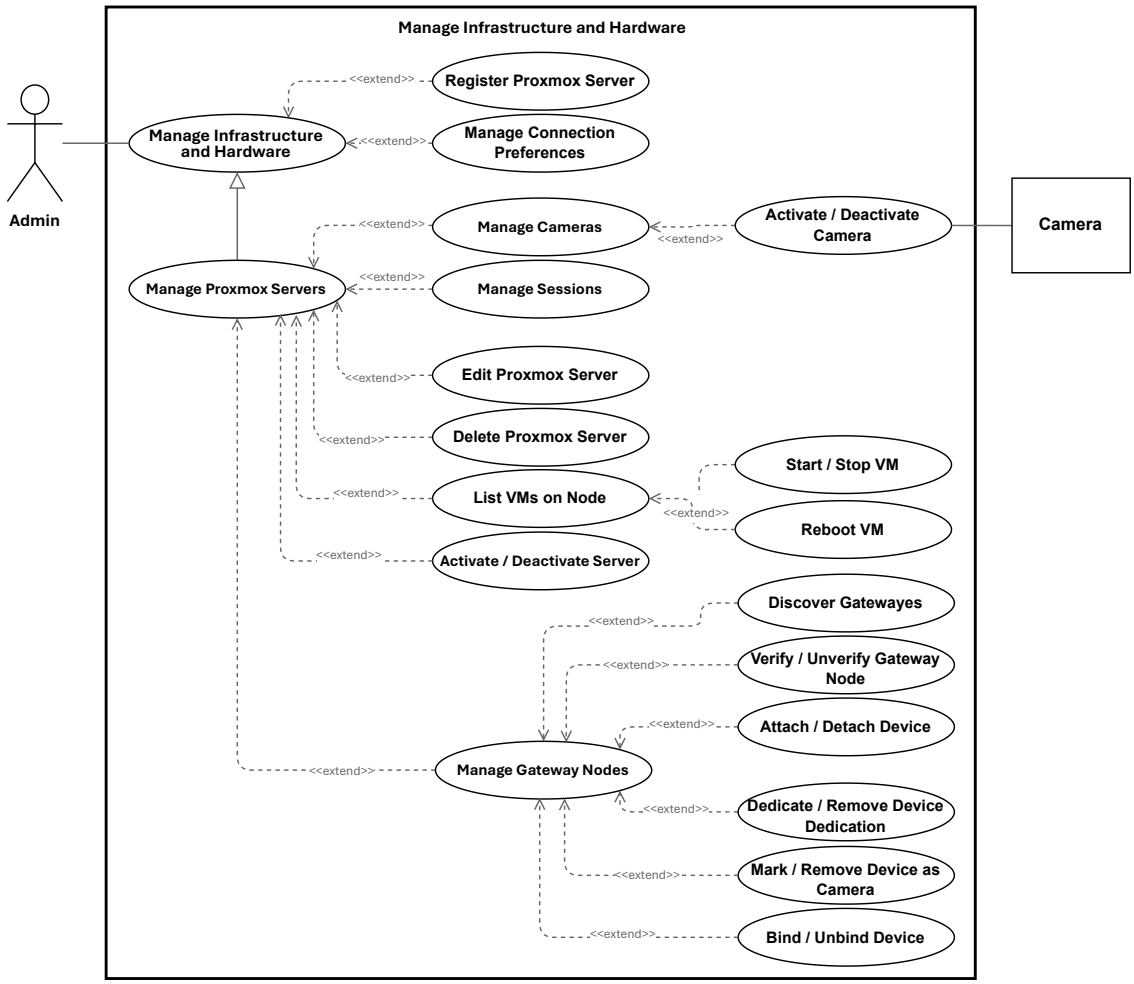


Figure 11: Use Case Manage Infrastructure and Hardware

1.2.2.1 Specification of the Use Case Register Proxmox Server

Table 9: Use Case Specification: Register Proxmox Server

Title	Register Proxmox Server
Summary	Administrator registers a new Proxmox VE server to enable VM provisioning. System validates connectivity and discovers nodes before adding server to inventory.
Actor	Administrator
Precondition	Administrator is authenticated and authorized. Network access to Proxmox server and valid credentials are available.
Post-condition	Proxmox server is registered and appears in inventory. Discovered nodes are available for provisioning. Configuration is securely stored.
Basic Scenario	1. Administrator opens infrastructure management and selects Register New Server. 2. Administrator enters server details: host address, port, credentials, and optional resource configuration (e.g., VM ID ranges). 3. System validates submitted information. 4. System attempts to connect to Proxmox server using provided credentials. 5. System discovers available cluster nodes. 6. System registers server and stores configuration securely. 7. System displays registered server in inventory with discovered nodes. 8. Administrator can now use this server for provisioning.
Error Scenario	1. If submitted information is invalid or incomplete, system rejects request and displays validation error. 2. If system cannot connect to Proxmox server (unreachable, invalid credentials), system displays error and does not register server. 3. If server is already registered, system informs administrator and suggests updating existing entry. 4. If no cluster nodes are discovered, system flags this and allows administrator to review configuration before proceeding. 5. If system error occurs during registration, system displays error and leaves server unregistered.

1.2.2.2 Specification of the Use Case Edit Proxmox Server

Table 10: Use Case Specification: Edit Proxmox Server

Title	Edit Proxmox Server
Summary	Administrator updates the details of an existing Proxmox VE server so the infrastructure inventory remains accurate and usable for provisioning.
Actor	Administrator
Precondition	Administrator is authenticated and authorized. The target Proxmox server exists in the inventory.
Post-condition	The Proxmox server details are updated in the inventory and available for subsequent infrastructure management tasks.
Basic Scenario	1. Administrator opens the server details page and selects the edit option. 2. System displays the current server information. 3. Administrator modifies one or more server details. 4. Administrator submits the changes. 5. System validates the submitted information. 6. System updates the server record. 7. System confirms successful update and shows the revised server details.
Error Scenario	1. Validation failure: system rejects incomplete or invalid input and preserves the entered values for correction. 2. Unauthorized access: system denies the request if the Administrator is not permitted to edit the server. 3. Server not found: system informs the Administrator that the selected server is unavailable. 4. Update failure: system informs the Administrator that the changes could not be saved and keeps the original data unchanged.

1.2.2.3 Specification of the Use Case Delete Proxmox Server

Table 11: Use Case Specification: Delete Proxmox Server

Title	Delete Proxmox Server
Summary	Administrator removes a registered Proxmox VE server from the infrastructure inventory when it is no longer needed.
Actor	Administrator
Precondition	Administrator is authenticated and authorized to manage infrastructure. The target Proxmox server exists in the inventory.
Post-condition	The Proxmox server is removed from the inventory and is no longer available for provisioning.
Basic Scenario	1. Administrator selects a Proxmox server and chooses Delete. 2. System asks the Administrator to confirm the deletion. 3. Administrator confirms the deletion. 4. System checks that the server can be deleted. 5. System removes the server from the inventory. 6. System confirms successful deletion.
Error Scenario	1. Deletion canceled: Administrator closes the dialog or selects Cancel; no action is taken. 2. Server not eligible for deletion: system informs the Administrator that the server cannot be removed. 3. Server not found: system informs the Administrator that the selected server is unavailable. 4. Unauthorized access: system denies the request if the Administrator is not permitted to delete infrastructure entries. 5. Deletion failure: system informs the Administrator that the server could not be deleted and keeps the current state unchanged.

1.2.2.4 Specification of the Use Case Manage Connection Preferences

Table 12: Use Case Specification: Manage Connection Preferences

Title	Manage Connection Preferences
Summary	Administrator creates, updates, tests, or deletes connection preferences (endpoints, ports, protocols, and credentials) for external platform services (Proxmox, Guacamole, MQTT, Gateway). Preferences are stored securely, auditable, and optionally tested for connectivity.
Actor	Administrator
Precondition	Administrator authenticated and authorized; SecretStore available; UI or API accessible; network access to target service if testing is requested.
Post-condition	Preference persisted with secrets stored securely (encrypted); audit record created; test result recorded if requested; dependent processes (e.g., node discovery) triggered when applicable.
Basic Scenario	1. Administrator opens the Connection Preferences editor and submits a create or update request. 2. Controller validates input and forwards to Service. 3. Service encrypts credentials via SecretStore and persists metadata to the repository. 4. If requested, Service invokes an ExternalClient to test connectivity and records the result. 5. Service writes an audit entry (redacting secrets) and returns the persisted preference and test outcome. 6. Controller responds to the Administrator with success and updated resource.
Error Scenario	1. Validation failure: Controller returns HTTP 422; no changes persisted. 2. Unauthorized: Controller returns HTTP 403; attempt logged. 3. SecretStore/repository failure: Service rolls back and returns HTTP 500. 4. Connection test failure: Service records failed test and returns details; persistence may be allowed or rejected per policy.

1.2.2.5 Specification of the Use Case Discover Gateway Nodes

Table 13: Use Case Specification: Discover Gateway Nodes

Title	Discover Gateway Nodes
Summary	Administrator initiates a discovery scan to find all available gateway nodes on the registered Proxmox infrastructure and checks their online status.
Actor	Administrator
Precondition	Administrator is authenticated and authorized to manage infrastructure. At least one Proxmox server is registered and reachable.
Post-condition	System identifies all available gateway nodes in the infrastructure. Each discovered node's online/offline status is verified. A list of discovered nodes with their status is displayed to the administrator.
Basic Scenario	1. Administrator opens the infrastructure management interface. 2. Administrator initiates the *Discover Gateway Nodes* operation. 3. System queries all registered Proxmox servers for available gateway nodes. 4. System checks the online status of each discovered node. 5. System displays a list of discovered nodes, including their names, IP addresses, and current status (online/offline). 6. Administrator can review the list and register newly discovered nodes or update existing entries.
Error Scenario	1. If the administrator lacks permission, the request is denied and an authorization error is displayed. 2. If no Proxmox servers are registered, the system informs the administrator and does not attempt discovery. 3. If a Proxmox server is unreachable during discovery, the system skips it and continues with remaining servers. 4. If no gateway nodes are found, the system displays an empty result and suggests verifying the infrastructure setup. 5. If the discovery operation fails due to a system error, the system displays an error message with any partial results found before the failure.

1.2.2.6 Specification of the Use Case Dedicate / Remove Device Dedication

Table 14: Use Case Specification: Dedicate / Remove Device Dedication

Title	Dedicate / Remove Device Dedication
Summary	Administrator permanently assigns a USB device to a specific VM for automatic attachment when sessions are created on that VM, or revokes the assignment.
Actor	Administrator
Precondition	Administrator is authenticated and authorized to manage hardware. The USB device exists in the inventory. The target Proxmox server and VM are valid and reachable.
Post-condition	1. (Dedicate) The device is bound to the target VM. Any current attachment or gateway binding is cleared. The device is ready for automatic attachment to future sessions on that VM. Any linked camera is assigned to the VM. 2. (Remove Dedication) The device is no longer bound to any specific VM. Any linked camera assignment is cleared.
Basic Scenario	1. Administrator navigates to the device details in the admin interface. 2. Administrator selects either *Dedicate to VM* or *Remove Dedication*. 3. For dedication: Administrator specifies the target Proxmox server and VM. 4. System validates the request and checks that the device is not actively in use. 5. System applies the dedication (or removes it) and confirms success. 6. System displays the updated device status and binding information.
Error Scenario	1. If the administrator lacks permission, the request is rejected and an authorization error is displayed. 2. If the specified VM or server does not exist, the system rejects the request and asks for valid input. 3. If the device is currently attached to an active session, the system prevents the operation and instructs the user to detach it first. 4. If the operation fails (e.g., due to unavailable services), the system displays an error and leaves the device in its previous state.

1.2.2.7 Specification of the Use Case Manage Sessions

Table 15: Use Case Specification: Manage Sessions

Title	Manage Sessions
Summary	Administrator manages VM sessions: view, extend, or terminate. Actions are auditable and may trigger cleanup (Guacamole, Proxmox, hardware).
Actor	Administrator (primary). System services: VMSessionController, VMSessionCleanupService, ExtendSessionService, TerminateVMJob, GuacamoleClient, ProxmoxClient, GatewayService, CameraService (secondary).
Precondition	Administrator is authenticated and authorised. Target session exists and is manageable. Guacamole/Proxmox services are reachable.
Post-condition	On success: session expiry is updated (extend) or session is ended and resources are released (terminate). Guacamole connections are removed and devices/camera controls are released. Actions are audited.
Trigger	Administrator opens session management UI or selects Extend/Terminate on a session.
Basic Scenario	1. Administrator opens session view. 2. System expires overdue sessions and lists active sessions. 3. Administrator selects a session and views details. 4. Administrator chooses Extend or Terminate. 5. System validates authorization and session state. 6. If extend: system applies rules, updates `expires_at`, and returns updated session. 7. If terminate: system deletes Guacamole connection, releases camera controls and USB attachments, optionally stops or reverts the VM, and marks session as ended. 8. System refreshes the list and displays confirmation.
Error Scenario	1. If unauthorized, system returns HTTP 403 and logs the attempt. 2. If the session is already ended, system returns no-op or validation error. 3. If extension violates quota or time-window rules, system returns validation error and does not update expiry. 4. If termination cleanup fails (Guacamole/Proxmox/Hardware), system logs the error, retries according to job policy, and returns a warning; the session is force-marked as ended for later reconciliation when safe.

1.2.3 Refinement of the Use Case Manage Content Studio

Figure 12 presents the refinement of the Use Case Manage Content Studio for the Teacher actor. This use case includes the following sub-use cases:

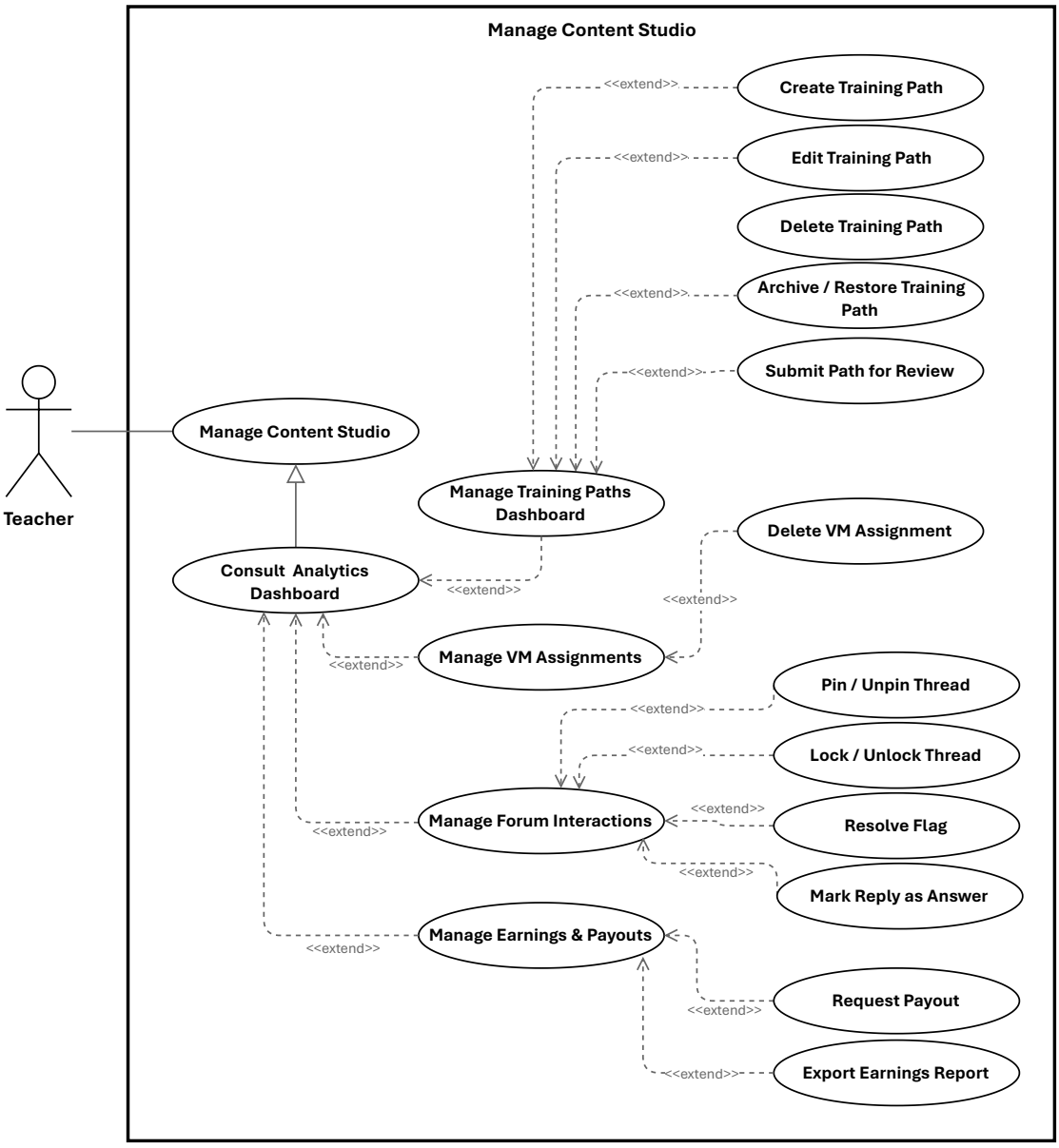


Figure 12: Use Case Manage Content Studio

1.2.3.1 Specification of the Use Case Create Training Path

Table 16: Use Case Specification: Create Training Path

Title	Create Training Path
Summary	Teacher creates a new training path by providing the basic information needed to define the learning content.
Actor	Teacher
Precondition	Teacher is authenticated and authorized to create training content.
Post-condition	A new training path is created and associated with the Teacher in a draft state, ready for further editing.
Basic Scenario	1. Teacher selects the option to create a new training path. 2. System displays the creation form. 3. Teacher enters the required information and any optional details. 4. Teacher submits the form. 5. System validates the submitted information. 6. System creates the new training path. 7. System confirms successful creation and presents the training path for further editing.
Error Scenario	1. Validation failure: system rejects invalid or incomplete input and preserves the entered values for correction. 2. Unauthorized access: system denies the action if the Teacher is not allowed to create training content. 3. Creation failure: system informs the Teacher that the training path could not be created and allows retrying later.

1.2.3.2 Specification of the Use Case Edit Training Path

Table 17: Use Case Specification: Edit Training Path

Title	Edit Training Path
Summary	Teacher updates the details of an existing training path so its content, description, and related information remain accurate and aligned with the teaching plan.
Actor	Teacher
Precondition	Teacher is authenticated, owns the target training path, and the path is available for editing.
Post-condition	The training path details are updated and the Teacher can view the revised information. On failure, the original information remains unchanged.
Basic Scenario	1. Teacher opens the training path editor. 2. System displays the current training path information. 3. Teacher modifies one or more editable fields. 4. Teacher submits the changes. 5. System validates the submitted information. 6. System updates the training path. 7. System confirms successful update and shows the revised details.
Error Scenario	1. Validation failure: system rejects incomplete or invalid input and preserves the entered values for correction. 2. Unauthorized access: system denies the request if the Teacher is not allowed to edit the training path. 3. Training path not found: system informs the Teacher that the requested path is unavailable. 4. Update failure: system informs the Teacher that the changes could not be saved and keeps the original data unchanged.

1.2.3.3 Specification of the Use Case Delete Training Path

Table 18: Use Case Specification: Delete Training Path

Title	Delete Training Path
Summary	Teacher permanently removes one of their own training paths after confirming the action.
Actor	Teacher
Precondition	Teacher is authenticated, owns the target training path, and the path is eligible for deletion.
Post-condition	The training path is removed and no longer appears in the teacher's active listings.
Basic Scenario	1. Teacher selects a training path and chooses Delete. 2. System asks the Teacher to confirm the deletion. 3. Teacher confirms the deletion. 4. System verifies that the Teacher is allowed to delete the training path. 5. System removes the training path from active listings. 6. System confirms successful deletion.
Error Scenario	1. Deletion canceled: Teacher closes the dialog or selects Cancel; no action is taken. 2. Training path not eligible for deletion: system informs the Teacher that the path cannot be deleted. 3. Training path not found: system informs the Teacher that the requested path is unavailable. 4. Unauthorized access: system denies the request if the Teacher does not own the training path. 5. Deletion failure: system informs the Teacher that the path could not be deleted and keeps the current state unchanged.

1.2.3.4 Specification of the Use Case Archive / Restore Training Path

Table 19: Use Case Specification: Archive / Restore Training Path

Title	Archive / Restore Training Path
Summary	Teacher archives a training path to hide it from active listings while preserving content; later restores it to active listings.
Actor	Teacher
Precondition	Teacher is authenticated and owns the training path; it exists in the system.
Post-condition	Archive: path marked archived, removed from active listings, content preserved. Restore: archived flag cleared, path returns to active listings.
Basic Scenario	1. Teacher selects path and chooses "Archive". 2. System asks for confirmation. 3. Teacher confirms. 4. System verifies ownership and archivable state. 5. System marks path as archived, records audit, and removes from active listings. 6. System confirms success; item no longer appears in active lists. 7. Later, Teacher navigates to archived items and chooses "Restore". 8. System asks for confirmation. 9. Teacher confirms. 10. System verifies ownership and archived state, clears archived status, records audit, and restores to active listings. 11. System confirms success; item appears again in active lists.
Error Scenario	1. If Teacher does not own path, system denies action and displays authorization message. 2. If path is not found or already in requested state, system displays validation message. 3. If system persistence error occurs, system preserves consistent state, logs incident, and advises retry. 4. If auxiliary indexing fails, system completes in primary store, records inconsistency for reconciliation, and informs Teacher that listings may update shortly.

1.2.3.5 Specification of the Use Case Delete VM Assignment

Table 20: Use Case Specification: Delete VM Assignment

Title	Delete VM Assignment
Summary	Teacher deletes an existing VM assignment they created; the system releases any provisioned VM and remote connections, records the action for audit, and notifies relevant parties.
Actor	Teacher (primary). System (secondary).
Precondition	Teacher is authenticated and authorized to manage the assignment; the VM assignment exists and is eligible for deletion.
Post-condition	On success: the assignment is removed (or soft-deleted per policy); associated external resources are released or scheduled for cleanup; an audit entry is recorded; affected users are notified.
Basic Scenario	1. Teacher locates the VM assignment and chooses Delete. 2. System asks the Teacher to confirm the deletion. 3. Teacher confirms the deletion. 4. System verifies the Teacher's authorization and that the assignment may be deleted. 5. System releases or schedules cleanup of any provisioned VM and removes any remote access/connection resources associated with the assignment. 6. System removes (or marks as deleted) the assignment record and records the deletion in the audit log. 7. System notifies the Teacher and any other relevant parties about the deletion and cleanup status. 8. The UI reflects the deletion and updated resource status.
Error Scenario	1. Assignment not found: system informs the Teacher and makes no changes. 2. Unauthorized (not owner or insufficient privileges): system denies the action and explains required permissions. 3. External resource cleanup fails (e.g., VM removal or remote connection deletion): system logs the failure, retries or enqueues a cleanup job, may perform a soft-delete of the assignment per policy, and notifies the Teacher of the partial outcome and next steps. 4. System error during deletion: system preserves consistent state, records the incident for investigation, and advises the Teacher to retry later or contact support.

1.2.3.6 Specification of the Use Case Pin / Unpin Thread

Table 21: Use Case Specification: Pin / Unpin Thread

Title	Pin / Unpin Thread
Summary	Teacher pins or unpins a forum thread within a training path to highlight or remove emphasis. The system updates the thread's visibility and records the moderation action for audit.
Actor	Teacher (primary)
Precondition	Teacher is authenticated and has moderation permission for the training path; the target thread exists and is not deleted.
Post-condition	The thread's pinned state is updated and persisted; forum listings and thread views reflect the change; a moderation audit entry is recorded.
Basic Scenario	1. Teacher opens the forum inbox or thread detail and selects the target thread. 2. Teacher chooses the Pin or Unpin action. 3. System verifies the teacher's authorization and that the thread is in a valid state for moderation. 4. System updates the thread's pinned status and persists the change. 5. System records a moderation audit entry (actor, action, thread identifier, timestamp). 6. System updates forum listings and thread ordering so the change is visible to authorized users. 7. UI displays confirmation and the thread's new pinned state.
Error Scenario	1. Unauthorized: If the teacher lacks moderation permission, the system rejects the action and informs the teacher. 2. Not found / invalid state: If the thread is missing or deleted, the system reports the condition and takes no action. 3. Persistence failure: If the update cannot be saved, the system aborts the change, logs the failure, records an error audit, and notifies the teacher to retry later.

1.2.3.7 Specification of the Use Case Request Payout

Table 22: Use Case Specification: Request Payout

Title	Request Payout
Summary	Teacher requests withdrawal of an amount from available earnings. The system validates amount and payment method according to payout policies, creates a pending payout request, and confirms submission.
Actor	Primary: Teacher. Secondary: System.
Precondition	Teacher is authenticated and has earnings data with a non-zero available balance. Earnings/payout interface is available.
Post-condition	Payout request is created with status pending and associated with the teacher account. System confirms successful submission.
Basic Scenario	1. Teacher opens the earnings section and selects Request Payout. 2. System displays payout form with available balance and payment options. 3. Teacher enters payout amount and selects payment method. 4. Teacher submits the request. 5. System validates fields and checks available balance and minimum threshold. 6. System verifies amount and payment method validity. 7. System creates payout request with pending status and confirms submission to the Teacher.
Error Scenario	1. Amount exceeds available balance: system rejects request and displays a validation message; Teacher may retry with a valid amount. 2. Amount below minimum threshold: system rejects request and prompts Teacher to increase amount; Teacher may retry. 3. Invalid payment method: system rejects request and requests a valid method; Teacher may retry. 4. Persistence or service failure: system preserves previous state, logs the incident, records an error audit, and informs the Teacher to retry later.

1.2.3.8 Specification of the Use Case Export Earnings Report

Table 23: Use Case Specification: Export Earnings Report

Title	Export Earnings Report
Summary	Teacher exports a CSV report of completed earnings transactions for a selected date range so the income history can be reviewed, reconciled, or archived for accounting purposes. The system generates a downloadable file without changing any financial records.
Actor	Primary: Teacher. Secondary: System.
Precondition	Teacher is authenticated and can access the earnings page. Earnings data is available in the system. If a date range is provided, it is valid.
Post-condition	CSV report is generated for the teacher's completed payments within the selected date range. Browser receives streamed download named earnings-YYYY-MM-DD.csv. No financial data is modified.
Basic Scenario	1. Teacher opens the earnings page and requests an export. 2. System resolves the authenticated teacher and determines the export date range (selected or default). 3. System generates a CSV from the teacher's completed payments for the date range. 4. System streams the CSV file to the teacher's browser and the download starts.
Error Scenario	1. Invalid or empty date range: supplied date range is malformed or inverted; system rejects the request and displays a validation message; no download occurs. 2. Export processing failure: data retrieval or CSV generation fails; system aborts the export, preserves current state, logs the incident, and surfaces a recoverable error. 3. No matching payments: selected date range contains no completed payments; system generates a CSV with header row only and streams it successfully.

1.2.4 Refinement of the Use Case Manage Enrolled Training Paths

Figure 13 presents the refinement of the Use Case Manage Enrolled Training Paths for the Engineer actor. This use case includes the following sub-use cases:

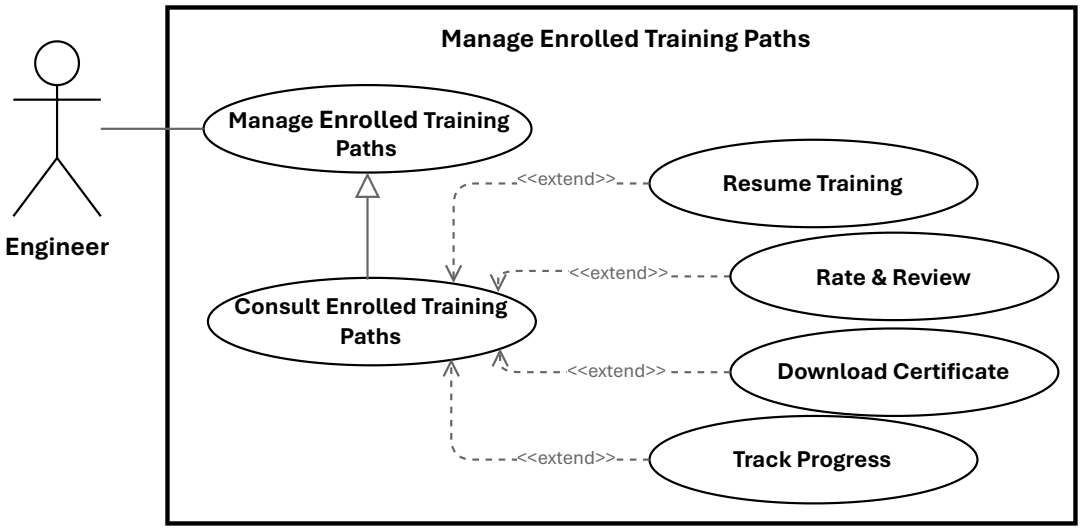


Figure 13: Use Case Manage Enrolled Training Paths

1.2.4.1 Specification of the Use Case Resume Training

Table 24: Use Case Specification: Resume Training

Title	Resume Training
Summary	Learner resumes a previously started training unit or module at the last saved position so they can continue without losing progress.
Actor	Primary: Engineer (Learner). Secondary: System.
Precondition	Learner is authenticated and enrolled in the training path; the platform stores progress for the unit.
Post-condition	On success: learner resumes at the saved position and the system continues tracking and persisting progress.
Basic Scenario	1. Learner clicks “Resume” on the training unit page. 2. System verifies the learner’s authentication status. 3. System verifies that the learner is enrolled and has access to the training unit. 4. System retrieves the last saved progress for the selected unit. 5. System loads the content metadata and associated learning resource. 6. System returns the resume details, including the content location and the starting position. 7. Learner starts playback or continues reading from the saved position. 8. System continues to record progress updates as the learner resumes training.
Error Scenario	1. Authentication failure: session is invalid or expired; the system blocks access and requests re-authentication. 2. Content retrieval failure: the learning resource cannot be loaded; the system displays an error and may allow retry. 3. Progress persistence failure: the system cannot save updates; the previous state is preserved and a recoverable error is shown.

1.2.4.2 Specification of the Use Case Rate & Review

Table 25: Use Case Specification: Rate & Review

Title	Rate & Review
Summary	Engineer submits a star rating and optional written review for a training path. The system validates the submission, stores the review, recalculates the training path's average rating, and applies moderation rules when required.
Actor	Engineer
Precondition	Engineer is authenticated and has access to the training path review interface; the target training path exists.
Post-condition	On success: the review is saved or updated, the training path average rating is recalculated, and the appropriate visibility or approval state is set.
Post-condition (Failure)	No review is stored or updated; the previous state is preserved and an error is reported.
Basic Scenario	1. Engineer opens the review form for a training path. 2. Engineer selects a star rating and optionally enters review text, then submits the form. 3. System validates the engineer's eligibility and the submitted input. 4. System saves the review and recalculates the training path's average rating. 5. If an existing review is detected, the system updates the existing record. 6. System confirms the submission, or marks it as pending if moderation is required.
Error Scenario	1. Missing rating: the system rejects the submission with a validation error. 2. Persistence failure: the system aborts the save, preserves the previous state, logs the error, and notifies the engineer. 3. Moderation required: the system stores the review as pending and hides it until approval.

1.2.4.3 Specification of the Use Case Download Certificate

Table 26: Use Case Specification: Download Certificate

Title	Download Certificate
Summary	Engineer requests and downloads a completion certificate (PDF) for a training path they have completed. The system verifies eligibility, retrieves or generates the certificate, and returns it as a downloadable file while logging the action.
Actor	Primary: Engineer. Secondary: System.
Precondition	The engineer is authenticated; the engineer has completed the training path and a certificate record exists or can be generated.
Post-condition	The engineer receives the certificate file; a download audit record is created; certificate delivery metadata is recorded.
Basic Scenario	1. Engineer requests a certificate download. 2. System authenticates the request and identifies the engineer. 3. System verifies that the engineer is eligible to download the certificate. 4. System checks whether a certificate file already exists. 5. If a file exists, the system retrieves it; otherwise, the system generates the certificate and stores it. 6. System returns the certificate file or a download link to the engineer. 7. The download starts in the browser or client application. 8. System records the successful download in the audit log.
Error Scenario	1. Unauthenticated request: the system denies access and requests re-authentication. 2. Not eligible: the system rejects the request if the engineer is not entitled to the certificate. 3. Certificate generation or storage error: the system aborts the operation, preserves the current state, and logs the failure.

1.2.5 Refinement of the Use Case Manage Virtual Lab

Figure 14 presents the refinement of the Use Case Manage Virtual Lab for the Engineer actor. This use case includes the following sub-use cases:

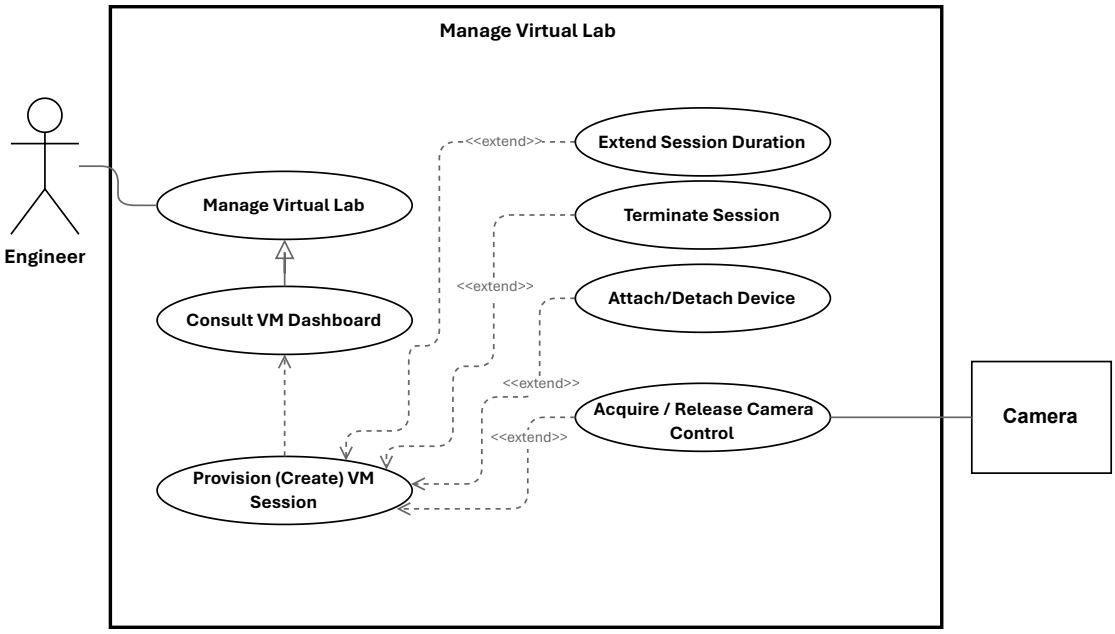


Figure 14: Use Case Manage Virtual Lab

1.2.5.1 Specification of the Use Case Provision (Create) VM Session

Table 27: Use Case Specification: Provision (Create) VM Session

Title	Provision (Create) VM Session
Summary	Engineer requests an ephemeral VM session by selecting a template and duration. The system validates the request, checks quota, provisions a virtual machine, creates remote access, persists the session, and schedules cleanup.
Actor	Engineer
Precondition	Engineer is authenticated and authorized; the requested template exists and is available; the virtualization and remote-access services are reachable; background processing is operational.
Post-condition	A VM session record exists with status active or provisioning; the VM is provisioned with a host and IP address; remote-access details are created and associated; a cleanup job is scheduled.
Basic Scenario	1. Engineer submits a VM session request with a template and duration. 2. System validates the request and authorizes the engineer. 3. System checks whether the engineer can create a new session. 4. System selects an available node for provisioning. 5. System provisions the virtual machine from the selected template. 6. System waits until provisioning completes and retrieves the VM details. 7. System creates and stores the VM session record. 8. System creates the remote-access connection and associates it with the session. 9. System updates the session status to active and schedules cleanup after the requested duration. 10. System returns the created VM session to the engineer.
Error Scenario	1. Quota exceeded: the system rejects the request and displays an explanatory message. 2. Provisioning failure: the system aborts the operation, attempts cleanup, and marks the session as failed. 3. Persistence failure: the system attempts to roll back the created VM and reports an error. 4. Remote-access setup failure: the system records a partial failure and either schedules remediation or rolls back according to policy.

1.2.5.2 Specification of the Use Case Extend Session Duration

Table 28: Use Case Specification: Extend Session Duration

Title	Extend Session Duration
Summary	Engineer requests additional time for an active VM session. The system validates eligibility, checks quota or billing constraints, updates the session expiry, reschedules cleanup, and notifies the engineer.
Actor	Engineer. Secondary: System services.
Precondition	Engineer is authenticated and owns an active VM session; session information is available and the quota or billing service can be consulted.
Post-condition	On success: the session expiry is updated and saved; the cleanup job is rescheduled; quota or billing counters are updated; the engineer is notified and an audit entry is created.
Post-condition (Failure)	No expiry change is made; quota or billing state remains unchanged; the engineer receives an explanatory error and the incident is logged.
Basic Scenario	1. Engineer requests an extension for an active VM session. 2. System authenticates the request and verifies the engineer's ownership of the session. 3. System checks that the session is active and eligible for extension. 4. System verifies quota or billing authorization. 5. If the request is valid, system updates the session expiry, reschedules cleanup, records an audit entry, and notifies the engineer. 6. System returns a success response with the updated session information.
Error Scenario	1. Session not found or not owned: the system rejects the request and returns an authorization or not-found error. 2. Session expired or terminated: the system rejects the request and indicates that a new session must be created. 3. Quota or billing failure: the system rejects the extension request. 4. Persistence or scheduler failure: the system rolls back the update if possible, logs the error, and raises an operational alert.

1.2.5.3 Specification of the Use Case Acquire / Release Camera Control

Table 29: Use Case Specification: Acquire / Release Camera Control

Title	Acquire / Release Camera Control
Summary	Engineer acquires exclusive PTZ control of a laboratory camera and later releases it so other users may use the device. The system manages leases, queues, and notifications to coordinate shared access.
Actor	Primary: Engineer. Secondary: Other engineers/users, System Administrator (override), NotificationService, CameraDevice.
Precondition	Engineer is authenticated, has an active session, and the camera is online with PTZ capabilities; control-state tracking is available.
Post-condition	On success: ownership is assigned with a lease expiry; PTZ commands are relayed and acknowledged; on release or lease expiry the camera becomes available and subscribers are notified.
Basic Scenario	1. Engineer requests control of camera C. 2. System verifies authentication, session, and device capabilities. 3. If available, system reserves control and issues a lease with an expiry. 4. System returns success; UI enables PTZ controls and broadcasts a ControlGranted notification. 5. While holding control, the engineer sends PTZ commands which the system relays to the device and acknowledges. 6. Engineer releases control or lease expires; system clears ownership, marks the camera available, and broadcasts ControlReleased.
Error Scenario	1. Camera offline or unreachable: request fails and no ownership change occurs. 2. Unsupported capability: acquire is rejected and PTZ controls remain disabled. 3. Persistence/write failure: system attempts rollback, logs the error, and alerts operations; ownership remains unchanged. 4. Camera busy: system returns BUSY; UI may offer a queue option and the user can re-enter the flow when promoted.

2 Class Diagram

Figure 15 presents the class diagram of the IoT-REAP platform.

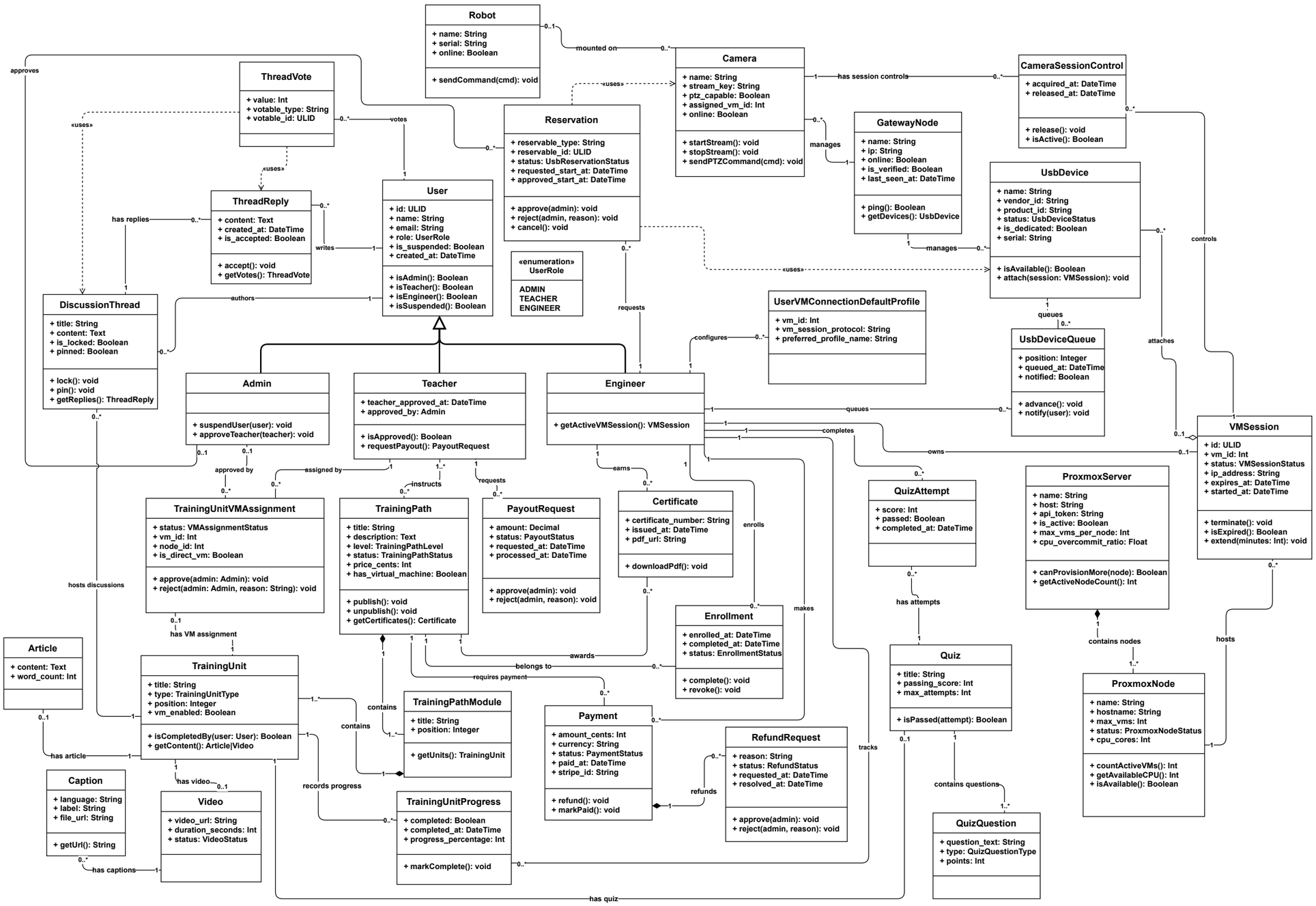


Figure 15: Class Diagram of the IoT-REAP Platform

The class diagram encompasses the full domain model of the IoT-REAP platform. The classes are organised into five functional clusters: *user management* (User, Admin, Teacher, Engineer, UserRole), *learning content* (TrainingPath, TrainingPathModule, TrainingUnit and its subtypes, Quiz, Enrollment, Certificate), *infrastructure* (VMSession, ProxmoxServer, ProxmoxNode, GatewayNode, Robot), *hardware resources* (UsbDevice, Camera, Reservation, UsbDeviceQueue), and *financial operations* (Payment, RefundRequest, PayoutRequest). The following paragraphs describe each class individually.

Class User: The class User represents a general platform user and serves as the base entity for all person-like actors. It contains identifying and contact information (such as ULID, name and email), authentication attributes, role membership and suspension flags, and timestamps. User encapsulates shared behaviour and relationships that are common to specialized user types.

Class Admin: The class Admin represents platform administrators with elevated privileges. Admins are responsible for system governance tasks such as user suspension, quota management, teacher approvals, and oversight of infrastructure and financial operations.

Class Teacher: The class Teacher models instructional staff who author and manage training content. Teachers have approval metadata, authoring capabilities, and interfaces for requesting payouts and reviewing learner submissions.

Class Engineer: The class Engineer represents practitioners who consume training materials and operate remote laboratory resources. Engineers interact with VM sessions and hardware reservations.

Class UserRole: An enumeration that defines the possible user roles (ADMIN, TEACHER, ENGINEER). UserRole is used to gate permissions and to differentiate behaviour and available features at runtime.

Class TrainingPath: The class TrainingPath models a purchasable or enrollable course consisting of a sequence of modules and units. It stores metadata such as title, description, level, pricing and publication status.

Class TrainingPathModule: TrainingPathModule groups related TrainingUnit objects into a coherent module or chapter within a TrainingPath. Modules provide ordering, summarization and module-level metadata.

Class TrainingUnit: The class TrainingUnit represents an atomic learning unit (for example a lesson or exercise). A unit may be of various types (Article, Video, Quiz) and tracks progress, assignment status and completion criteria.

Class Article: Article is a content subtype representing textual learning material, including body content, attachments and metadata for rendering.

Class Video: Video is a content subtype representing embedded or hosted video media used within a TrainingUnit.

Class Quiz: The Quiz class models assessment units composed of questions. It references QuizQuestion objects and aggregates attempts and scoring rules.

Class QuizQuestion: QuizQuestion represents individual questions within a Quiz, including question text, possible answers and correct answer metadata.

Class QuizAttempt: QuizAttempt records a learner's attempt at a Quiz, including answers chosen, score achieved and timestamps.

Class DiscussionThread: DiscussionThread models a forum-style discussion attached to a TrainingUnit. It aggregates posts and provides moderation and thread metadata.

Class ThreadReply: ThreadReply represents an individual reply within a DiscussionThread; replies may be accepted, voted on, and linked to their authors.

Class Enrollment: Enrollment links a User to a TrainingPath, capturing enrolment date, progress, and entitlement to access path materials and resources.

Class TrainingUnitProgress: TrainingUnitProgress tracks a learner's status within a specific TrainingUnit (e.g., not started, in progress, completed) and records timestamps and completion evidence.

Class Certificate: Certificate models credential issuance after successful completion of a TrainingPath or assessment; it contains recipient data, issuance date and validation metadata.

Class VMSession: VMSession represents a provisioned virtual machine session used by Engineers. It captures VM identifiers, node allocation, status, expiry time and associations to user reservations, Guacamole connections and cleanup jobs.

Class TrainingUnitVMAssignment: This class links TrainingUnits to VM templates or assignments that require a live virtual machine; it stores assignment status, VM identifiers and approval metadata.

Class UserVMConnectionDefaultProfile: UserVMConnectionDefaultProfile captures per-user connection preferences and default profiles for remote desktop or terminal connections to provisioned VMs.

Class ProxmoxServer: ProxmoxServer represents the Proxmox deployment as a logical entity, containing configuration references, API endpoints and a set of ProxmoxNode objects.

Class ProxmoxNode: ProxmoxNode models an individual compute node within the Proxmox cluster; nodes host VM instances and expose resource metrics used by the scheduler and load balancer.

Class GatewayNode: GatewayNode models infrastructure gateways (for example a dedicated VM or host that proxies hardware or network access) and coordinates services such as USB/IP and camera streaming.

Class UsbDevice: UsbDevice models physical USB devices available for reservation and attachment to VMs, tracking device identifiers, availability status and reservation links.

Class Camera: Camera represents camera hardware in the inventory, including streaming endpoints, supported formats and reservation and attachment state.

Class CameraSessionControl: CameraSessionControl encapsulates session-level control for cameras (attach/detach, format negotiation and stream lifecycle) when cameras are assigned to a VM session.

Class Reservation: Reservation is a polymorphic record representing a requested allocation of a reservable resource (USB device, camera, VM slot). It stores requested and approved time windows, status and references to the reservable entity.

Class UsbDeviceQueue: UsbDeviceQueue models the waiting queue for an unavailable USB device, enabling FIFO assignment and notifications when a device becomes available.

Class Robot: Robot models a remote hardware unit that serves as a container for laboratory cameras; it includes identifiers, status, and network addressing.

Class Payment: Payment records financial transactions (amount, currency, status, user and training path references) and serves as the source for refund and payout workflows.

Class RefundRequest: RefundRequest models a user-initiated refund claim, including reason, linked payment, eligibility checks and administrative review status.

Class PayoutRequest: PayoutRequest represents teacher-initiated requests to withdraw earned funds; it includes payout amount, recipient details and approval state.

3 Sequence Diagrams

To complement the class diagram, this section presents sequence diagrams for key IoT-REAP workflows: VM session provisioning and termination, camera control, user authentication, and training unit progression. These diagrams illustrate interactions between Controllers, Services, external APIs (Proxmox, Guacamole, MQTT broker), and frontend Views, linking the system architecture to its implementation in Chapter 3.

3.1 Manage Session Sequence

Figure 16 illustrates the manage session sequence.

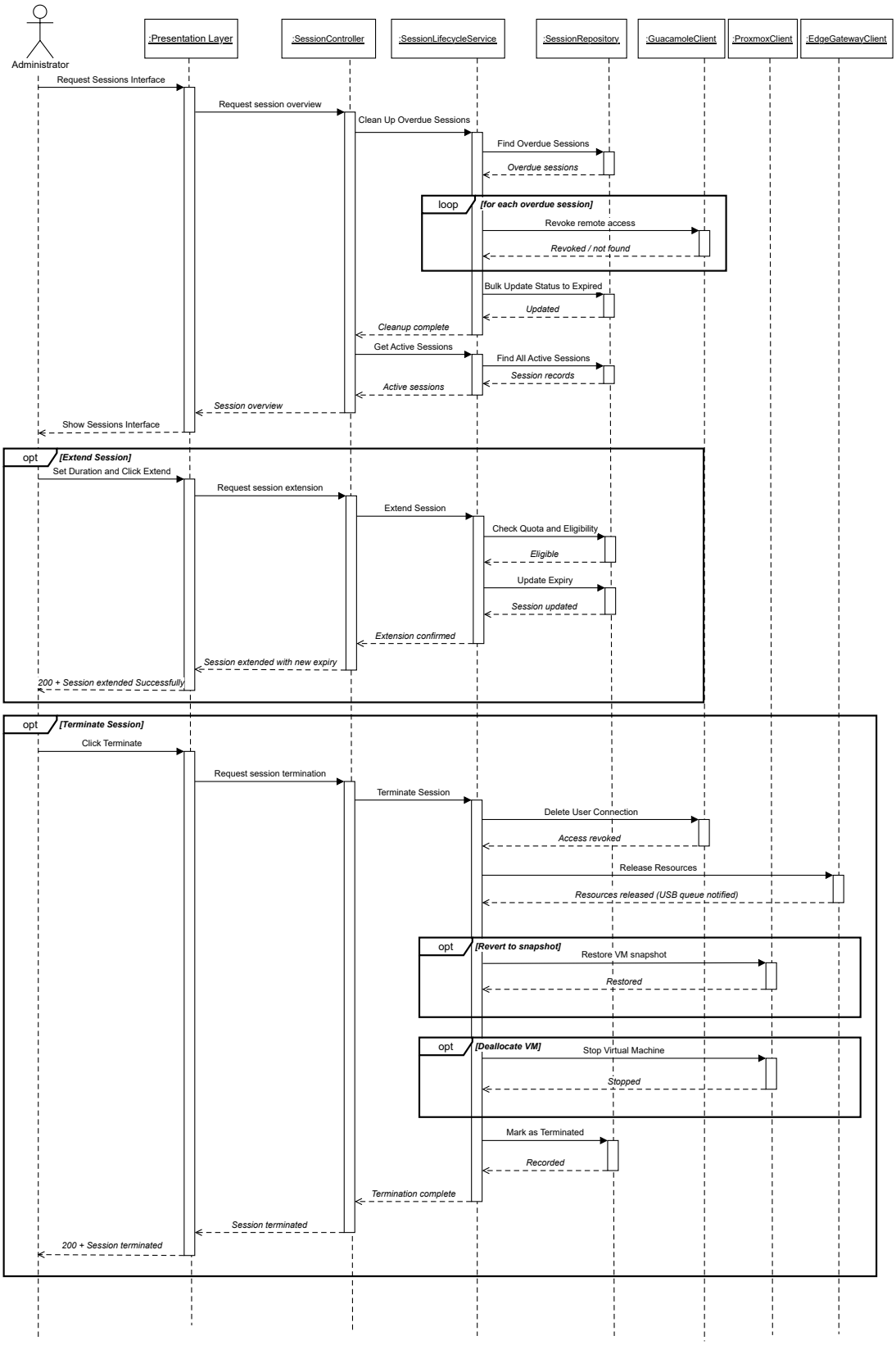


Figure 16: Sequence Diagram: Manage Session

3.2 Discover Gateway Node Sequence

Figure 17 illustrates the discover gateway node sequence.

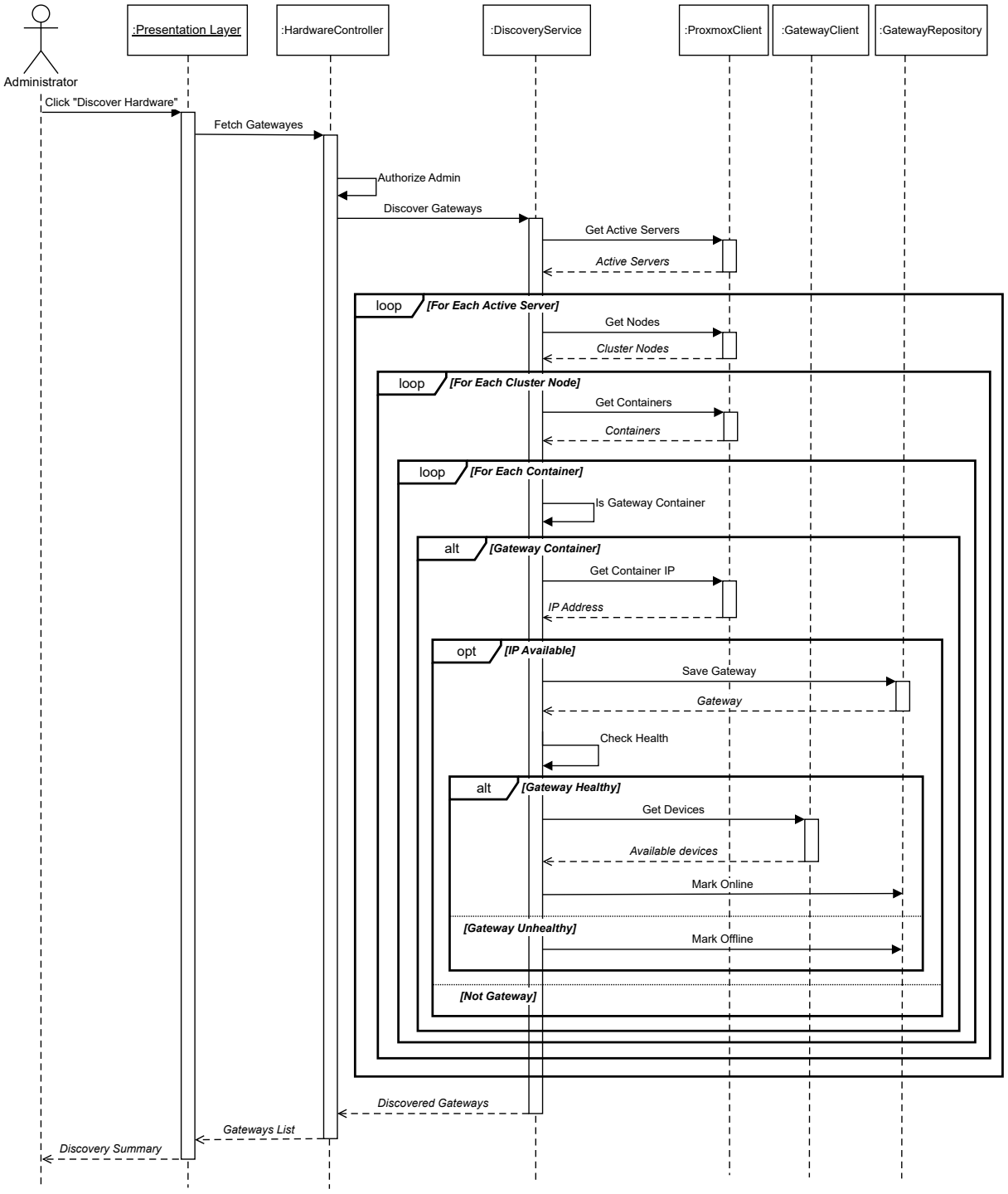


Figure 17: Sequence Diagram: Discover Gateway Node

3.3 Payout Request Sequence

Figure 18 illustrates the payout request sequence.

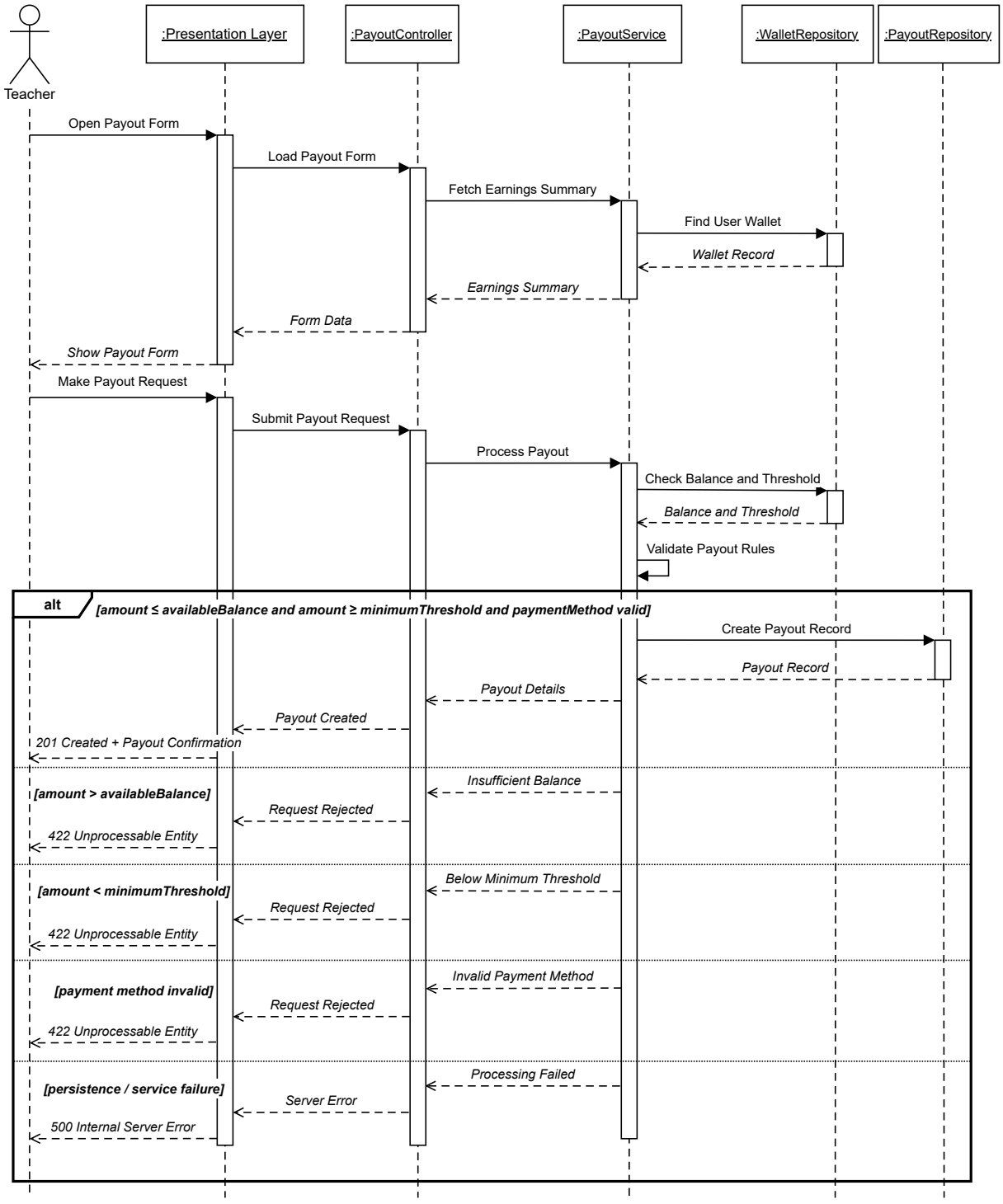


Figure 18: Sequence Diagram: Payout Request

3.4 Provision VM Session Sequence

Figure 19 illustrates the provision VM session sequence.

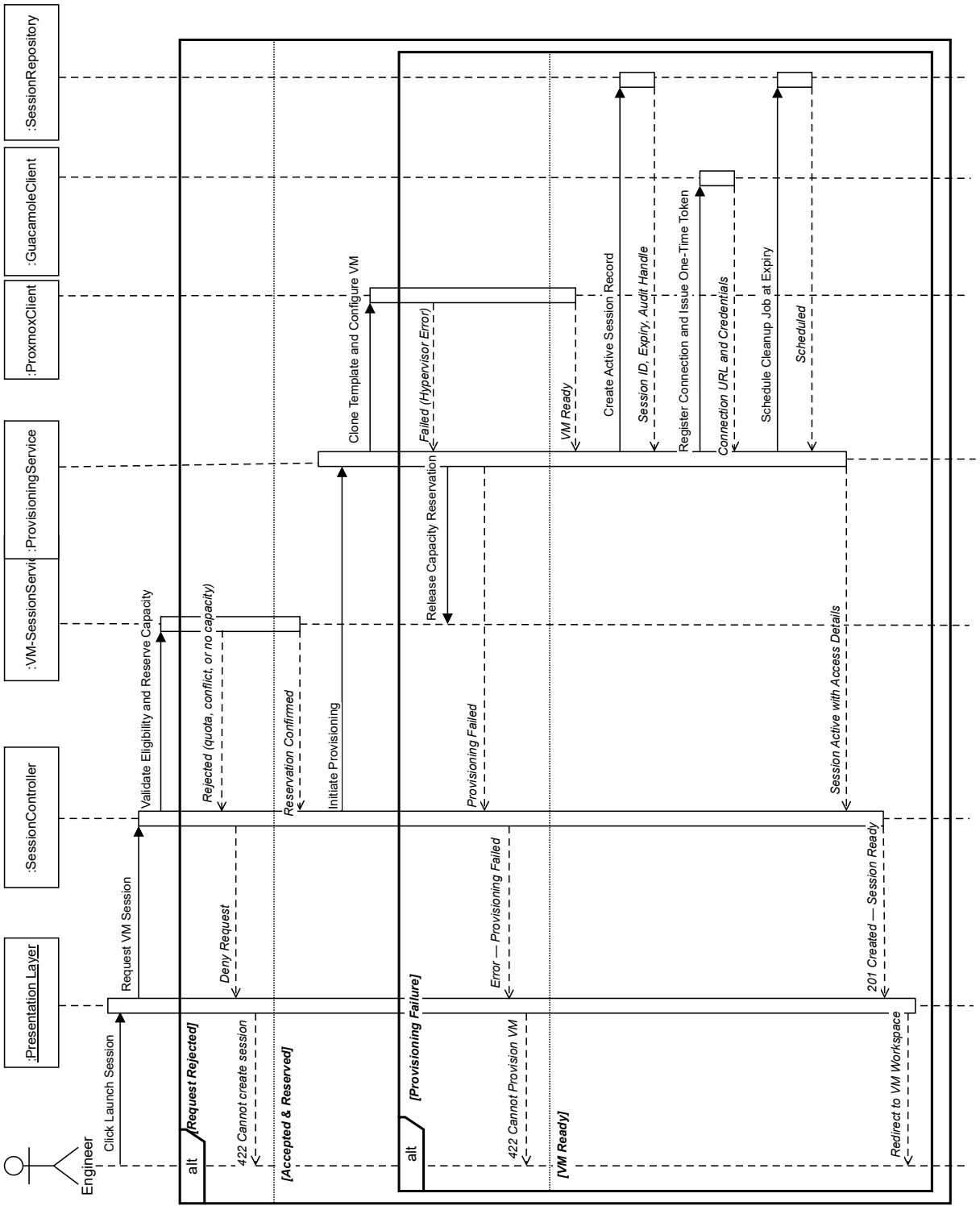


Figure 19: Sequence Diagram: Provision VM Session

3.5 User Authentication Sequence

Figure 20 illustrates the user authentication sequence.

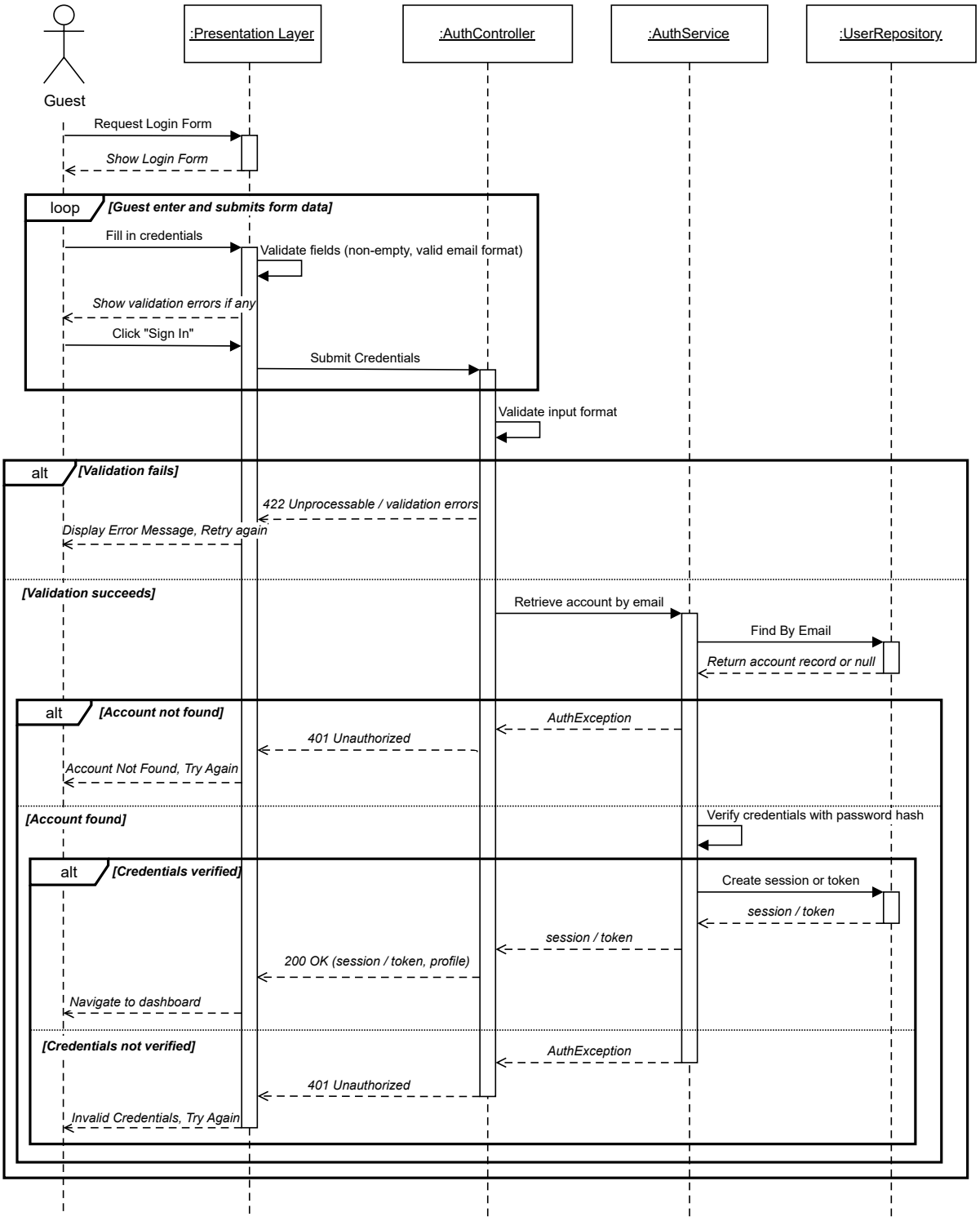


Figure 20: Sequence Diagram: User Authentication

Conclusion

This chapter established the conceptual foundation of the IoT-REAP platform across three complementary dimensions. Section 1 modelled system behaviour through a general use case diagram and actor-specific refinements covering all four user roles. Section 2 provided a comprehensive class-level view of the domain model, grouping the thirty-one identified classes into five functional clusters: user management, learning content, infrastructure, hardware resources, and financial operations. Finally, Section 3 captured the dynamic behaviour of critical workflows through sequence diagrams. Together, these artefacts form the design contract that guides the implementation and validation presented in the following chapter.

Chapter 3

Realization and Experimentation

Introduction

This chapter presents IoT-REAP platform implementation and validates operation. It is organized into six sections. Section 1 describes software architecture, asynchronous provisioning pipeline. Section 2 covers hardware, tools, and technology stack. Section 3 presents user interfaces and backend workflows. Section 4 details network topology and segmentation. Section 5 covers deployment, security, and automation. Finally, Section 6 reports verification of functional correctness, access control, remote laboratory workflows, security, and performance.

1 Adopted Software Architecture

IoT-REAP is implemented as a **layered monolithic Laravel application** combining synchronous request-response processing with event-driven asynchronous orchestration for infrastructure provisioning and state reconciliation.

1.1 Application Layers

The application is organized into four internal layers and an external infrastructure boundary, with dependencies flowing strictly downward. Each layer has a well-defined responsibility and does not reach across boundaries. Figure 21 illustrates the component organization and inter-layer dependencies.

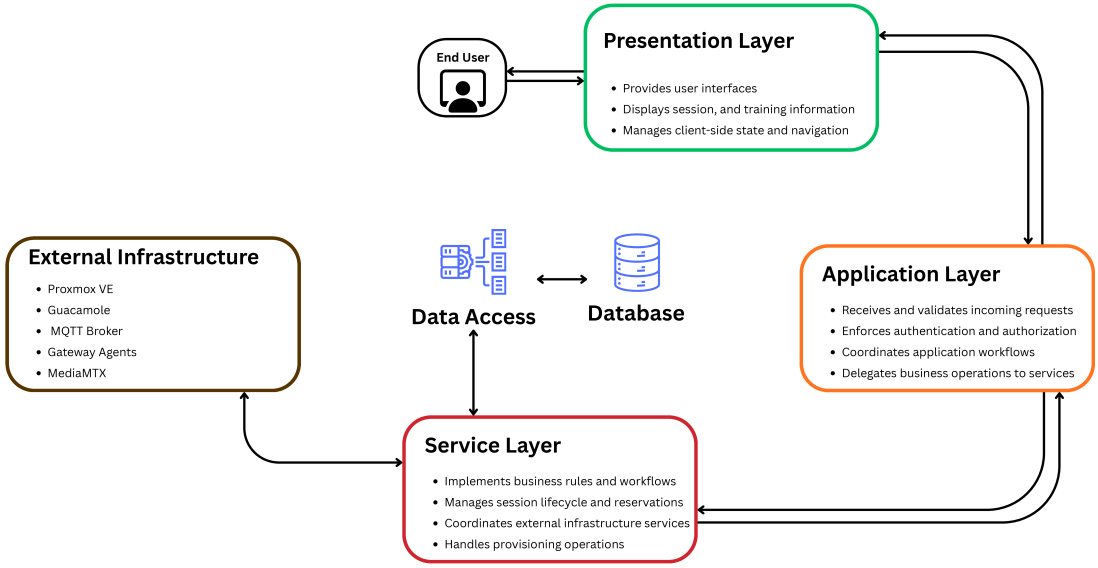


Figure 21: IoT-REAP Layered Architecture - Component Overview

Presentation Layer: React 18 with TypeScript constitutes the user interface for all roles. Inertia.js acts as a transport adapter between Laravel routes and React components, enabling SPA navigation without a separate API. React state is managed by Zustand and TanStack Query, independent of server-side rendering [26, 27, 45, 28, 58].

Application Layer (HTTP Boundary): Laravel controllers handle HTTP requests, validate input, enforce authentication and authorization, and delegate domain work to the Service layer. Controllers are intentionally kept thin, delegating business logic and persistence to lower layers.

Service Layer (`app/Services`): Service classes encapsulate domain workflows: session management, reservation conflict detection, quota enforcement, certificate eligibility, and payment processing. This directory also contains typed client classes communicating with external systems, isolating protocol-specific details.

Data Access Layer (`app/Repositories`): Repository classes abstract persistence operations, exposing typed interfaces and shielding the Service layer from database-specific queries. Eloquent ORM Models define entity relationships and query scopes. The Observer pattern on Models feeds the audit logging pipeline without coupling it to business logic.

External Infrastructure: The platform coordinates with five external systems: Proxmox VE for VMs, Apache Guacamole for remote desktop, MediaMTX and Frigate for camera streaming, Mosquitto for MQTT telemetry, and edge gateways for USB/IP hardware. These systems are accessed exclusively through typed client classes in `app/Services`, preventing protocol details from leaking into application layers.

1.2 Architectural Benefits

Operational simplicity: A single deployable unit reduces deployment complexity and cross-service versioning compared to microservices.

Failure isolation: External API and hardware failures are absorbed by the queue worker and retried independently, never reaching the user-facing HTTP layer.

State consistency: The reconciliation worker ensures the database reflects physical reality, addressing the risk of managing distributed hardware centrally.

Testability: The Service layer is tested with mocked external clients. Provisioning jobs are tested against a synchronous queue driver. React components are tested with Vitest and React Testing Library [32, 33].

Extensibility: New hardware or external systems require only a new client class and job handler; existing layers need minimal modification.

2 Tools and Development Environment

The implementation of IoT-REAP required a development environment capable of supporting backend web application development, reactive frontend interfaces, virtual machine orchestration, and industrial remote access workflows. The working environment therefore combined hardware resources suitable for local development and testing with a modern software stack centered on Laravel, React, TypeScript, and MySQL [23, 26, 27, 34].

2.1 Hardware Environment

The hardware environment of IoT-REAP comprises the physical machines, network equipment, and tangible devices that provide the computational and operational foundation for remote laboratory work. Table 30 summarizes the hardware components and their respective roles within the platform.

Table 30: Physical Hardware Components of IoT-REAP

Hardware Category	**Main Components**	**Role in IoT-REAP**
Compute hosts	Proxmox VE server, dedicated VM host	Provides the physical compute, memory, and storage resources for hosting virtual machines. Session VMs use the dedicated VMID range 200–999 to avoid conflicts with reusable templates [2].
Gateway node	Dedicated edge server or industrial PC	Acts as the physical bridge between the virtualized environment and external devices. This machine hosts the integration services that handle browser-based access, media streaming, camera handling, and peripheral management.
Camera hardware	USB webcams, IP cameras, ESP32-CAM devices, PTZ-capable cameras	Supplies live video sources for laboratory monitoring and experimentation. Default camera streams are configured at 640×480 resolution, 15 fps, and MJPEG input format.
USB peripherals	Cameras, serial adapters, and other attachable USB devices	Physical devices that can be bound to a running session when required, with tracked states: *available*, *bound*, *attached*, and *dedicated*.
Industrial devices	MQTT-connected robots, sensors, and automation hardware	Provides the physical interaction layer for robotics and industrial control scenarios [4].

Hardware Category	Main Components	Role in IoT-REAP
Network infrastructure	Internal lab switches, routers, and cabling	Ensures reliable connectivity between compute hosts, gateway nodes, cameras, and industrial devices within the laboratory network.

This hardware structure reflects the layered physical architecture of the platform: compute hosts provide processing and storage resources, the gateway node enables connectivity between virtual and physical domains, camera and USB peripherals supply visual and interactive capabilities, and industrial devices deliver the hands-on experimentation layer required for remote laboratory work.

2.2 Development Environment

The development environment comprises the tools and utilities used to build, test, and document the IoT-REAP platform. Since the platform is implemented as a Laravel application using Inertia.js and React, both PHP and JavaScript tooling were required throughout the development cycle. Table 31 summarizes the main development tools and their roles.

Table 31: Development Environment

Tool	Logo	Role in the Project
Visual Studio Code		Used as the main integrated development environment for editing PHP, TypeScript, React components, route definitions, configuration files, and LaTeX report sources [35].
Git and GitHub		Used to manage source code history, feature evolution, and recovery of stable project states during iterative development [36, 37].
GitHub Actions		Used to automate the continuous integration and deployment pipeline, executing test suites on every push to the `main` branch.
Composer		Used to install and manage Laravel framework packages and backend dependencies [38].
NPM		Used to manage frontend dependencies including React, TypeScript, Vite, Tailwind CSS, Radix UI components, Zustand, React Query, and Vitest [39].
Postman		Used to verify HTTP routes, request payloads, responses, authentication behavior, and external integration endpoints during development [40].

Table 31 – Continued

Tool	Logo	Role in the Project
Draw.io		Used to prepare architecture, workflow, and UML diagrams integrated into the academic report [41].
OWASP ZAP		Used to perform automated security scanning against the running application, covering vulnerability detection, authentication flow analysis, and session management validation [30].
Cisco Packet Tracer		Used to simulate the network topology and segmentation of the platform, demonstrating isolation between the corporate LAN, virtualization cluster, and external access points [42].
VMware Workstation		Used to create and manage virtual machine templates for the Proxmox VE environment, including Windows and Linux images with preconfigured software for training sessions [43].

2.3 Technology Stack

The IoT-REAP platform relies on a diverse technology stack that supports application development, data management, remote laboratory operations, real-time communication, and multimedia services. Table 32 presents the main technologies organized by architectural layer.

Table 32: Technology Stack Used in the Platform

Technology	Logo	Role in the Platform
Service and Application Layer		
PHP		Main backend programming language used to implement controllers, services, models, jobs, events, policies, and platform business logic [44].
Laravel		Backend framework used for routing, authentication, queue processing, validation, authorization, database access, and modular organization of application logic [23].
Inertia.js		Connects Laravel routes to React pages without building a separate REST-only frontend, enabling a smooth single-page navigation model [45].
Stripe		Supports checkout, payment tracking, refund workflows, and payout-related administrative features [24].
Presentation Layer		

Table 32 – Continued

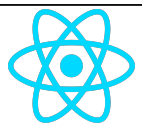
Technology	Logo	Role in the Platform
React		Used to build the interactive frontend interfaces for training paths, dashboards, teaching pages, administration panels, and user settings [26].
TypeScript		Provides type safety across the frontend codebase, improving maintainability and reducing interface integration errors [27].
Vite		Used for modern frontend asset bundling, hot module replacement in development, and production asset compilation [46].
Tailwind CSS		Used for responsive and utility-based user interface styling across public, engineer, instructor, and administration pages [47].
Guacamole Common JS		Provides the JavaScript implementation of the Guacamole protocol for browser-based remote desktop access [3].
Data Access Layer		
MySQL		Stores structured application data including users, reservations, certificates, quizzes, notifications, sessions, and payments [34].
External Infrastructure		
MediaMTX		Media server that handles RTSP, HLS, and WebRTC streaming for live camera feeds. Exposes ports 8554 (RTSP), 8888 (HLS), 8889 (WebRTC), and 9997 (API) [48, 49, 29, 50].
Frigate		Network video recorder that manages camera streams, motion detection, and recording for IP and USB cameras [51].
FFmpeg		Handles video transcoding across three quality profiles (360p, 720p, 1080p), HLS segment generation, thumbnail extraction, format normalization, and camera stream adaptation for browser-compatible playback [52, 29].
Apache Guacamole		Enables clientless browser-based remote desktop access to virtual machines without requiring local desktop client installation [3].
Mosquitto		MQTT broker that handles real-time camera control and stream management using QoS 1 message delivery for reliable industrial control operations [53, 4].

Table 32 – Continued

Technology	Logo	Role in the Platform
Testing and Quality Assurance		
PHPUnit		Used to verify backend logic, route behavior, business rules, and Laravel feature flows [33].
Vitest		Used to test React interface logic and component behavior in the frontend codebase [32].

2.4 Technology Stack Justification

The technology stack was selected to align each architectural layer with a stable, mature ecosystem that reduces integration risk and maintains strict separation of concerns across application layers. Laravel provides a robust backend framework with built-in ORM, queue management, and authentication scaffolding, reducing boilerplate through its convention-over-configuration approach. MySQL serves as the relational database engine, offering proven transactional guarantees and native support within the Laravel Eloquent ORM. React 18 with TypeScript ensures type-safe component reusability across a large UI surface, while Zustand provides a lightweight, hook-based global state management solution that avoids the overhead of heavier alternatives such as Redux. Laravel Reverb delivers first-party WebSocket broadcasting without reliance on external cloud services, which is essential for real-time camera feedback in constrained network environments [54]. Proxmox VE was chosen as the open-source hypervisor to avoid proprietary licensing costs while exposing a full REST API for automated VM lifecycle management [2]. Apache Guacamole enables browser-native remote desktop access without client-side installation, which is essential for accessibility in educational contexts [3]. MQTT serves as the lightweight publish-subscribe broker for IoT telemetry, offering minimal overhead and reliable message delivery under constrained network conditions [4].

3 Implementation

This section presents the principal interfaces and functional areas implemented in IoT-REAP. The objective is to explain how the platform organizes the complete user experience across public access, technical learning, VM-based practice, teaching workflows, and administrative control, with emphasis on the technical decisions and integration points underlying each interface.

3.1 Landing Page

The landing page is the public entry point of the platform. It fetches featured training paths via Inertia.js shared page props from Laravel Eloquent queries, renders enrollment calls-to-action, and manages SEO metadata through Inertia's `Head` API. A hero section communicates the platform's educational scope and primary features, while a curated catalog block presents popular training paths with title, category, price, and ratings. The interface is accessible without authentication and exposes navigation toward the full catalog, individual path details, and the authentication flow. Public routes are guarded by Laravel's guest middleware to prevent authenticated users from landing on registration pages after session establishment.

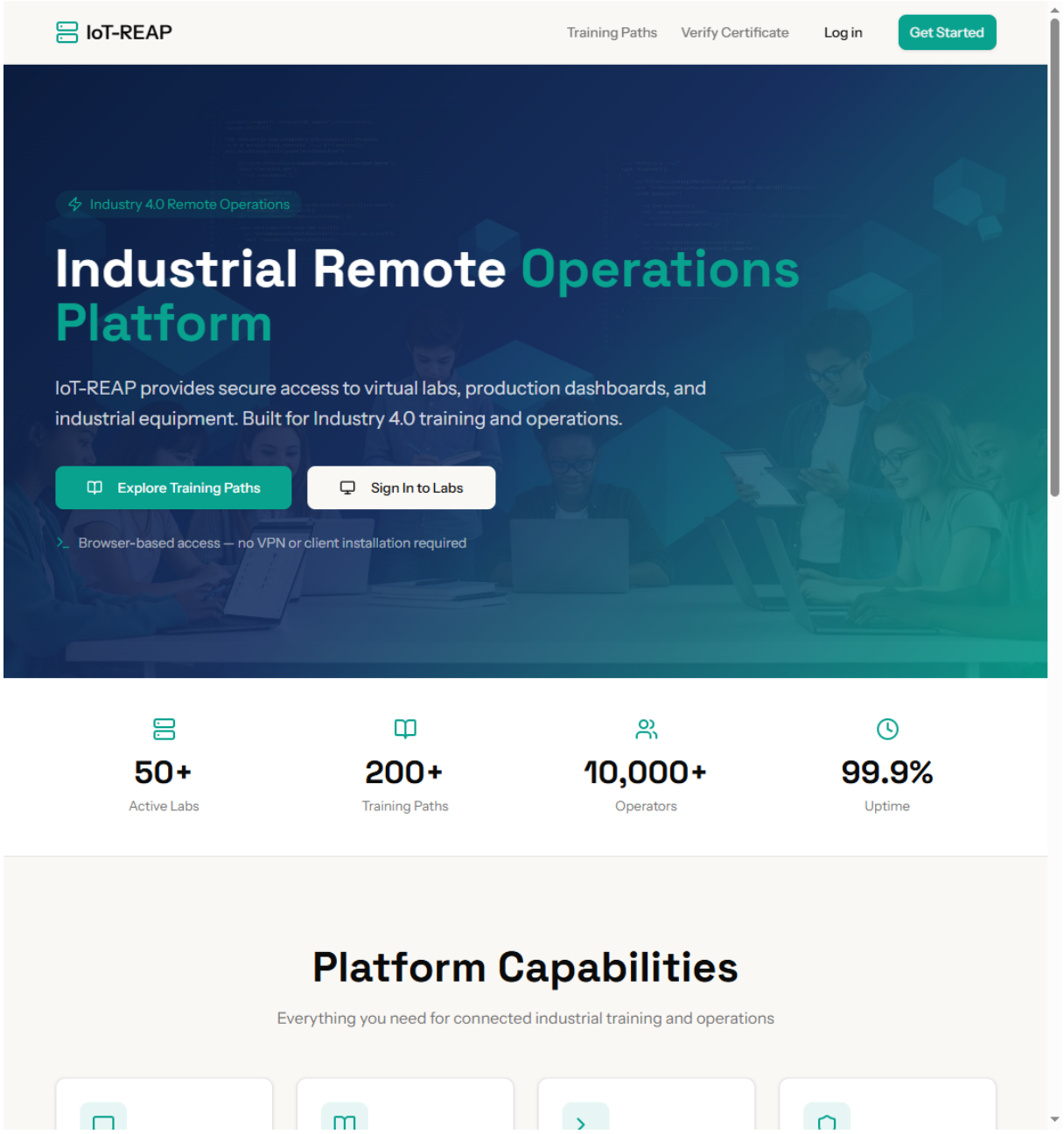


Figure 22: Landing Page Interface

3.2 Authentication and Role Selection

The authentication interface manages the identity lifecycle of platform users. Registration collects profile information and triggers an email verification through Laravel Fortify [55]. Login supports credential-based and Google OAuth flows via Laravel Socialite [56, 57]. After authentication, users who have not committed to a role are presented with a role-selection screen allowing them to self-identify as a learner, teacher, or engineer. Two-factor authentication is available as an optional security layer via TOTP. Role assignment is persisted to the user record and governs all subsequent policy evaluations via Laravel's Policy classes, ensuring that role-specific dashboards, permissions, and content are applied from the first session.

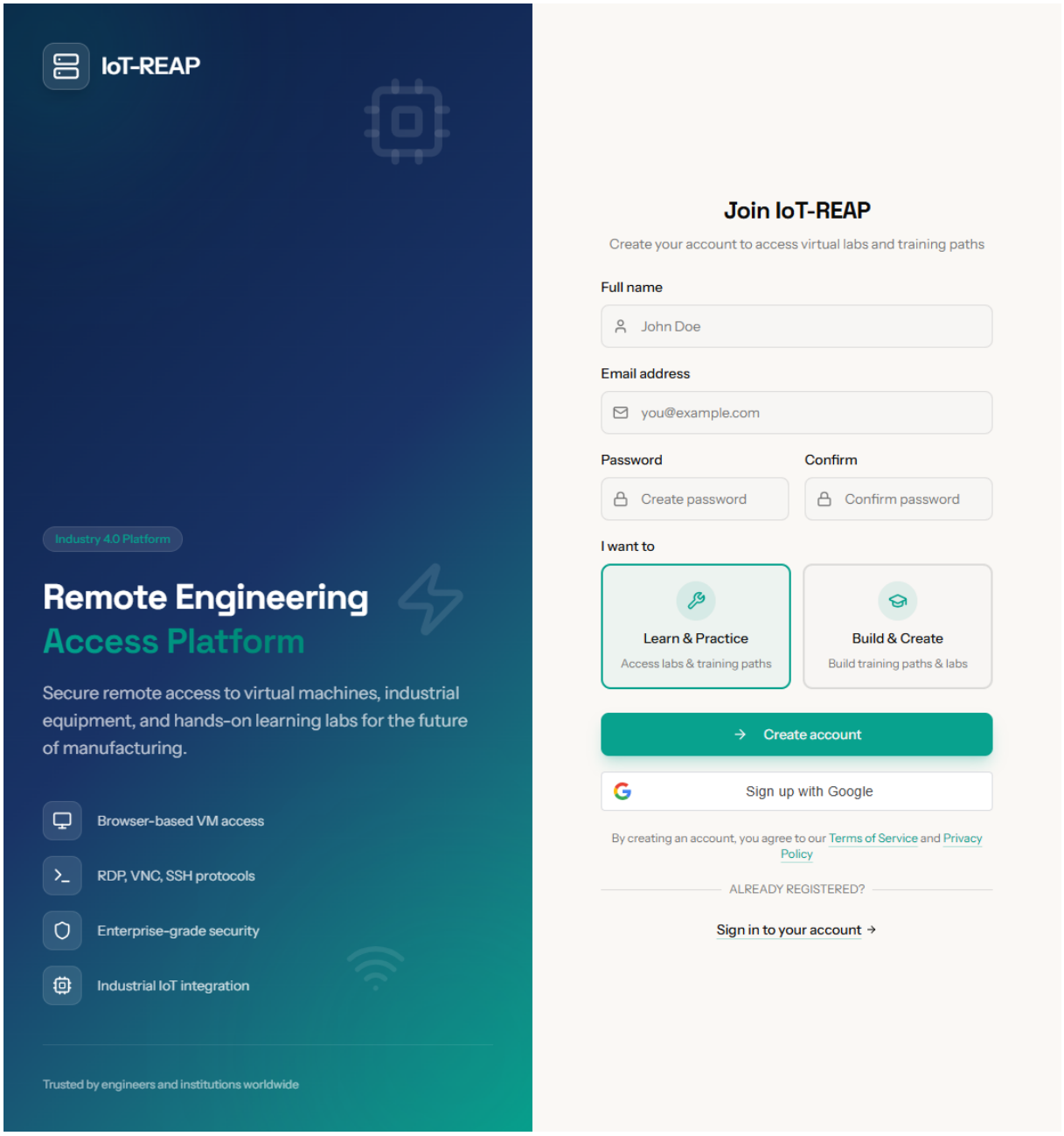


Figure 23: Authentication and Role Selection Interface

3.3 Engineer Dashboard

The engineer dashboard provides authenticated users with a consolidated view of their active learning context. It displays enrolled training paths alongside a visual progress indicator for each, recent activity entries, unread notifications, and quick access to earned or claimable certificates. A recommended content block surfaces paths related to the engineer's existing enrollments. Dashboard data is fetched via TanStack React Query with Zustand managing lightweight client-side UI state [58]; the query cache is invalidated on enrollment and unit-completion events to ensure the displayed progress reflects the actual database state without a full page reload.

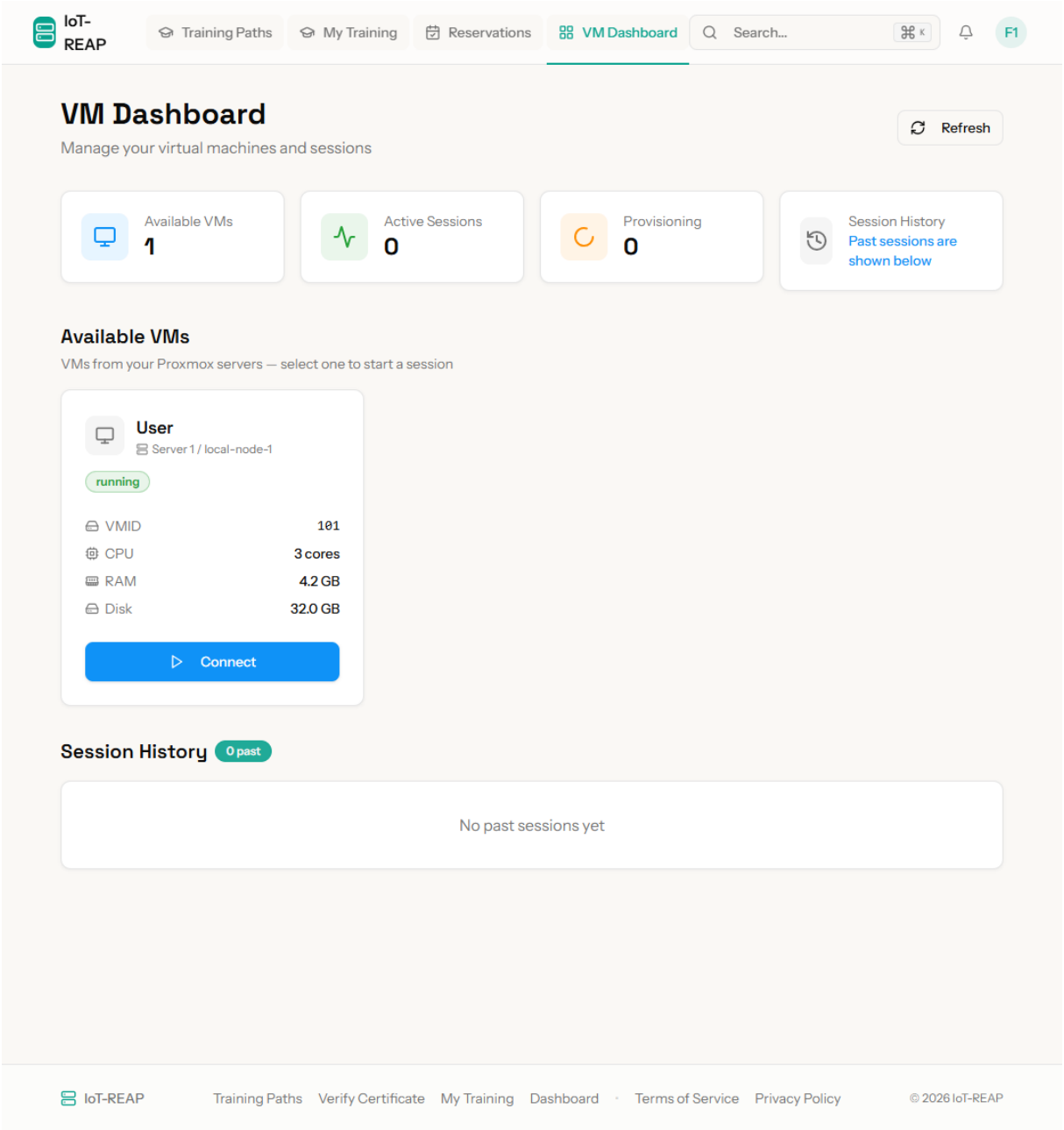


Figure 24: Engineer Dashboard Interface

3.4 Training Unit Learning Interface

The training unit interface is the core workspace. It combines a content area - displaying videos, articles, or external resources - with a left-hand panel for unit navigation, completion tracking, and discussion access. Engineers can mark units as complete, take notes at specific content positions, and access the quiz when one is configured. For units linked to a VM session, a launch button is rendered conditionally through a React component that receives the session authorization state as a prop from the Inertia page share, allowing seamless transition from theory to hands-on practice. Unit completion triggers an Eloquent Observer that recalculates path progress and emits a WebSocket notification via Laravel Reverb [54].

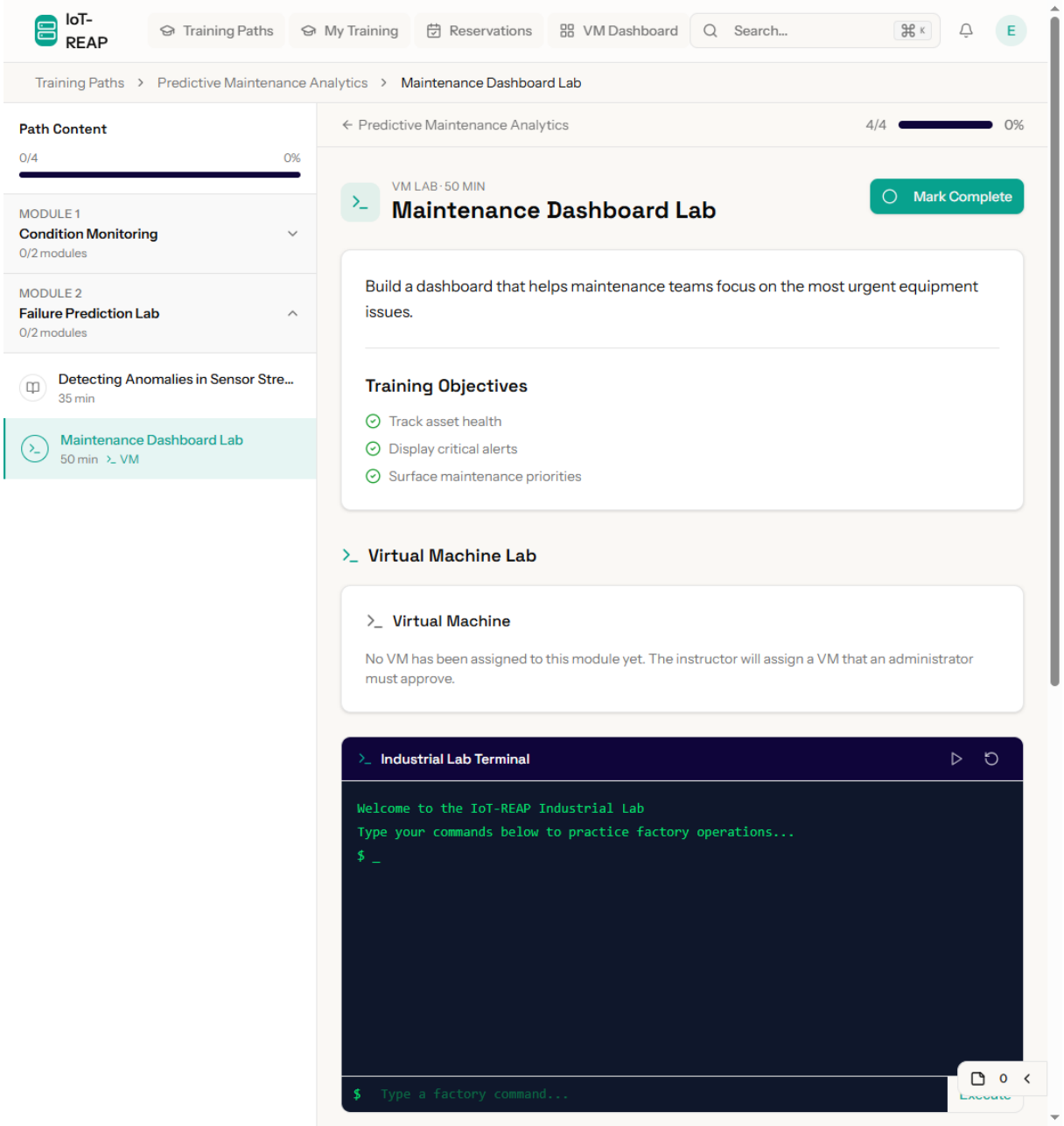


Figure 25: Training Unit Learning Interface

3.5 Virtual Machine Session Interface

Virtual machine session interface gives engineers browser-based access to a provisioned laboratory environment. It displays the VM state, connection status, and session remaining, and embeds the Apache Guacamole client via Guacamole Common JS to establish a clientless remote desktop tunnel within the page [3]. The Guacamole token is obtained server-side by `GuacamoleClient` and injected into the page props by the controller, ensuring it is never exposed as a URL parameter. Controls allow users to extend session within authorized limits, request a full-screen view, and terminate session. If a session is lost due to a connectivity interruption, the interface provides a reconnect action without requiring a new reservation.

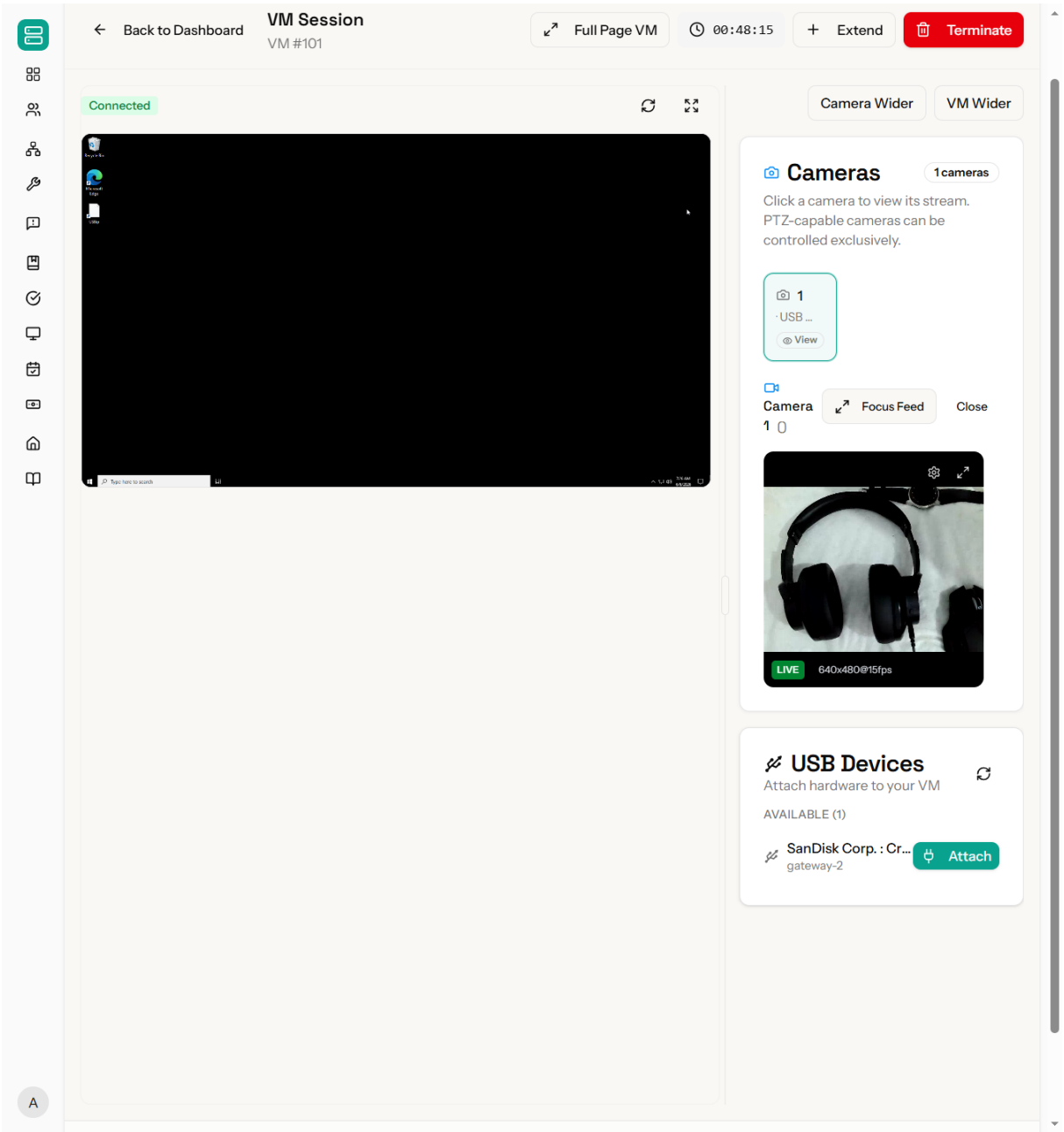


Figure 26: Virtual Machine Session Interface

3.6 Reservations Interface

The reservations interface allows users to plan and manage scheduled access to laboratory resources. Available time slots are displayed in a calendar-based view, with resource availability derived from the current session schedule and administrator-defined maintenance windows through a conflict-detection query in `ReservationService`. Users can submit reservation requests specifying the desired resource type, duration, and preferred time range. The interface displays the real-time status of submitted reservations (*pending*, *confirmed*, *active*, *completed*, or *cancelled*), and notifies users of status changes through the platform notification system backed by Laravel Reverb [54].

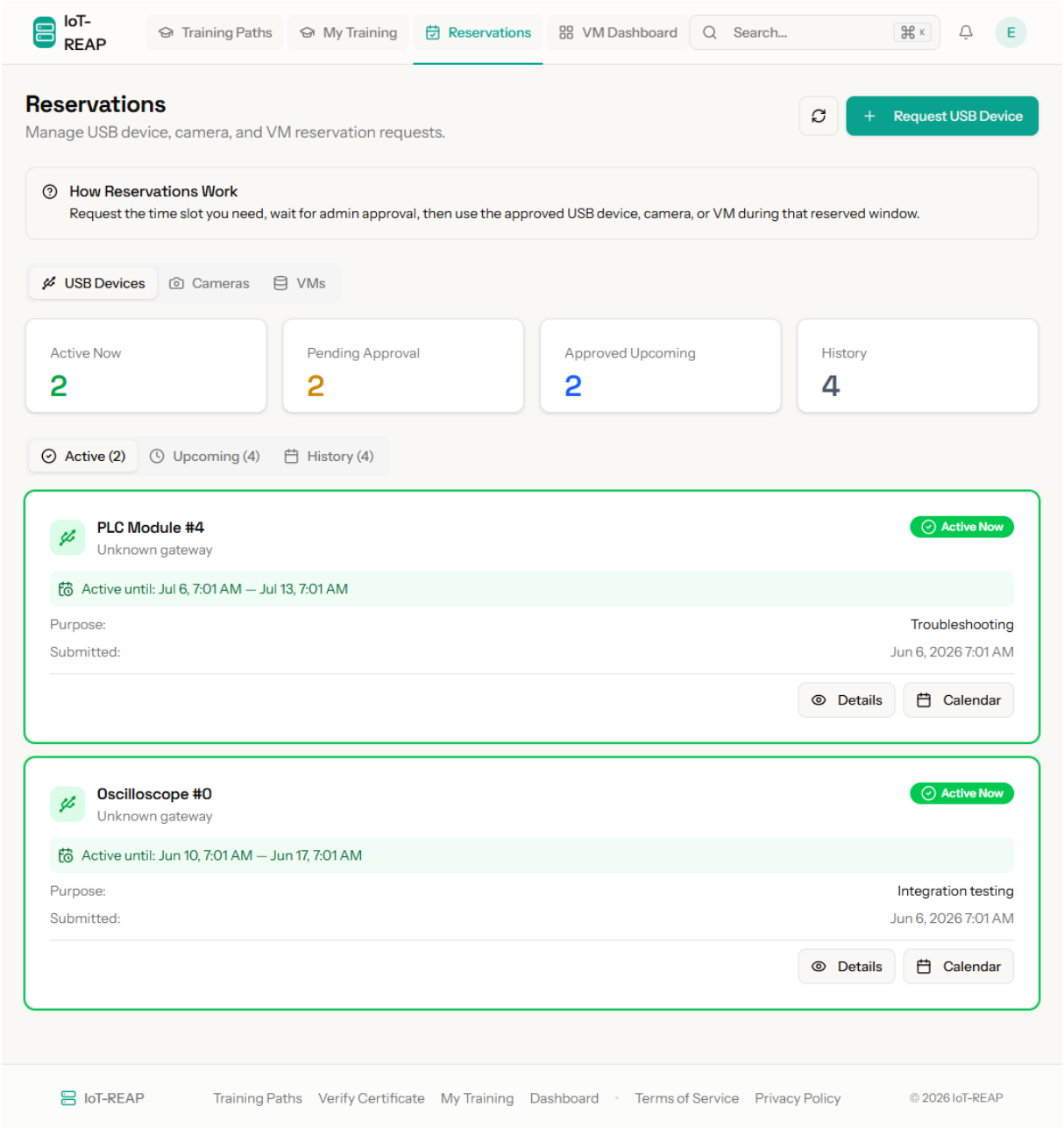


Figure 27: Reservations Interface

3.7 Teacher Analytics and Earnings

The teacher analytics and earnings interface presents instructors with a quantitative view of their content's performance. Key indicators include total enrollments per path, active engineer count, average quiz scores, unit completion rates, and overall training path rating. Revenue evolution is rendered as a time-series chart using Recharts [59], with the underlying data aggregated by a dedicated Eloquent scope to avoid loading raw payment records into the component. Teachers can submit payout requests when their accumulated balance exceeds the minimum threshold; the balance calculation is performed at the service layer on each request to guarantee consistency with the payment table.

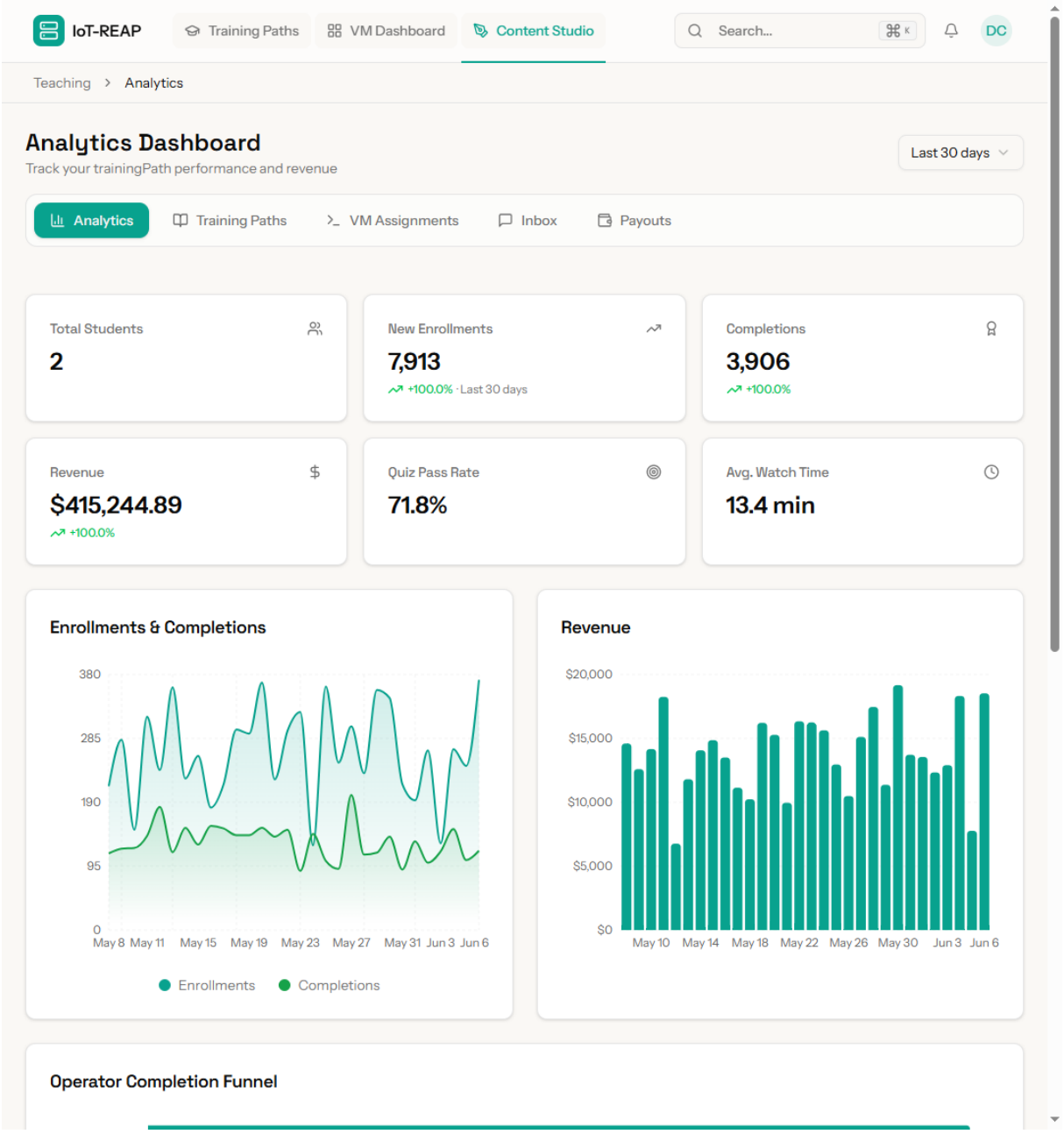


Figure 28: Teacher Analytics and Earnings Interface

3.8 Teacher Content Studio

The teacher content studio is the primary authoring environment for instructors. Teachers create and structure training paths by defining metadata, organizing units in a drag-and-drop sequence, and attaching content - videos, articles, quizzes - to each unit. Video uploads are processed through an FFmpeg transcoding pipeline into three profiles (360p, 720p, 1080p), dispatched asynchronously to avoid blocking the upload response [52]. Each unit can be linked to a VM session type, defining the practical environment engineers will access. The submission and revision workflow ensures teaching content enters a pending state awaiting administrator review before becoming publicly accessible, enforced through a Policy-gated status transition.

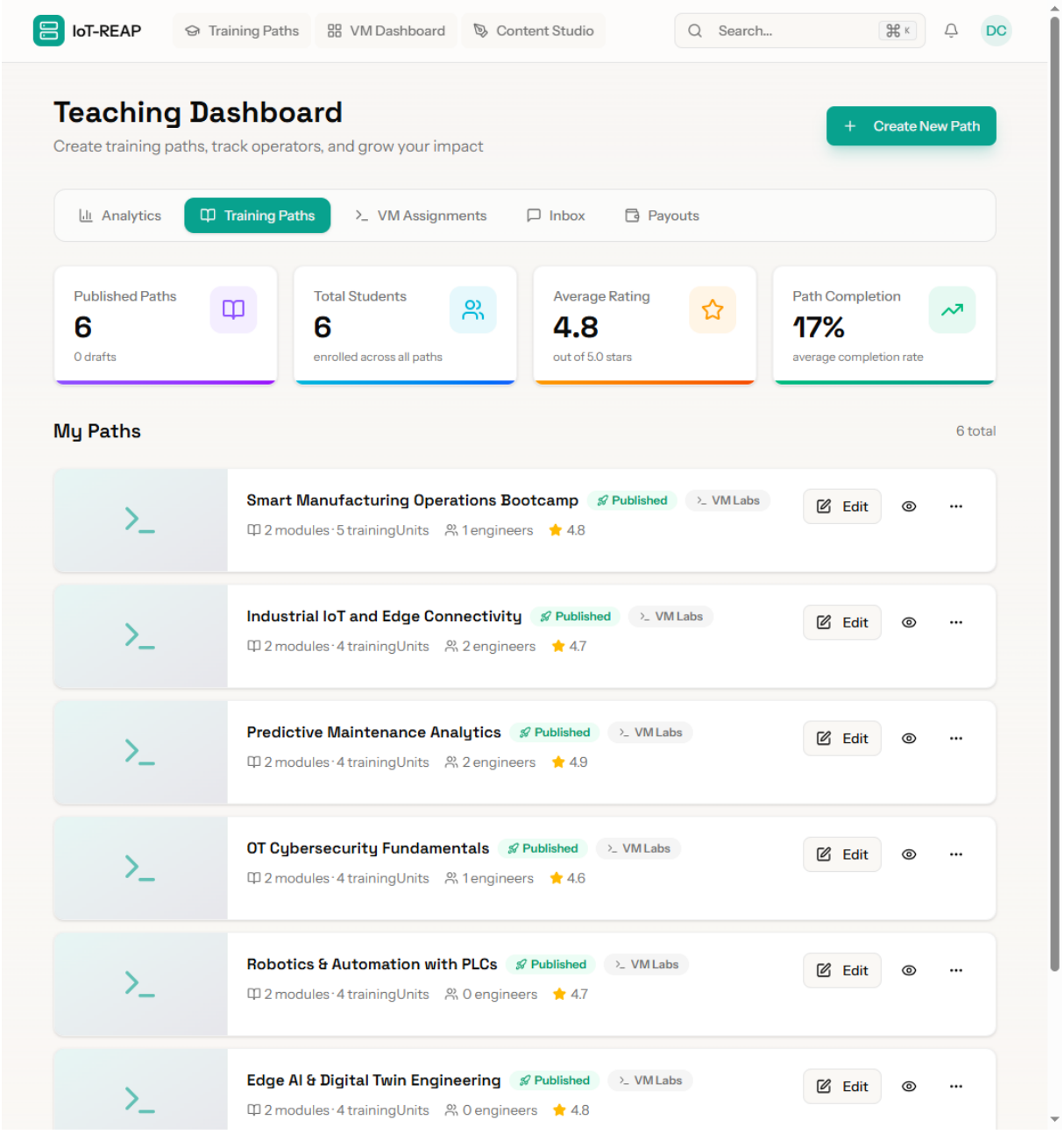


Figure 29: Teacher Content Studio Interface

3.9 Administration Dashboard

The administration dashboard provides platform operators with centralized control over operational dimensions of IoT-REAP. It organizes management functions into panels: user management (creation, role assignment, suspension), training path review, reservation supervision, infrastructure monitoring (VM pool status, gateway connectivity, cameras), financial oversight (payments, refunds, processing), and alert management (system warnings, scheduling, moderation flags). Real-time operational events are delivered via WebSocket channels through Laravel Reverb [54], enabling administrators to observe session provisioning progress, gateway connectivity changes, and payment confirmations without manual refresh.

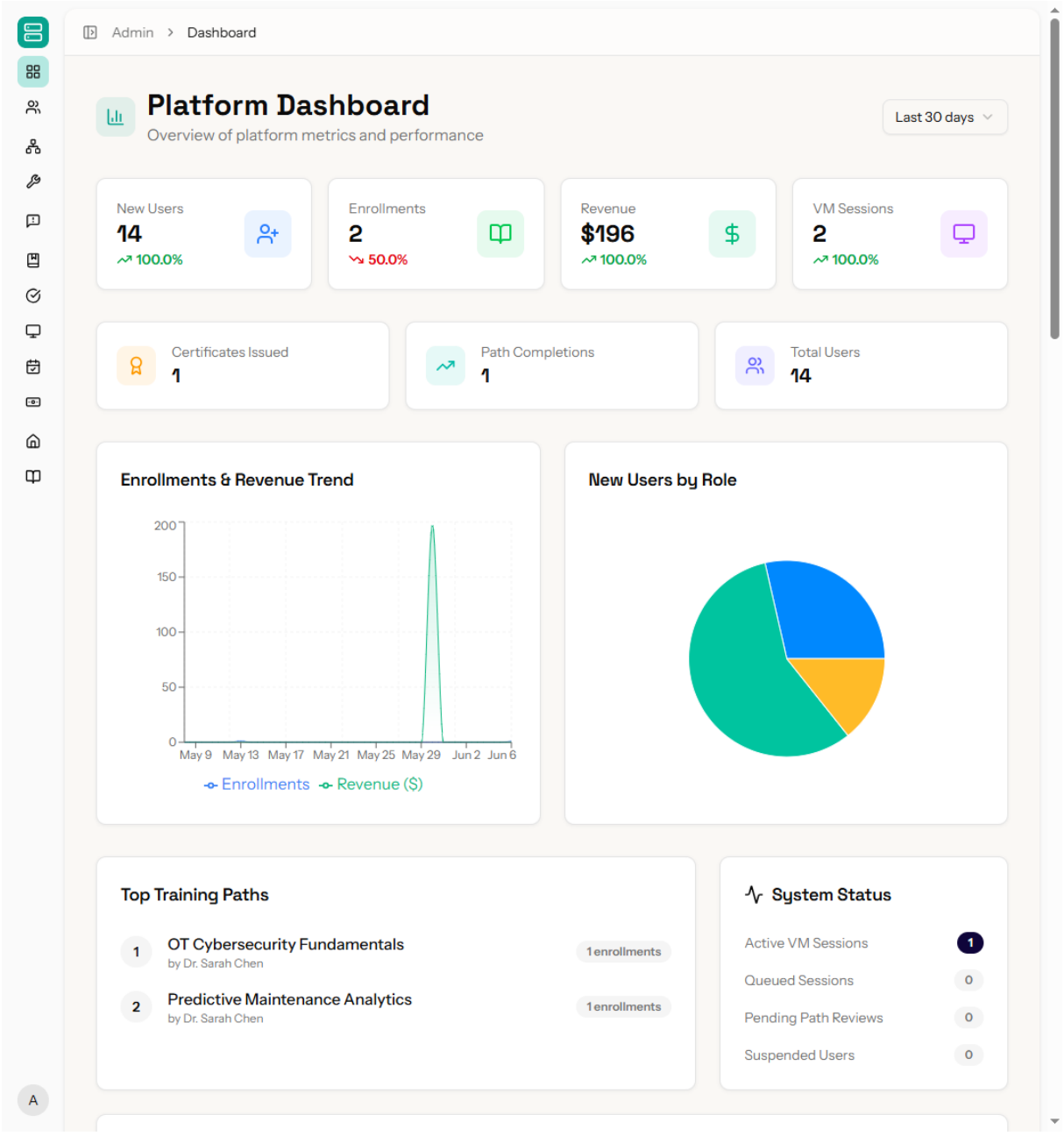


Figure 30: Administration Dashboard Interface

3.10 Teacher Training Path Management

The training path management interface allows instructors to update the core identity and configuration of their educational content. Teachers can modify titles, descriptions, categories, and difficulty levels, as well as manage visibility settings and pricing. The interface provides a centralized view of the path's status and allows for the management of high-level metadata that governs how the path is discovered by engineers in the catalog. All changes are validated against business rules to ensure consistency before being persisted.

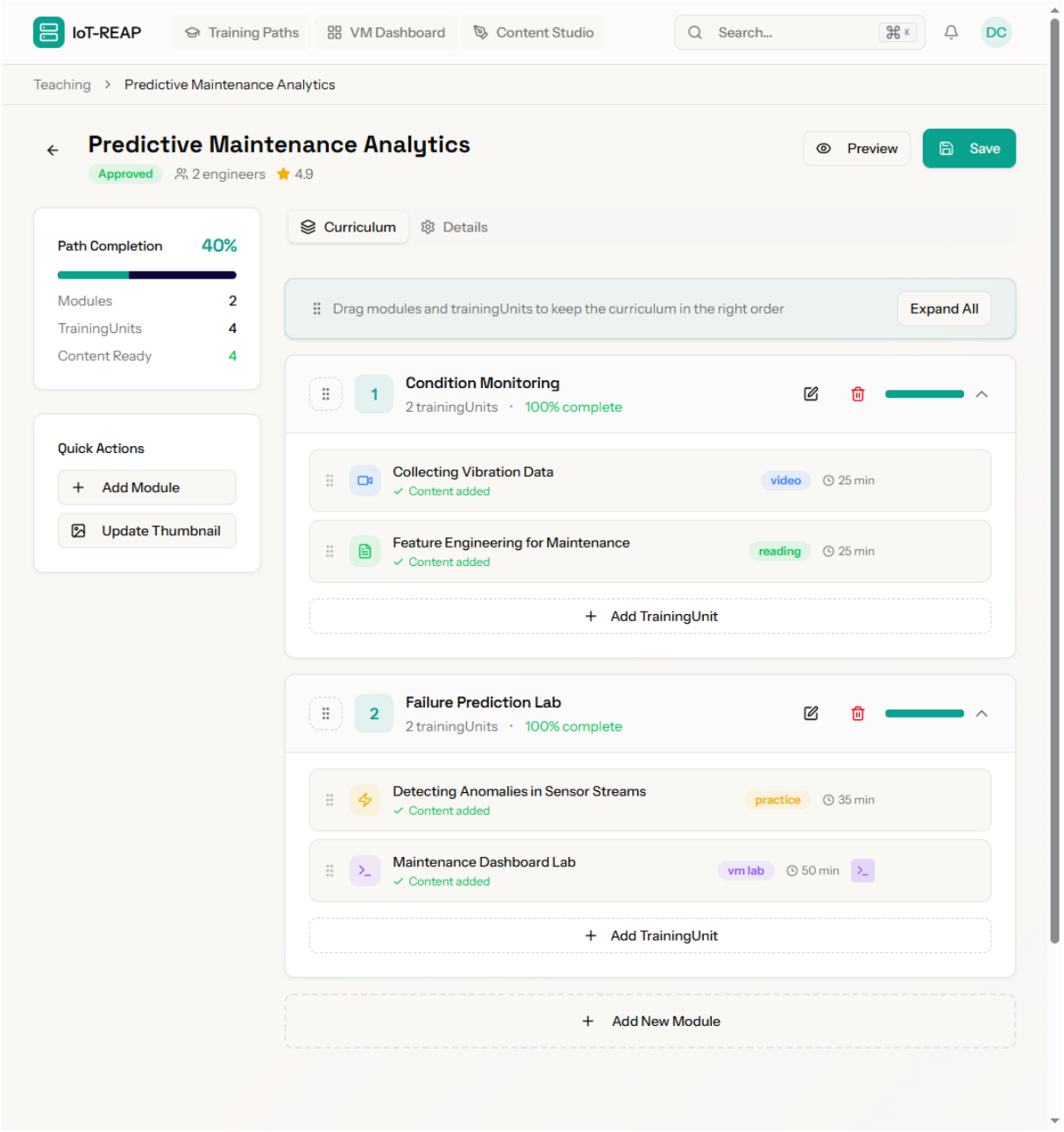


Figure 31: Training Path Management Interface

3.11 Teacher Virtual Machine Assignments

The virtual machine assignment interface enables teachers to link specific training units with the required computational resources. Instructors can select from available VM templates or session types that define the software environment and hardware capabilities needed for practical exercises. This mapping ensures that when an engineer launches a practical session from a unit, the platform provisions the correct environment as specified by the instructor. The interface also allows for configuring persistent storage and network isolation settings per unit.

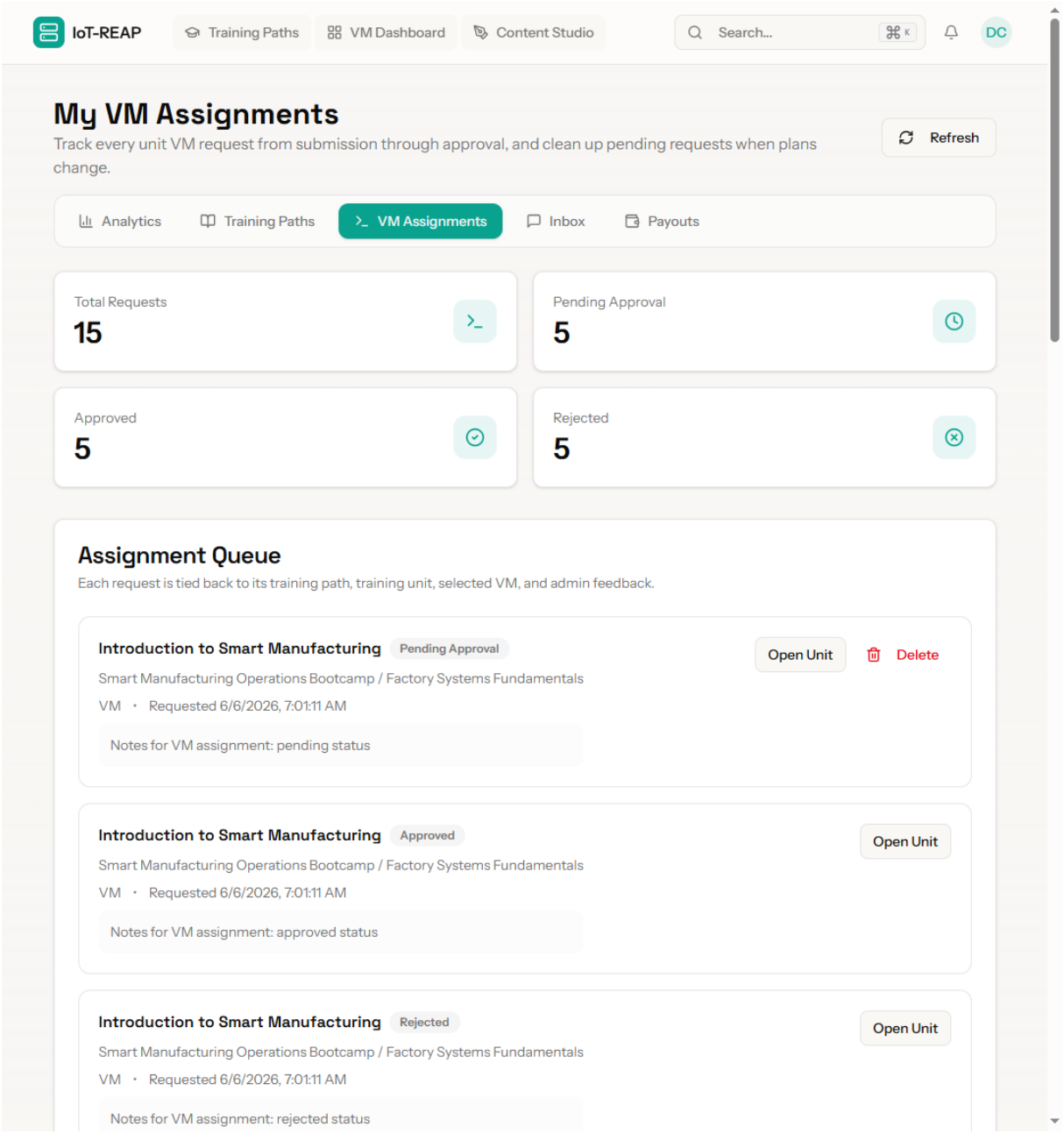


Figure 32: Virtual Machine Assignment Interface

3.12 Administrative User Management

The user management interface provides administrators with a comprehensive view of the platform's user base. It supports advanced filtering by role, status, and registration date. Administrators can use this interface to adjust user roles, manually verify accounts, reset passwords, or suspend users who violate platform policies. Integration with Laravel Fortify ensures that role transitions and status changes are reflected across all authentication guards and policy evaluations in real-time.

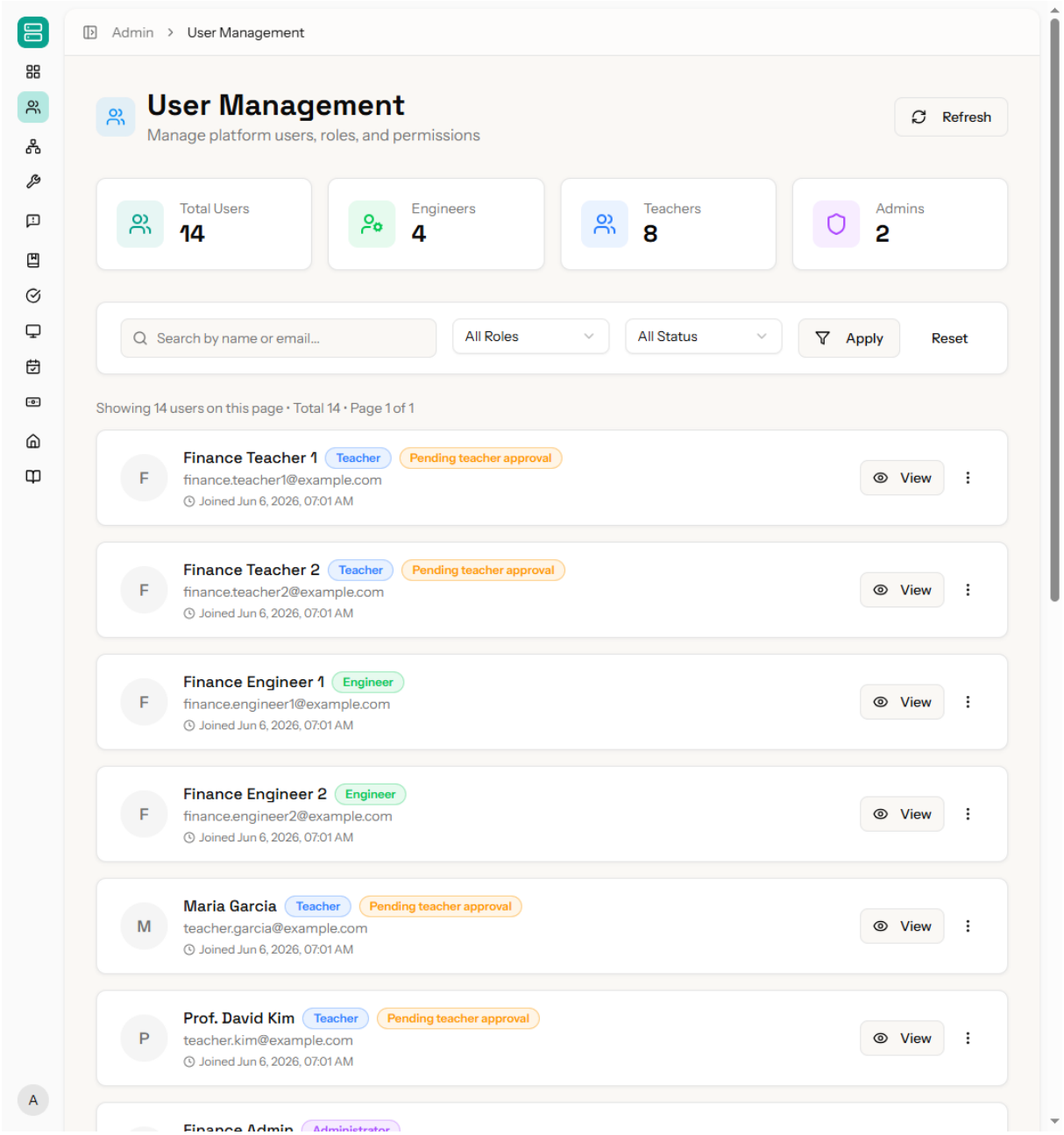


Figure 33: Administrative User Management Interface

3.13 Infrastructure Monitoring and Control

The infrastructure management interface provides real-time visibility into the physical and virtual resources powering the platform. Administrators can monitor the health and load of Proxmox nodes, the connectivity status of edge gateways, and the availability of shared peripherals like cameras and USB devices. Controls are provided to trigger manual maintenance tasks, restart services, and reconcile the database state with physical hardware when discrepancies are detected. Real-time updates are delivered via WebSockets to provide an immediate view of system health.

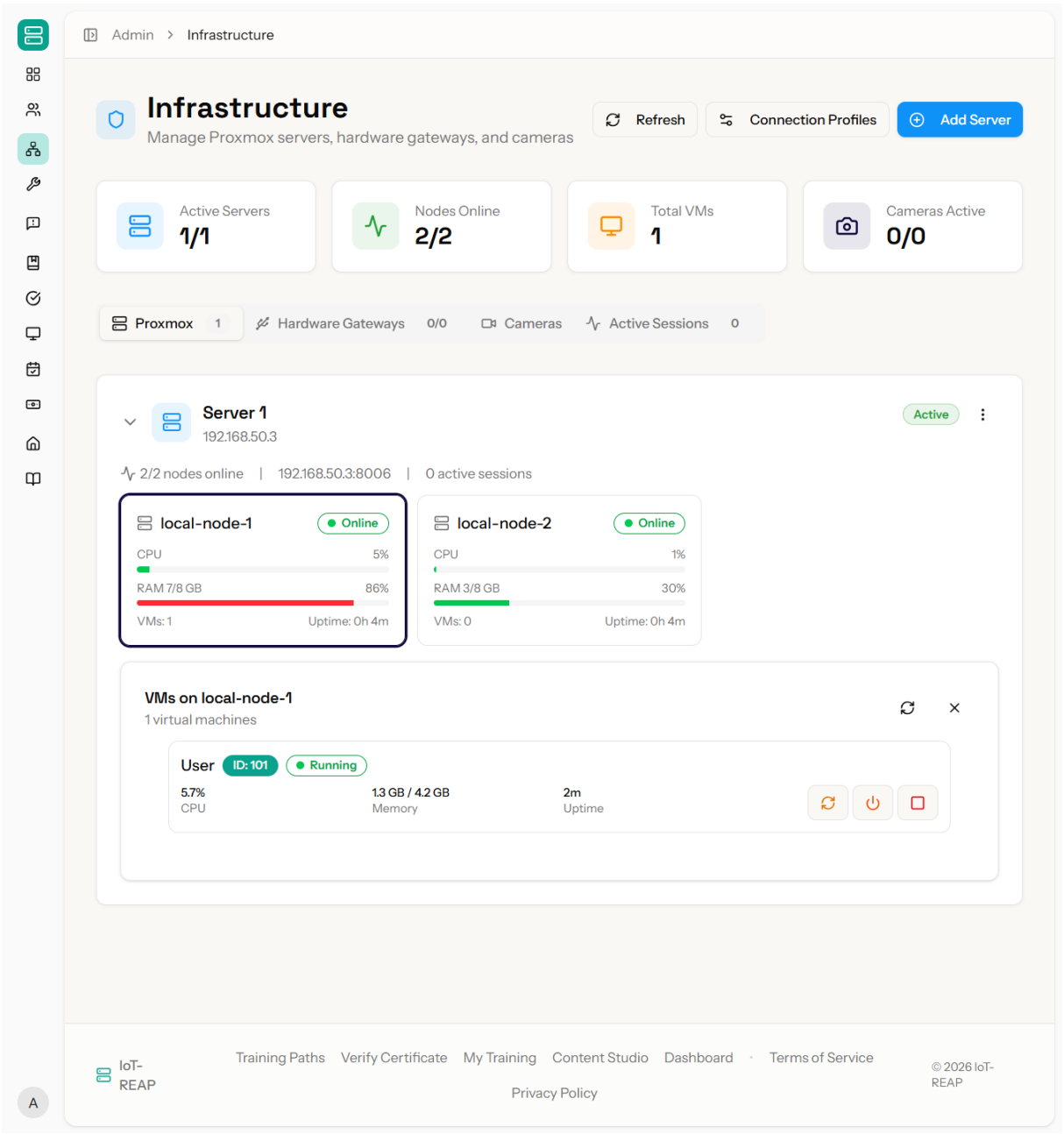


Figure 34: Infrastructure Monitoring Interface

3.14 Training Path Review and Approval

The training path approval interface serves as the quality control hub for new educational content. Administrators can review submitted paths, examining unit structures, video quality, and quiz correctness before granting public visibility. The interface supports a feedback loop where reviewers can request modifications or provide comments to instructors, ensuring that all published content meets the platform's pedagogical and technical standards. Status transitions are managed through a state machine that prevents unauthorized publication of unreviewed material.

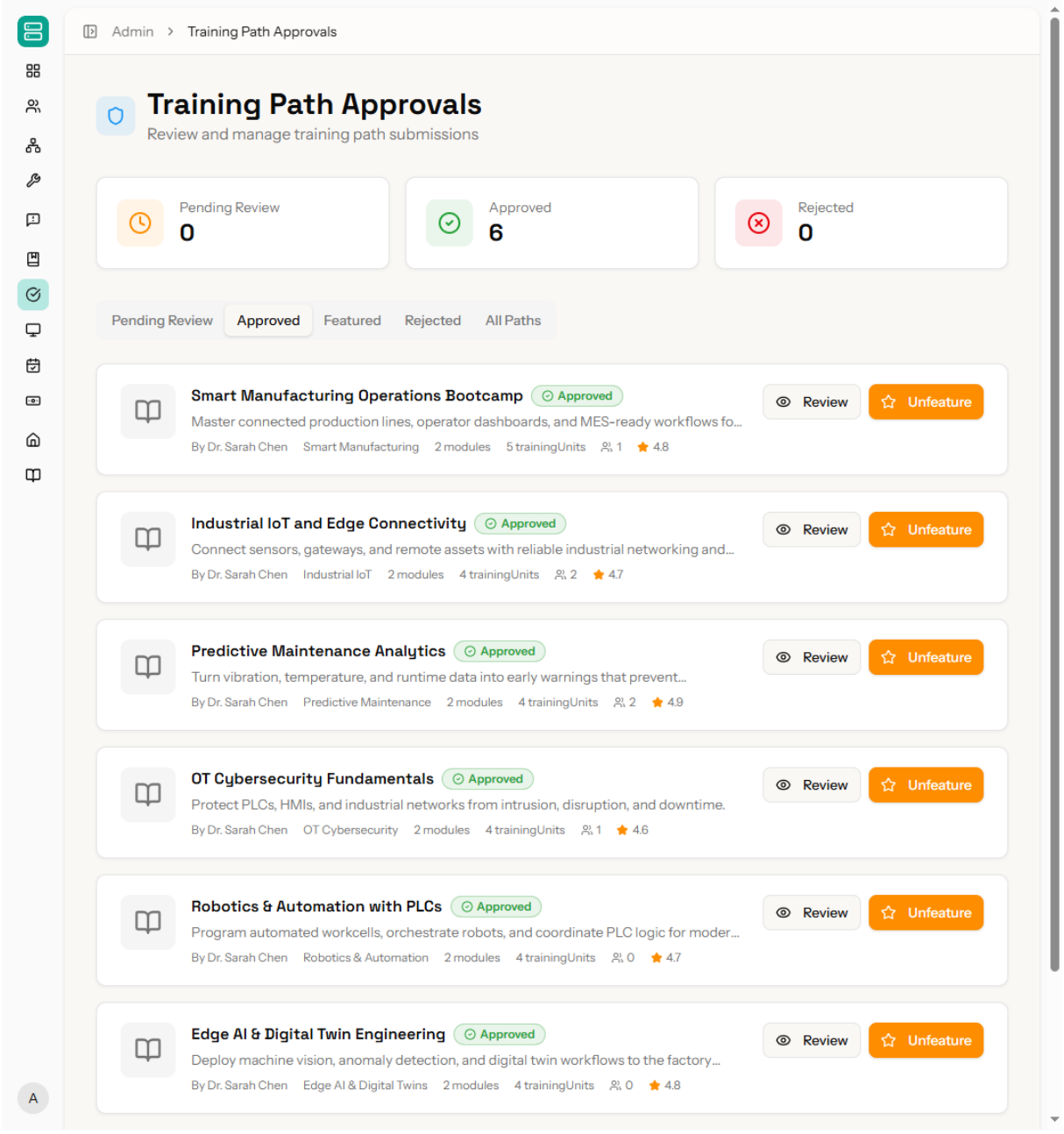


Figure 35: Training Path Review Interface

3.15 Financial Oversight: Payouts and Refunds

The financial management interface allows administrators to oversee the platform's economic lifecycle. It provides detailed logs of all transactions, including successful enrollments and processed refunds. Administrators can review and approve payout requests from teachers, ensuring that earnings are distributed correctly according to platform fees and minimum thresholds. This interface serves as the primary point of reconciliation between internal payment records and external payment gateway states, such as Stripe, to ensure financial integrity.

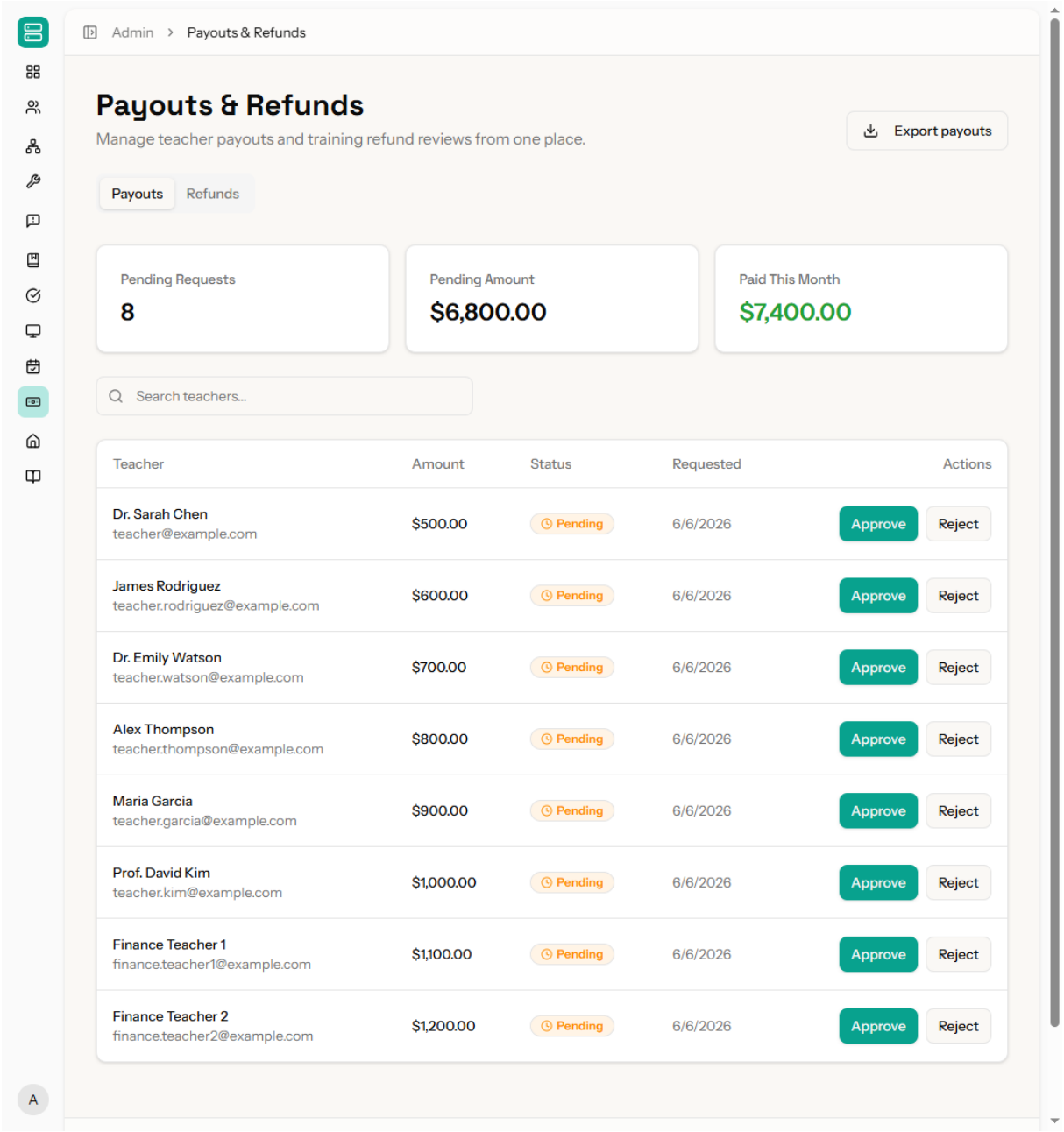


Figure 36: Financial Oversight Interface

4 Network Topology and Segmentation

To validate the remote access architecture proposed in the preceding chapter, a network simulation was conducted using Cisco Packet Tracer [42]. The simulated topology reflects a real-world scenario in which strict network segmentation is enforced between the corporate LAN, the virtualization cluster, and external access points.

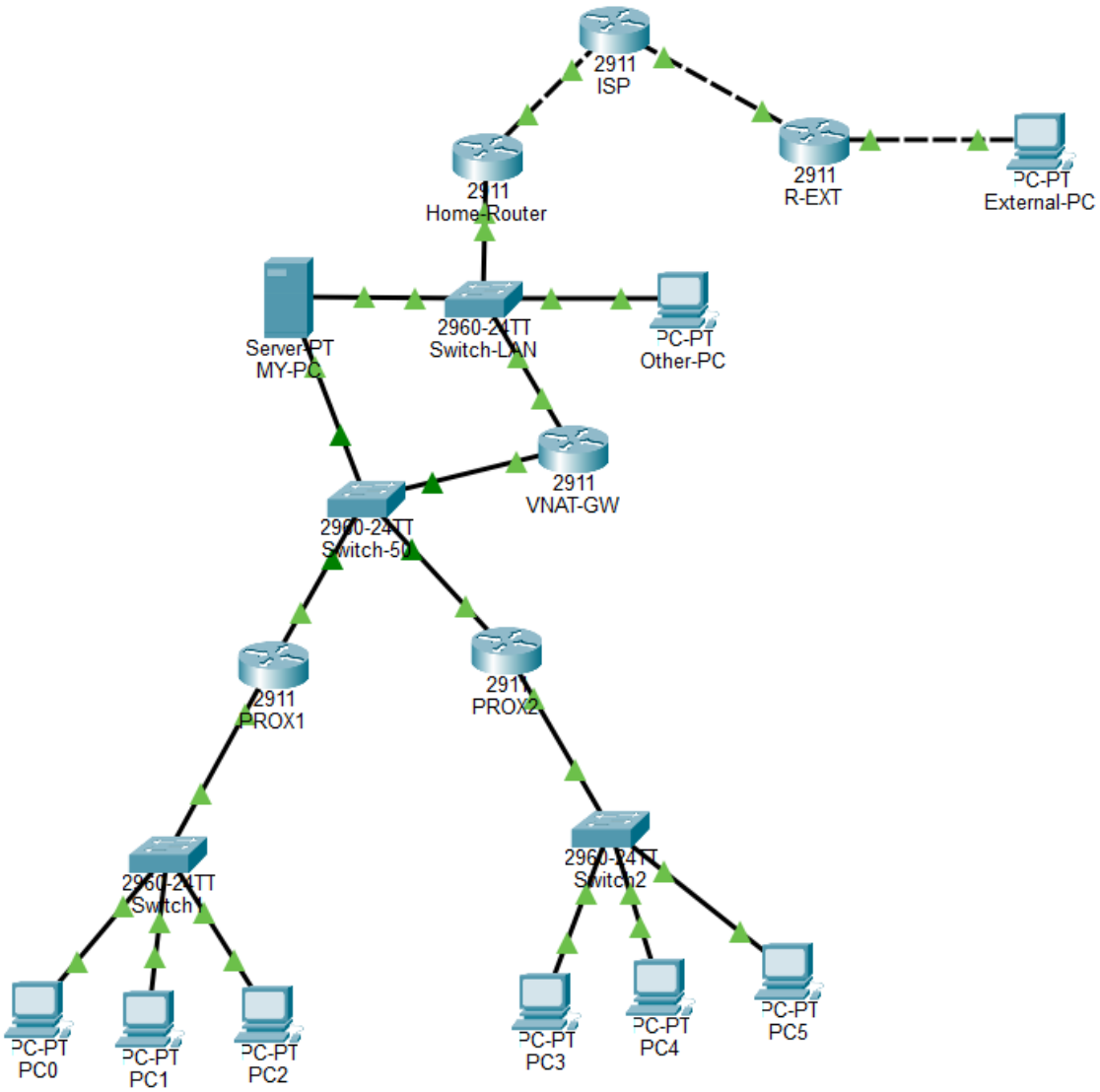


Figure 37: Simulated Network Topology - Cisco Packet Tracer

The topology is structured into four distinct network segments, each serving a specific security and operational role:

- **Home/Corporate LAN (192.168.1.0/24):** Represents the end-user environment. Only authorized devices within this segment are permitted to interact with the virtualization layer. Unauthorized devices such as general workstations are explicitly blocked from

accessing internal cluster networks through access control lists enforced at the gateway level.

- **VMware NAT Gateway (VNAT-GW):** Acts as the sole entry point between the corporate LAN and the Proxmox cluster network. This device enforces Network Address Translation (NAT) and routing policies, ensuring that the internal cluster addressing scheme (192.168.50.0/24) remains hidden from the broader network, consistent with the principle of least exposure.
- **Proxmox Cluster Network (192.168.50.0/24):** Hosts two Proxmox virtualization servers (PROX1 and PROX2), each managing an isolated VM subnet (192.168.51.0/24 and 192.168.52.0/24 respectively). This segment is entirely unreachable from the public internet and is only accessible through the VNAT-GW, enforcing strict east-west traffic control within the OT-adjacent environment.
- **External VPN Access:** Remote access is simulated through a dedicated external router (R-EXT) connected via an ISP segment. A VPN tunnel (10.10.10.0/30) is established between the external endpoint and the internal gateway, replicating the behavior of a zero-trust network access tool such as Netbird. This ensures that external clients never directly expose or access internal subnet addresses, and all traffic is encapsulated within the authenticated tunnel.

This segmented architecture directly addresses the security concerns outlined in the previous chapter. By enforcing layer-3 isolation between network zones, applying NAT to conceal internal addressing, and restricting external access exclusively through an encrypted VPN tunnel, the design adheres to the core principles of the Purdue model. No direct path exists between the public internet and the virtualization cluster, and lateral movement between network segments is controlled entirely through explicit routing policies and access control lists.

5 Hosting

Having defined the network architecture for laboratory access, this section describes how the platform itself is deployed on a production cloud infrastructure. IoT-REAP is deployed on OVH cloud infrastructure at `iotreap.novationcityapp.com`, providing production-grade hosting with enterprise-level reliability and secure HTTPS access through Let's Encrypt SSL/TLS certificates with automatic renewal [60, 61].

5.1 OVH Hosting Infrastructure

The production server is an OVH VPS configured with 4 virtual CPU cores, 8 GB RAM, and NVMe SSD storage. This configuration was selected to support concurrent VM provisioning requests, active WebSocket connections, and database query throughput simultaneously. The server runs Ubuntu 22.04 LTS for long-term stability. Nginx serves as the web server and reverse proxy with optimized Laravel settings including gzip compression, browser caching, and rate limiting. PHP-FPM handles execution using OPcache bytecode caching and optimized memory limits [62, 63, 44].

MySQL 8 operates with production-tuned configurations including InnoDB buffer pool optimization and appropriate indexing strategies. Automated daily backups provide disaster recovery capability and guarantee data integrity [34].

5.2 Domain Configuration and SSL Security

The domain `iotreap.novationcityapp.com` resolves to the OVH server via a DNS A record. Let's Encrypt certificates provide HTTPS encryption for all client-server communications, with Certbot handling automatic renewal before certificate expiry [61, 64].

Nginx is configured to enforce HTTPS redirection, automatically upgrading all HTTP requests to secure connections. Security headers including HSTS (HTTP Strict Transport Security), `X-Frame-Options`, and Content Security Policy (CSP) are applied at the reverse proxy level, protecting against common web vulnerabilities such as clickjacking and cross-site scripting (XSS) [63, 65, 66].

6 Tests and Verification

Verification of IoT-REAP combined black-box functional testing, automated coverage, security scanning, and performance baselining. Manual walkthroughs validated each role's journeys, while automated regression ran via GitHub Actions.

Functional correctness was validated across the complete user lifecycle: registration with email and OAuth, enrollment, progression, quizzes, and certificates. VM session provisioning was verified end-to-end, including template cloning, Guacamole tunnels, USB/IP hardware attachment, camera streams, and cleanup. Teacher authoring workflows were validated from drafts through review. All scenarios passed without regressions.

Role-based access control, enforced via Laravel Policies, was validated by cross-role route attempts. Administrators, teachers, and engineers were correctly restricted to authorized endpoints, confirmed manually and via PHPUnit.

Remote laboratory stability was verified through repeated execution of the full VM session lifecycle, including hardware state transitions and reconciliation resilience against gateway disconnection.

OWASP ZAP authenticated scans verified HTTPS, headers, cookie hardening, and absence of SQL injection and XSS. Stripe webhooks rejected invalid HMAC signatures. Automated coverage comprised PHPUnit backend and Vitest frontend tests, blocking deployment on failure.

In summary, IoT-REAP satisfies core requirements under nominal conditions with correct security enforcement. The principal limitation is the absence of large-scale load testing.

Conclusion

This chapter presented the technical realization of IoT-REAP, covering architecture, backend workflows, interfaces, network segmentation, deployment, and verification. The layered monolithic stack—Laravel, React, TypeScript, and MySQL—enforces separation of concerns across four layers and an infrastructure boundary. An asynchronous provisioning pipeline and state reconciliation worker coordinate distributed hardware, while backend workflows handle reservation conflicts, hardware state transitions, and session lifecycle management. Interfaces cover all user roles, and network topology simulation validated the segmentation strategy. Automated deployment via GitHub Actions CI/CD enforces test gates before release. Verification confirmed that engineers, teachers, and administrators can perform their functions, and security testing revealed no critical vulnerabilities. The absence of statistically rigorous load testing remains a noted limitation for future work.

General Conclusion

As part of this **four-month** internship at Novation City, IoT-REAP was designed and implemented, a secure industrial remote operations and training platform for Industry 4.0 learning. The platform unifies six functional domains-session management, VM provisioning, remote desktop access, camera supervision, IoT device control, and structured training management-within a role-differentiated interface serving Engineers, Teachers, and Administrators. The objective was to eliminate physical laboratory dependency by enabling users to launch sessions, interact with real equipment, follow guided learning paths, and monitor activity from a single web interface.

The development approach was iterative and architecture-driven, following Agile-inspired delivery cycles. The backend leverages Laravel organized around FormRequest validation, thin controllers, service-layer logic, and repository-based data access. The frontend uses React with TypeScript for typed API integrations and reusable components. Role-based access ensures each profile encounters only contextually relevant actions, forming a solid foundation for external system integration [23, 26, 27].

A major contribution was the remote lab session lifecycle. Proxmox integration enables on-demand VM provisioning and lifecycle control, while Guacamole-based browser access supports RDP, VNC, and SSH without local client installation. Tokenized access patterns, expiration-aware handling, and resource cleanup prevent orphaned VMs and maintain stability, bridging cloud orchestration and industrial training [2, 3].

A second contribution was integrating real-time hardware context into training sessions. Camera services and gateway integrations enable remote equipment observation, while PTZ control routes through structured, auditable MQTT command channels. USB device reservation workflows attach specific hardware to live sessions in a controlled manner, positioning IoT-REAP as an operational remote laboratory rather than a conventional e-learning portal [4, 67].

The platform supports instructor-driven training path management. Teachers define learning sequences linked to specific VM sessions, synchronizing pedagogical structure with practical execution. Learners progress through guided material while interacting with realistic industrial environments, developing operational competencies through direct engagement.

Delivering these capabilities required navigating significant engineering challenges. Coordinating heterogeneous technologies-Proxmox's asynchronous operations, state-sensitive remote desktop tunnels, MQTT topic discipline, and variable camera streaming-required isolating external provider logic in dedicated service classes, standardizing integration contracts, and improving observability through explicit logging. Preserving architectural quality

amid rapid feature expansion demanded disciplined coding standards and incremental API documentation to prevent business logic leakage across layers. Balancing delivery velocity with security requirements necessitated explicit authorization boundaries, input validation, role scoping, and regression tests on security-critical paths, producing a higher-assurance, auditable workflow.

Future extensions include completing high-impact API integrations and advancing frontend parity for administrative workflows. Deeper analytics on session usage, infrastructure efficiency, and learner progression would support data-driven decisions. Predictive scheduling and optimized resource allocation could reduce wait times, while enhanced zero-trust controls, compliance-oriented audit reporting, and mobile-optimized interfaces would broaden operational reach.

This internship produced tangible technical outcomes while developing competencies across full-stack architecture, distributed system integration, and security-aware design. IoT-REAP delivers a technically sound foundation connecting virtual infrastructure, remote hardware interaction, and structured learning. Core requirements were verified through unit and feature tests, confirming specification alignment. The system demonstrates the feasibility of secure, remote Industry 4.0 training at scale, with an architecture designed to accommodate a growing operational roadmap.

Webgraphy

[1] Novation City, "Novation City," *Novation City*, https://novationcity.com, Accessed: April 23, 2026.

[2] Proxmox Server Solutions GmbH, "Proxmox VE API," *Proxmox VE Wiki*, https://pve.proxmox.com/wiki/Proxmox_VE_API, Accessed: February 15, 2026.

[3] The Apache Software Foundation, "Apache Guacamole Manual," *Apache Guacamole*, https://guacamole.apache.org/doc/gug/, Accessed: February 4, 2026.

[4] OASIS Open, "MQTT Version 5.0," *OASIS Open*, https://www.oasis-open.org/standard/mqtt-v5-0/, Accessed: May 6, 2026.

[5] Carrillo, G.C. and others, "Adapting an Industrial Control Laboratory for Remote Access," *IFAC-PapersOnLine*, https://www.sciencedirect.com/science/article/pii/S2405896325005920, Accessed: February 2, 2026.

[6] da Silva, A. and Santos, A. and Pereira, F. and others, "Learning Automation from Remote Labs in Higher Education," *Eighth International Conference on Technological Ecosystems for Enhancing Multiculturality (TEEM 2020)*, https://dl.acm.org/doi/10.1145/3434780.3436689, Accessed: February 2, 2026.

[7] Cyolo, "Modernizing Secure Remote Access: Overcoming VPN Security Shortfalls," *Cyolo*, https://cyolo.io/blog/modernizing-secure-remote-access-overcoming-vpn-security-shortfalls, Accessed: February 6, 2026.

[8] Dragos, Inc., "2025 OT/ICS Cybersecurity Year in Review," *Dragos, Inc.*, https://www.dragos.com/resources/press-release/dragos-reports-ot-ics-cyber-threats-escalate, Accessed: February 6, 2026.

[9] BeyondTrust, "Operational Technology Security: Why Smarter OT Remote Access Matters," *BeyondTrust*, https://www.beyondtrust.com/blog/entry/ot-security-privileged-remote-access, Accessed: February 2, 2026.

[10] Secomea A/S, "The State of Industrial Remote Access 2026," *Secomea*, https://industrialcyber.co/reports/secomeas-state-of-industrial-remote-access-2026, Accessed: February 3, 2026.

[11] Forescout Vedere Labs, "3.4 Million RDP and VNC Servers Exposed," *Forescout Vedere Labs*, https://industrialcyber.co/industrial-cyber-attacks/forescout-finds-3-4-million-rdp-and-vnc-servers-exposed, Accessed: February 3, 2026.

[12] Cisco Systems, "Secure Remote Access for Industrial Operations Solution Brief," *Cisco*, https://www.cisco.com/c/en/us/products/collateral/security

/industrial-security/secure-remote-access-for-ot-sb.html, Accessed: February 2, 2026.

[13] Various, "A Cost-Effective Modular Laboratory Solution for Industrial Automation and Applied Engineering Education," *Various*, https://www.ncbi.nlm.nih.gov/pmc/articles/PMC12171526/, Accessed: February 6, 2026.

[14] Secomea A/S, "GateManager Access Management Server," *Secomea*, https://secomea.com/gatemanager-access-management-server/, Accessed: February 2, 2026.

[15] Secomea A/S, "Remote Access for Industrial Equipment – Security Certificates," *Secomea*, https://coevagi.com/wp-content/uploads/2024/01/Secomea-Remote-Access.pdf, Accessed: February 6, 2026.

[16] HMS Networks, "Industrial Remote Access," *HMS Networks*, https://www.hms-networks.com/industrial-remote-access, Accessed: February 6, 2026.

[17] HMS Networks, "eCatcher – Talk2M Remote Access VPN Client," *HMS Networks*, https://www.hms-networks.com/ecatcher, Accessed: February 5, 2026.

[18] HMS Networks, "Talk2M Pro," *HMS Networks*, https://www.hms-networks.com/talk2m/talk2m-pro, Accessed: February 5, 2026.

[19] HMS Networks, "Talk2M FAQ: About the Talk2M Light Plan," *HMS Networks*, https://support.hms-networks.com/hc/en-us/articles/18356216795282, Accessed: February 2, 2026.

[20] Siemens AG, "SINEMA Remote Connect – VPN Management Platform," *Siemens*, https://www.siemens.com/en-us/products/scalance/sinema-remote-connect/, Accessed: February 4, 2026.

[21] Siemens AG, "SINEMA Remote Connect Getting Started Guide," *Siemens*, https://manuals.plus/m/b94f635ab11e883db5a34df261b85e37cd8e70cedaa01d450a4fb87f7b527e56, Accessed: February 2, 2026.

[22] TeamViewer AG, "TeamViewer Debuts Agentless Access to Secure, Simplify Remote Operations in Industrial Environments," *TeamViewer*, https://industrialcyber.co/news/teamviewer-debuts-agentless-access-to-secure-simplify-remote-operations-in-industrial-environments/, Accessed: February 2, 2026.

[23] Laravel LLC, "Laravel 12.x Documentation," *Laravel*, https://laravel.com/docs/12.x, Accessed: March 4, 2026.

[24] Stripe, Inc., "Stripe Documentation," *Stripe Docs*, https://docs.stripe.com/, Accessed: February 18, 2026.

[25] Scott W. Ambler, "Agile Documentation," *Agile Modeling*, https://agilemodeling.com/essays/agiledocumentation.htm, Accessed: February 12, 2026.

[26] Meta Open Source, "React Documentation," *React*, https://react.dev/, Accessed: May 6, 2026.

[27] Microsoft Corporation, "TypeScript Documentation," *TypeScript*, https://www.typescriptlang.org/docs/, Accessed: February 14, 2026.

[28] pmndrs, "Zustand Documentation," *Zustand*, https://zustand.docs.pmnd.rs/, Accessed: April 28, 2026.

[29] RFC Editor, "RFC 8216: HTTP Live Streaming," *RFC Editor*, https://www.rfc-editor.org/rfc/rfc8216, Accessed: May 6, 2026.

[30] OWASP Foundation, "Zed Attack Proxy Documentation," *OWASP ZAP*, https://www.zaproxy.org/docs/, Accessed: May 26, 2026.

[31] Grady Booch, James Rumbaugh, and Ivar Jacobson, "The Unified Modeling Language User Guide, 2nd ed.," *Addison-Wesley*, https://personal.utdallas.edu/~chung/Fujitsu/UML_2.0/Rumbaugh--UML_2.0_Reference_CD.pdf, Accessed: April 11, 2026.

[32] Vitest, "Vitest Documentation," *Vitest*, https://vitest.dev/guide/, Accessed: March 16, 2026.

[33] Sebastian Bergmann, "PHPUnit Documentation," *PHPUnit*, https://docs.phpunit.de/, Accessed: May 9, 2026.

[34] Oracle Corporation, "MySQL 8.4 Reference Manual," *MySQL Documentation*, https://dev.mysql.com/doc/refman/8.4/en/, Accessed: April 17, 2026.

[35] Microsoft Corporation, "Visual Studio Code Documentation," *Visual Studio Code*, https://code.visualstudio.com/docs, Accessed: February 12, 2026.

[36] Software Freedom Conservancy, "Git Documentation," *Git*, https://git-scm.com/docs, Accessed: March 2, 2026.

[37] GitHub, Inc., "GitHub Docs," *GitHub Docs*, https://docs.github.com/en, Accessed: April 6, 2026.

[38] Nils Adermann and Jordi Boggiano, "Composer Documentation," *Composer*, https://getcomposer.org/doc/, Accessed: April 19, 2026.

[39] npm, Inc., "npm Docs," *npm Docs*, https://docs.npmjs.com/, Accessed: February 4, 2026.

[40] Postman, Inc., "Postman Docs," *Postman Learning Center*, https://learning.postman.com/, Accessed: April 13, 2026.

[41] JGraph Ltd., "diagrams.net," *diagrams.net*, https://www.diagrams.net/, Accessed: February 26, 2026.

[42] Cisco Systems, Inc., "Cisco Packet Tracer Documentation," *Cisco Networking Academy*, https://www.netacad.com/cisco-packet-tracer, Accessed: May 12, 2026.

[43] Broadcom Inc., "VMware Technical Documentation," *Broadcom TechDocs*, https://techdocs.broadcom.com/, Accessed: April 3, 2026.

[44] The PHP Group, "PHP Manual," *PHP Documentation*, https://www.php.net/docs.php, Accessed: May 3, 2026.

[45] Inertia.js, "Inertia.js Documentation," *Inertia.js*, https://inertiajs.com/, Accessed: April 25, 2026.

[46] Vite, "Vite Documentation," *Vite*, https://vite.dev/guide/, Accessed: April 17, 2026.

[47] Tailwind Labs, "Tailwind CSS Documentation," *Tailwind CSS*, https://tailwindcss.com/docs, Accessed: March 8, 2026.

[48] bluenviron, "MediaMTX," *GitHub*, https://github.com/bluenviron/mediamtx, Accessed: May 15, 2026.

[49] RFC Editor, "RFC 2326: Real Time Streaming Protocol," *RFC Editor*, https://www.rfc-editor.org/rfc/rfc2326, Accessed: February 21, 2026.

[50] World Wide Web Consortium, "WebRTC: Real-Time Communication in Browsers," *W3C Recommendation*, https://www.w3.org/TR/webrtc/, Accessed: May 1, 2026.

[51] Frigate NVR, "Frigate Documentation," *Frigate*, https://docs.frigate.video/, Accessed: March 27, 2026.

[52] FFmpeg, "FFmpeg Documentation," *FFmpeg*, https://ffmpeg.org/documentation.html, Accessed: March 16, 2026.

[53] Eclipse Foundation, "Eclipse Mosquitto Documentation," *Eclipse Mosquitto*, https://mosquitto.org/documentation/, Accessed: February 20, 2026.

[54] Laravel LLC, "Laravel Reverb Documentation," *Laravel*, https://laravel.com/docs/12.x/reverb, Accessed: March 1, 2026.

[55] Laravel LLC, "Laravel Fortify Documentation," *Laravel*, https://laravel.com/docs/12.x/fortify, Accessed: May 1, 2026.

[56] Laravel LLC, "Laravel Socialite Documentation," *Laravel*, https://laravel.com/docs/12.x/socialite, Accessed: April 11, 2026.

[57] Google, "Using OAuth 2.0 to Access Google APIs," *Google Identity*, https://developers.google.com/identity/protocols/oauth2, Accessed: March 26, 2026.

[58] TanStack, "TanStack Query Documentation," *TanStack*, https://tanstack.com/query/latest/docs/framework/react/overview, Accessed: February 3, 2026.

[59] Recharts Group, "Recharts Documentation," *Recharts*, https://recharts.org/en-US/, Accessed: May 9, 2026.

[60] OVHcloud, "VPS," *OVHcloud*, https://www.ovhcloud.com/en/vps/, Accessed: February 14, 2026.

[61] Internet Security Research Group, "Let's Encrypt Documentation," *Let's Encrypt*, https://letsencrypt.org/docs/, Accessed: February 12, 2026.

[62] Canonical Ltd., "Ubuntu Documentation," *Ubuntu Help*, https://help.ubuntu.com/, Accessed: March 21, 2026.

[63] F5, Inc., "nginx Documentation," *nginx*, https://nginx.org/en/docs/, Accessed: February 13, 2026.

[64] Electronic Frontier Foundation, "Certbot Documentation," *Certbot*, https://eff-certbot.readthedocs.io/en/stable/, Accessed: March 18, 2026.

[65] Mozilla, "Strict-Transport-Security," *MDN Web Docs*, https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Strict-Transport-Security, Accessed: May 20, 2026.

[66] Mozilla, "Content Security Policy (CSP)," *MDN Web Docs*, https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP, Accessed: March 17, 2026.

[67] The Linux Kernel Organization, "USB/IP Protocol," *Linux Kernel Documentation*, https://docs.kernel.org/usb/usbip_protocol.html, Accessed: February 28, 2026.

Abstract

IoT-REAP (Internet of Things - Remote Equipment Access Platform) is a secure web platform for industrial remote laboratory operations, developed as an end-of-study internship at Novation City, a Tunisian Industry 4.0 cluster. It addresses gaps in solutions - client installation dependency, absent VM lifecycle management, fragmented tooling, and no integrated training - within a browser-native interface.

Built on Laravel 12, React 18, TypeScript, and Inertia.js, it integrates Proxmox VE for VM provisioning, Apache Guacamole for clientless RDP/VNC/SSH access, and camera integration with camera feeds via WebRTC/HLS through MediaMTX. An e-learning module covers training paths, video lessons, quizzes, and certificates. Security relies on zero-trust, comprehensive audit logging, Purdue IT/OT isolation, and HMAC webhook validation; OWASP ZAP found no critical vulnerabilities. Deployment runs via GitHub Actions CI/CD to OVH.

Keywords: Industry 4.0, Remote Laboratory, Virtual Machine Provisioning, Apache Guacamole, MQTT, Zero-Trust Security, E-Learning

Résumé

IoT-REAP (Internet of Things - Plateforme d'Accès aux Équipements à Distance) est une plateforme web sécurisée pour les opérations industrielles à distance, développée lors d'un stage de fin de études à Novation City, cluster tunisien orienté Industrie 4.0. Elle répond aux lacunes des solutions existantes - dépendance à l'installation client, absence de gestion du cycle de vie des machines virtuelles, outils fragmentés et absence de formation intégrée - au sein d'une interface native navigateur.

Développée avec Laravel 12, React 18, TypeScript et Inertia.js, la plateforme intègre Proxmox VE pour le provisionnement de VM, Apache Guacamole pour l'accès bureau à distance sans client via RDP/VNC/SSH, et Mosquitto MQTT pour le contrôle des caméras. Les flux caméra transitent via WebRTC/HLS par MediaMTX. Un module e-learning propose des parcours de formation, des leçons vidéo, des quiz et des certificats vérifiables. La sécurité repose sur zero-trust, un audit complet, l'isolation IT/OT selon Purdue, et la validation HMAC des webhooks ; OWASP ZAP n'a révélé aucune vulnérabilité critique. Le déploiement est automatisé via GitHub Actions CI/CD vers un serveur OVH.

Mots-clés : Industrie 4.0, Laboratoire Distant, Provisionnement de Machines Virtuelles, Apache Guacamole, MQTT, Sécurité Zero-Trust, E-Learning